



天数智芯
Iluvatar CoreX

天数智芯

软件栈工具使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.9.10

适用产品：天垓 150| 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 修订记录	9
2 软件栈工具概览	13
3 系统管理工具 ixSMI 使用指南	14
3.1 功能概览	14
3.2 回显字段说明	17
3.2.1 执行 ixSMI 后回显的字段说明	17
3.2.2 执行 ixSMI -q 后回显的字段说明	18
3.2.3 执行 ixSMI -q -d HARDWARE_MONITOR 后回显的字段说明	20
3.2.4 执行 ixsmi dmon 后回显的字段说明	21
3.2.5 执行 ixsmi pmon 回显的字段说明	22
3.3 General options	22
3.3.1 ixsmi	22
3.3.2 -h, --help	22
3.4 Summary options	23
3.4.1 -L, --list-gpus	23
3.4.2 -l [SEC], --loop [SEC]	23
3.4.3 -lms [MS], --loop-ms [MS]	23
3.4.4 -f [FILE], --filename [FILE]	23
3.4.5 -i [IDENTIFIER], --id [IDENTIFIER]	23
3.5 Query options	23
3.5.1 -q, --query	23
3.5.2 -q -d MEMORY, --display MEMORY	23
3.5.3 -q -d UTILIZATION, --display UTILIZATION	24
3.5.4 -q -d ECC, --display ECC	24
3.5.5 -q -d TEMPERATURE, --display TEMPERATURE	24
3.5.6 -q -d POWER, --display POWER	24
3.5.7 -q -d CLOCK, --display CLOCK	24
3.5.8 -q -d COMPUTE, --display COMPUTE	24
3.5.9 -q -d PIDS, --display PIDS	25
3.5.10 -q -d PERFORMANCE, --display PERFORMANCE	25
3.5.11 -q -x	25
3.5.12 --dtd	25
3.5.13 --query-gpu	25
3.5.14 --query-compute-apps	25
3.5.15 stats	26
3.5.16 stats -i [IDENTIFIER], --id [IDENTIFIER]	26
3.5.17 stats -f [FILE], --filename [FILE]	26
3.5.18 stats -d [SUBOPTION], --display [SUBOPTION]	26
3.5.19 stats -c [NUM], --count [NUM]	26
3.6 Monitoring options	27
3.6.1 dmon	27
3.6.2 dmon -i [IDENTIFIER], --id [IDENTIFIER]	27
3.6.3 dmon -d [SEC], --delay [SEC]	27

3.6.4	dmon -c [NUM], --count [NUM]	27
3.6.5	dmon -o [D/T]	27
3.6.6	dmon -f [FILE]	27
3.6.7	daemon	28
3.6.8	daemon -i [IDENTIFIER], --id [IDENTIFIER]	28
3.6.9	daemon -d [SEC], --delay [SEC]	28
3.6.10	daemon -p [DIR], --path [DIR]	28
3.6.11	daemon -j, --join	28
3.6.12	daemon -t, --term	28
3.6.13	replay -i [IDENTIFIER], --id [IDENTIFIER]	29
3.6.14	replay -f [FILE], --filename [FILE]	29
3.6.15	replay -b [HH:MM:SS], --begin [HH:MM:SS]	29
3.6.16	replay -e [HH:MM:SS], --end [HH:MM:SS]	29
3.6.17	pmon	29
3.6.18	pmon -i [IDENTIFIER], --id [IDENTIFIER]	29
3.6.19	pmon -d [SEC], --delay [SEC]	30
3.6.20	pmon -c [NUM], --count [NUM]	30
3.6.21	pmon -o [D/T]	30
3.6.22	pmon -f [FILE], --filename [FILE]	30
3.7	Device modification options	30
3.7.1	-r, --gpu-reset	30
3.7.2	--pl, --power-limit [POWER VALUE] (for BI-V150S)	30
3.8	Topology options	31
3.8.1	topo -m, --matrix	31
3.8.2	topo -mp, --matrix_pci	31
3.8.3	topo -t, --tree	31
3.8.4	topo -R, --RoCE	31
3.8.5	topo -B, --infiniBand	31
3.8.6	topo -c [CPU IDX], --cpu [CPU IDX]	31
3.8.7	topo -i [GPU IDX] -n [TYPE]	32
3.8.8	topo -p -i [GPU_A_IDX,GPU_B_IDX]	32
3.8.9	topo --p2p [r/w], --p2pstatus [r/w]	32
3.9	GRID vGPU	32
3.9.1	vgpu	32
3.9.2	vgpu -i, --id TEXT	33
3.9.3	vgpu -q, --query	33
3.9.4	vgpu -u, --utilization	33
3.9.5	vgpu -p, --pmon	33
3.9.6	vgpu -s, --supported	33
3.9.7	vgpu -c, --creatable	33
3.9.8	vgpu -v, --verbose	34
3.9.9	vgpu -l, --loop	34
3.10	常见问题	34

4 Profiler 工具 ixPROF 使用指南 35

4.1	ixPROF 使用方法	35
-----	-------------	----

4.2	ixPROF 功能列表	35
4.2.1	-h	36
4.2.2	-V, --version	36
4.2.3	[cuda-app] [app-args]	37
4.2.4	--print-api-summary [cuda-app] [app-args]	37
4.2.5	--track-memory-allocations TEXT:{off,on} [cuda-app] [app-args]	37
4.2.6	--print-gpu-summary [cuda-app] [app-args]	37
4.2.7	--print-api-trace [cuda-app] [app-args]	37
4.2.8	--print-gpu-trace [cuda-app] [app-args]	37
4.2.9	--demangling TEXT:{off,on} [cuda-app] [app-args]	38
4.2.10	--profile-child-processes [cuda-app] [app-args]	38
4.2.11	--profile-from-start TEXT:{off,on}	38
4.2.12	--query-events	38
4.2.13	--query-metrics	38
4.2.14	-f, --force-overwrite	39
4.2.15	-e, --events TEXT [cuda-app] [app-args]	39
4.2.16	-m, --metrics TEXT [cuda-app] [app-args]	39
4.2.17	-o, --export-profile TEXT [cuda-app] [app-args]	39
4.2.18	-i, --import-profile TEXT [cuda-app] [app-args]	39
4.2.19	--kernels TEXT [cuda-app] [app-args]	39
4.2.20	--query-events --devices TEXT [OR] -e all --devices TEXT [cuda-app] [app-args]	40
4.2.21	--devices TEXT [cuda-app] [app-args]	40
4.2.22	--replay-mode TEXT:{application, kernel}	40
4.2.23	-c, --launch-count TEXT [cuda-app] [app-args]	40
4.2.24	-s, --launch-skip TEXT [cuda-app] [app-args]	40
4.2.25	--target-processes-filter TEXT [cuda-app] [app-args]	41
4.2.26	-t, --timeout TEXT [cuda-app] [app-args]	41
5	系统性能分析工具 ixSYS 使用指南	42
5.1	ixSYS 工具概述	42
5.2	1. 使用 ixSYS 命令行工具抓取 trace	42
5.2.1	系统要求	42
5.2.2	开始之前	42
5.2.2.1	在 Linux 环境上准备	43
5.2.2.2	在 Docker 环境/云环境上准备	43
5.2.3	抓取 trace 信息	44
5.2.4	ixSYS 命令行工具功能列表	46
5.2.5	使用命令行强制结束 trace 抓取	49
5.2.6	支持抓取的 API List	49
5.2.6.1	NVTX 函数列表	49
5.2.6.2	cuDNN 函数列表	50
5.2.6.3	OSRT 函数列表	52
5.2.6.4	JPU 相关函数列表	57
5.2.6.5	VPU 相关函数列表	58
5.2.6.6	cuINFER 函数列表	58
5.2.7	常见问题	61

5.3	2. 使用 ixSYS 图形界面工具分析 trace	63
5.3.1	打开 ixSYS 图形界面工具	63
5.3.2	导入 trace 文件	64
5.3.2.1	导入 trace 文件 (小于 400MB)	64
5.3.2.2	导入大 trace 文件 (大于 400MB)	64
5.3.3	查看 trace 信息	69
5.3.3.1	选择区域	69
5.3.3.2	选择单个 slice	74
5.3.3.3	跳转至 correlation slice	75
5.3.3.4	置顶 track	75
5.3.3.5	展开或折叠 track 和 group	76
5.3.3.6	显示/隐藏无 kernel stream	76
5.3.3.7	查看 callstack trace	77
5.3.3.8	分析 callstack 数据	77
5.3.3.9	置顶栏命令菜单	79
5.3.3.10	Viz 功能	81
5.3.3.11	Event Sync 功能	82
5.3.3.12	ixSYS 命令生成器	82
5.3.4	常见问题	82
5.4	3. ixSYS 工具最佳实践	85
5.4.1	GPU Profiling	85
5.4.2	CPU Profiling	89
5.4.2.1	OSRT	89
5.4.2.2	callstack	90
5.4.3	管理 ixTX 上报的数据	90
5.4.3.1	ixTX 提供的注释类型	90
5.4.3.2	ixTX Domain 介绍	91
5.4.3.3	ixTX 使用示例	91
5.4.3.4	使用 ixSYS 抓取 ixTX 上报的数据并分析	104
5.4.4	GPU metrics	106
5.4.4.1	使用说明	106
5.4.4.2	示例	107
5.4.5	Event Sync	108
5.4.5.1	使用说明	108
5.4.5.2	示例	108
6	核函数性能分析工具 ixKN 使用指南	110
6.1	ixKN 工具概述	110
6.2	1. 使用 ixKN CLI 工具抓取数据	110
6.2.1	使用方法	110
6.2.2	功能列表	112
6.3	2. 使用 ixKN-UI 工具分析性能	116
6.3.1	在 Windows 环境上安装 ixKN-UI 工具	116
6.3.2	在 Linux 环境上安装 ixKN-UI 工具	118
6.3.3	使用 ixKN-UI 工具	120

7 代码调试工具 ixGDB 使用指南	130
7.1 ixGDB 使用方法	130
7.2 ixGDB 功能列表	131
7.3 常见问题	132
8 加速卡管理工具集 ixDCGM 使用指南	133
8.1 ixDCGM 工具集概述	133
8.2 安装 ixDCGM	133
8.3 ixDCGM 组件概述	134
8.3.1 用户空间库 libixdcmg.so	134
8.3.1.1 支持的 API 列表	134
8.3.1.2 支持的 Field 列表	137
8.3.2 主机侧服务端代理软件 ix-hostengine	139
8.3.2.1 ix-hostengine 功能列表	139
8.3.2.2 ix-hostengine 使用示例	140
8.3.3 验证工具集 Iluvatar Validation Suite (ixvs) 及其测试插件	141
8.3.3.1 ixvs 功能列表	141
8.3.3.2 ixvs 使用示例	142
8.3.4 主机侧客户端命令行控制程序 ixdcmgi	143
8.3.4.1 ixdcmgi diag	143
8.3.4.2 ixdcmgi discovery	145
8.3.4.3 ixdcmgi group	146
8.3.4.4 ixdcmgi fieldgroup	148
8.3.4.5 ixdcmgi health	150
8.3.4.6 ixdcmgi dmon	151
8.3.4.7 ixdcmgi topo	152
8.3.4.8 ixdcmgi profile	153
8.3.4.9 ixdcmgi policy	154
8.3.4.10 ixdcmgi introspect	155
8.3.4.11 ixdcmgi set	156
9 Log 收集工具 ixLogCollect 使用指南	158
9.1 环境要求	158
9.2 安装指南	158
9.3 功能说明	158
9.4 使用说明	158
9.5 系统 Log 收集范围及如何自定义	159
9.5.1 默认系统 Log 收集范围	159
9.5.2 修改系统 Log 收集范围	159
9.6 UMD 相关 Log 收集范围及如何自定义	159
9.6.1 默认 UMD 相关 Log 收集范围	159
9.6.2 修改 UMD 相关 Log 收集范围	160
9.7 KMD 相关 Log 收集范围	160
9.8 使用说明	160
10 内存检测工具 ixSAN 使用指南	162
10.1 ixSAN 概述	162

10.2 ixSAN 使用方法	162
10.2.1 命令行选项	162
10.2.2 编译选项	162
10.2.3 环境变量	163
10.3 memcheck 工具使用指南	163
10.3.1 memcheck 支持的功能	164
10.3.2 memcheck 使用方法	164
10.3.3 memcheck 错误报告详述	164
10.3.3.1 内存泄漏	164
10.3.3.2 内存访问错误数量提示	165
10.3.3.3 内存访问错误	165
10.4 racecheck 工具使用指南	166
10.4.1 racecheck 支持的功能	166
10.4.2 racecheck 使用方法	166
10.4.3 racecheck 错误报告详述	166
10.4.3.1 analysis 模式	166
10.4.3.2 hazard 模式	167
10.5 initcheck 工具使用指南	167
10.5.1 initcheck 支持的功能	167
10.5.2 initcheck 使用方法	168
10.5.3 initcheck 错误报告详述	168
10.6 ixSAN 使用技巧	168
11 二进制文件分析工具 ixobjdump 使用指南	169
11.1 ixobjdump 概述	169
11.2 ixobjdump 使用方法	169
11.3 ixobjdump 命令选项	171
11.4 伪汇编指令集	172

1 修订记录

- COREX01-BI430-UG04-01: 2025/9/10
 - 更新 ixSYS 工具用户指南，细化 ixSYS 命令行工具功能列表
- COREX01-BI430-UG04-00: 2025/6/30
 - ixSMI 工具
 - * 新增“回显字段说明”
 - * 新增以下命令功能：
 - **-lms [MS], --loop-ms [MS]**
 - 支持查询 vGPU 相关信息 **GRID vGPU**
 - * 更新 **-i** 参数描述
 - ixML 库
 - * 删除 nvmlDeviceGetPowerManagementLimitConstraints 接口
 - * 提供 ixML API 的参数和功能详情，具体可参阅《软件栈 API 参考》
 - ixSYS 工具
 - * 更新 **-t, --trace** 的默认抓取数据，新增抓取 temperature 数据
 - * 新增在 Apple arm 平台上使用 trace_processor_shell 辅助工具
 - ixKN 工具，新增如下命令行功能：
 - * **--page TEXT**
 - * **--print-source TEXT**
 - * **--resolve-source-file**
 - * **--import-source TEXT**
 - ixDCGM 工具
 - * 新增以下 API group 支持：
 - Process Statistics
 - Job Statistics
 - Health Monitor
 - Topology
 - Metadata
 - Modules
 - Profiling
 - * 更新 System group API
 - * 更新 Fields group API
 - * 更新支持的 Field 列表
 - 新增“Log 收集工具 ixLogCollect 使用指南”
 - 新增“内存检测工具 ixSAN 使用指南”
 - 新增“二进制文件分析工具 ixobjdump 使用指南”
- COREX01-BI420-UG04-03: 2025/5/8
 - ixSYS 工具
 - * 更新 **-t, --trace** 的命令描述
 - * 新增“使用命令行强制结束 trace 抓取”方法

- * "支持抓取的 API List" 新增"cuINFER 函数列表"
- * 新增常见问题 5
- COREX01-BI420-UG04-03: 2025/4/17
 - 删除"Log 收集工具 ixogcollect 使用指南", 该工具暂未正式发布。如有需要, 请联系天数智芯应用工程师
- COREX01-BI420-UG04-02: 2025/3/19
 - ixSYS 工具, 更新在 Docker 环境/云环境上使用 ixSYS 需要开启特权模式的描述
- COREX01-BI420-UG04-01: 2025/3/12
 - ixDCGM 工具, ixdcgmi 命令行工具新增"ixdcgmi diag" 子系统命令
- COREX01-BI420-UG04-00: 2025/1/24
 - ixSYS 工具
 - * "抓取 trace 信息" 新增以下使用场景:
 - 抓取 IB 网卡流量
 - 抓取代码中使用 cudaEventRecord 和 cudaStreamWaitEvent 同步的同步点信息
 - * 更新 -t, --trace 命令参数描述
 - * 新增以下功能:
 - --nic-metrics
 - --event-sync
 - * "2. 使用 ixSYS 图形界面工具分析 trace" 中的"查看 trace 信息" 小节, 新增如下功能:
 - 置顶栏命令菜单
 - Viz 功能
 - Event Sync 功能
 - ixSYS 命令生成器
 - * 新增常见问题 4 及对应解决方法
 - * "ixSYS 工具最佳实践", 新增"Event Sync" 场景说明
 - ixGDB 工具, 新增以下命令参数:
 - * **info locals**
 - * **print <variable name>**
 - ixDCGM 工具
 - * libixdcgm.so 软件库
 - 新增拓扑查询, 健康监测等主要功能
 - 新增部分 DCGM API, 定义兼容主流生态
 - 新增部分 Fields 列表, 定义兼容主流生态
 - * ix-hostengine 主机侧服务端软件, 新增监听 Unix Socket 的功能
 - * ixvs 验证工具集及插件, 新增 targeted_stress 插件和 pluse 插件
 - * 新增提供 ixdcgmi 命令行工具
 - 新增 Log 收集工具 ixlogcollect 工具使用指南
- COREX01-BI411SP1-UG04-01: 2024/12/27
 - ixGDB 工具, 新增"常见问题" 小节

- COREX01-BI411SP1-UG04-00: 2024/11/28
 - ixSMI 工具, 更新 **-q -d TEMPERATURE, --display TEMPERATURE** 命令参数回显, 新增天垓 150 OAM 加速卡型号支持显示 VCORE 电源温度
- COREX01-BI411-UG04-01: 2024/9/6
 - ixSMI 工具
 - * 删除 PAGE_RETIREMENT 相关信息, PAGE_RETIREMENT 已不再支持
 - ixSYS 工具
 - * **--trace** 命令参数新增 trace 选项
 - * " 导入大 trace 文件" 中, "SSH 通过堡垒机/跳转机连接远程机器" 补充 Linux 环境下编译安装 Nginx 的步骤说明
 - * " 常见问题" 新增" 问题 6. 在使用 trace_processor_shell 工具时, 用户区域选中 slice 时报错"
- COREX01-BI411-UG04-00: 2024/8/26
 - ixSMI 工具
 - * 删除 **topo -E, --ethernet** 命令参数
 - * 新增 **topo -R, --RoCE** 命令参数
 - ixKN 工具, 更新 ixKN-UI 工具安装包的获取路径, 提示用户可在 [天数智芯官网](#) 下载该安装包
 - 新增 ixDCGM 工具
- COREX01-BI410-UG04-00: 2024/6/30
 - ixSMI 工具
 - * 新增以下命令参数:
 - **--pl, --power-limit [POWER VALUE]**
 - **topo -E, --ethernet**
 - **topo -B, --infiniBand**
 - * 更新以下命令示例:
 - **-q, --query**
 - **-q -d ECC, --display ECC**
 - ixML 工具, 新增以下 API:
 - * `nvmIDeviceGetPcieThroughput`
 - ixPROF 工具, 新增以下命令参数:
 - * **-t, --timeout TEXT [cuda-app] [app-args]**
 - ixSYS 工具
 - * 新增以下命令参数:
 - **ixsys --devices 0,1 -o output.pttrace ./my_app**
 - **--devices**
 - * 更新"2. 使用 ixSYS 图形界面工具分析 trace" 小节
 - ixKN 工具, 新增以下命令参数:
 - * **--kill {on, off}**
- COREX01-BI340-UG04-02: 2024/4/16

- ixSYS 工具，“导入大 trace 文件”小节新增“场景二在远程 Linux 服务器使用 trace_processor_shell 导入大 trace 文件”的操作指导

- COREX01-BI340-UG04-01：2024/4/8

- ixSYS 工具，删除默认抓取 OSRT 相关信息
- ixKN 工具，新增以下命令参数：
 - * **--replay-mode TEXT: {application,kernel}**
 - * **--target-processes-filter TEXT**

- COREX01-BI340-UG04-00：2024/3/25

文档本次发布内容与 V3.3.0-BI150 文档相比 (COREX01-BI330-UG04-01)，有以下更新：

- ixSMI 工具，新增以下命令参数：
 - * **topo --t, --tree**
- ixML 工具，新增以下 API：
 - * **nvmIDeviceGetMultiGpuBoard**
- ixPROF 工具，新增以下命令参数：
 - * **-s,--launch-skipTEXT [cuda-app] [app-args]**
 - * **--target-processes-filter TEXT [cuda-app] [app-args]**
- ixSYS 工具
 - * 使用 ixSYS“抓取 trace 信息”，删除默认抓取 OSRT 相关信息，当前版本已默认不抓取 OSRT
 - * 新增“按照代码中指定的代码段抓取 cuda 数据”功能
 - * 新增 **--profile-from-start TEXT:{off,on}** 命令参数
 - * 更新“2. 使用 ixSYS 图形界面工具分析 trace”，删除 ixSYS 抓取 Host 侧 Memory 数据，当前版本仅抓取 GPU 侧 Memory 数据
 - * 更新“3. ixSYS 工具最佳实践 > GPU Profiling”小节，提供 **--profile-from-start TEXT:{off,on}** 命令参数的使用详情
- ixKN 工具，新增以下命令参数：
 - * **-s,--launch-skip TEXT [cuda-app] [app-args]**
 - * **--profile-from-start TEXT{off, on}**
 - * **--devices TEXT**
 - * **--disable-profiler-start-stop**
 - * **--filter-mode TEXT:{global,per-gpu}**

2 软件栈工具概览

安装完天数智算软件栈后您可以使用以下工具：

工具	描述
系统管理工具	ixML 库提供了监控和管理天数智芯加速卡多种状态的 API。 ixSMI 工具提供系统管理信息，您可使用该工具对天数智芯加速卡进行监控。
Profiler 工具	ixPROF 工具收集和查看追踪数据，您可使用该工具了解天数智芯加速卡运行状态，并进行系统性能分析。
性能分析工具	ixKN 工具在 CUDA 核函数 (kernel) 执行期间，抓取性能分析指标的数据，相关性能数据可做可视化展示。 ixSYS 工具在 CUDA 应用程序运行时对系统进行采样抓取 trace，trace 信息可用 web 图形化方式统计及展示。
代码调试工具	ixGDB 工具是针对天数智芯加速卡中 CUDA 应用程序的开源调试工具。
加速卡管理工具集	ixDCGM 工具集用于在集群中管理和监控天数智芯数据中心的加速卡。
Log 收集工具	ixLogCollect 工具用于帮助用户一键收集系统 Log 以及 UMD/KMD 相关 Log，方便排查问题。
内存检测工具集	ixSAN 工具集用于检查 CUDA 应用程序功能正确性，且包含多种检测工具。
二进制文件分析工具	ixobjdump 工具用于查看可执行文件中或库文件中包含的 elf 和 IR 文件，并执行查看和提取等相应的操作。

3 系统管理工具 ixSMI 使用指南

3.1 功能概览

命令参数	命令描述及示例
General options	
N/A	显示连接到系统的 GPU 摘要信息
-h, --help	打印工具的帮助信息
Summary options	
-L, --list-gpus	显示连接到系统的 GPU
-l [SEC], --loop [SEC]	按指定的时间间隔 (以秒为单位) 显示 GPU 信息, 直到按下 Ctrl-C [默认 =5 秒]
-lms [MS], --loop-ms [MS]	按指定的时间间隔 (以毫秒为单位) 显示 GPU 信息, 直到按下 Ctrl-C [默认 =100 毫秒]
-f [FILE], --filename [FILE]	将查询输出重定向到指定文件
-i [IDENTIFIER], --id [IDENTIFIER]	指定要操作的 GPU, 参数可以是 GPU 的 index, PCI BUS ID, 或 UUID
Query options	
-q, --query	显示 GPU 详细信息
-d [SUBOPTION], --display [SUBOPTION]	仅显示指定的信息, [SUBOPTION] 可选字段有: MEMORY, UTILIZATION, ECC, TEMPERATURE, POWER, CLOCK, COMPUTE, PIDS, PERFORMANCE。 可以用逗号组合多个字段, 例如"MEMORY,ECC"。 对于 POWER、UTILIZATION 和 CLOCK 信息, 会返回 max、min 和 avg 值。 注: --display 参数必须和--query 参数组合使用
-x, --xml-format	以 XML 格式输出查询结果, 也可以和--dtd 命令组合使用
--dtd	--dtd 与-x 一起使用, 在 XML 输出中嵌入 DTD。也可以单独使用, 仅输出 DTD
--query-gpu [SUBOPTION]	查询 GPU 的信息, 以逗号分隔要查询的属性列表, 例如, --query-gpu=pci.bus_id,pci.device_id --format=csv, 调用--help-query-gpu 获取更多信息
--query-compute-apps [SUBOPTION]	查询当前活动的计算进程列表, 以逗号分隔要查询的属性列表, 例如, --query-compute-apps=pid,name --format=csv, 调用--help-query-compute-apps 获取更多信息

命令参数	命令描述及示例
--format TEXT	以逗号分隔的格式选项列表，该参数和 --query-gpu, --query-compute-apps 参数组合使用，选项列表如下： csv - 逗号分隔值（强制） noheader - 不打印首行列标题 nounits - 不打印数值单位
stats	以 CSV 格式生成 GPU 统计数据，包括功耗、利用率、温度、内存、Encoder、Decoder、时钟、热限制的持续时间和时钟降频事件，可通过 ixsmi stats -h 查看更多详细信息
stats -i [IDENTIFIER], --id [IDENTIFIER]	查询指定 GPU 的 stats 信息
stats -f [FILE], --filename [FILE]	将查询输出重定向到指定文件
stats -d [SUBOPTION], --display [SUBOPTION]	仅显示指定的信息，[SUBOPTION] 可选字段有：pwrDraw, temp, memUtil, gpuUtil, encUtil, decUtil, memClk, procClk, violPwr, violThm
stats -c [NUM], --count [NUM]	循环显示指定数量的样本
Monitoring options	
dmon	以滚动方式显示每次取样时的 device 状态，每行显示一次取样，显示信息包括功率、温度、GPU 占用率、内存占用率、VPU 编码利用率、VPU 解码利用率、GPU 时钟频率和内存时钟频率，通过 ixsmi dmon -h 获取更多帮助信息
dmon -i [IDENTIFIER], --id [IDENTIFIER]	查询指定 GPU 的 dmon 信息
dmon -d [SEC], --delay [SEC]	设置每次显示的间隔时间（单位：秒），缺省值是 1，取值范围 [1, 600]
dmon -c [NUM], --count [NUM]	循环显示指定数量的样本
dmon -o [D/T], --options [D/T]	以下一项或多项： D - 在滚动输出中包含日期 (YYYYMMDD) T - 在滚动输出中包含时间 (HH:MM:SS)
dmon -f [FILE], --filename [FILE]	将查询输出重定向到指定文件
daemon	在后台运行并且作为守护进程监视设备，需要 root 权限执行，通过 ixsmi daemon -h 获取更多帮助信息
daemon -i [IDENTIFIER], --id [IDENTIFIER]	监控指定的 GPU
daemon -d [SEC], --delay [SEC]	设置每次记录的时间间隔（单位：秒），缺省值是 10，取值范围 [1, 600]
daemon -p [DIR], --path [DIR]	设置 daemon 记录数据的路径，默认是 /var/log/ixsmi

命令参数	命令描述及示例
daemon -j, --join	添加到 log 文件名后的字符串
daemon -t, --term	终止 daemon 守护进程
replay	展示守护进程 daemon 生成的统计数据，不带参数输出 help 信息，可通过 ixsmi replay -h 获取更多帮助信息
replay -i [IDENTIFIER], --id [IDENTIFIER]	查询指定 GPU daemon 生成的信息
replay -f [FILE], --filename [FILE]	指定查询日志文件的完整路径，replay 命令必须加上此参数
replay -b [HH:MM:SS], --begin [HH:MM:SS]	指定查询的开始时间，默认是 0，表示从文件的开始查询
replay -e [HH:MM:SS], --end [HH:MM:SS]	指定查询的结束时间，默认是 0，表示到文件的末尾结束
pmon	以滚动方式显示每次取样时的进程状态，每行显示一次取样，显示信息包括 pid、GPU 占用率、内存占用率、VPU 编码利用率、VPU 解码利用率和进程名称
pmon -i [IDENTIFIER], --id [IDENTIFIER]	查询指定 GPU 的 pmon 信息
pmon -d [SEC], --delay [SEC]	设置每次显示的间隔时间（单位：秒），缺省值是 1，取值范围 [1, 10]
pmon -c [NUM], --count [NUM]	循环显示指定数量的样本
pmon -o [D/T], --options [D/T]	以下一项或多项： D - 在滚动输出中包含日期 (YYYYMMDD) D - 在滚动输出中包含时间 (HH:MM:SS)
pmon -f [FILE], --filename [FILE]	将查询输出重定向到指定文件
Device modification options	
-r, --gpu-reset	重置 GPU，可以作为重启 GPU 硬件的替代方法 注： 1. 使用该命令需要 root 权限，并确保 GPU 处于闲置状态。 2. 该命令执行过程中被 Ctrl+c 终止，可能造成 GPU 丢失。
--pl, --power-limit [POWER VALUE]	指定最大功耗管理限值 (以瓦特为单位)，只支持设置 350W 或 450W，并且 只有在天垓 150S 产品中生效
Topology	
topo -m, --matrix	显示系统的 GPU to GPU 的拓扑链接类型矩阵
topo -mp, --matrix_pci	显示系统的 GPU to GPU 的拓扑链接类型矩阵 (PCI only)
topo -t, --tree	显示 GPU 到 CPU 的树状 topo 结构
topo -R, --RoCE	在拓扑结构中显示 RoCE 网卡，必须与 -t 参数结合使用

命令参数	命令描述及示例
topo -B, --infiniBand	在拓扑结构中显示 IB 网卡，必须与-t 参数结合使用
topo -c [CPU IDX], --cpu [CPU IDX]	显示与指定 CPU 相关联的 GPU
topo -i [GPU IDX] -n [TYPE]	显示给定遍历路径类型最近的 GPU
topo -p -i [GPU_A_IDX,GPU_B_IDX]	显示一对 GPU 的最直接路径遍历
topo --p2p [r/w], --p2pstatus [r/w]	显示给定 GPU 之间的 p2p 状态： r - p2p 读取能力 w - p2p 写入能力
GRID vGPU	
vgpu	显示 GPU 上 vGPU 的基础信息
vgpu -i,--id TEXT	查询指定 GPU 的 vGPU 信息
vgpu -q,--query	显示当前正在运行的 vGPU 的详细信息
vgpu -u,--utilization	显示当前 GPU 上活跃 vGPU 的资源使用情况
vgpu -p,--pmon	显示当前 GPU 上活跃的 vGPU 在 VM 中运行进程的资源使用情况
vgpu -s,--supported	显示支持的 vGPU 类型
vgpu -c,--creatable	显示当前可以创建的 vGPU 类型
vgpu -v,--verbose	显示可支持的或可创建的 vGPU 类型的详细信息，需要和 -c 和 -s 参数一起使用
vgpu -l,--loop	以指定的时间间隔 (以秒为单位) 显示信息，直到按下 Ctrl-C [默认值 =5 秒]

3.2 回显字段说明

3.2.1 执行 ixSMI 后回显的字段说明

字段	说明
IX-ML	ixSMI 的版本
Driver Version	驱动版本
GPU	GPU 编号
Name	GPU 名称
Fan	风扇转速比
Temp	GPU 芯片温度

字段	说明
Perf	GPU 性能状态。固定为 P0，表示最大性能，暂不支持修改
Pwr:Usage/Cap	功率和峰值功率
Bus-Id	PCIe 设备的 Bus ID
Memory-Usage	GPU 内存使用情况和总内存大小
Clock-SM	GPU 时钟频率
Clock-Mem	DDR 时钟频率
GPU-Util	GPU 算力的利用率
Compute M.	GPU 计算模式。默认 default，暂不支持修改
Processes GPU	GPU 编号。当 GPU 上有进程执行时显示该字段
Processes PID	GPU 上执行的进程 PID
Process name	GPU 上执行的进程名称
GPU Memory Usage (MiB)	GPU 上执行的进程占用的内存大小

3.2.2 执行 ixSMI -q 后回显的字段说明

字段	说明
Timestamp	查询信息时的时间戳
Driver Version	驱动的版本
Attached GPU device count	获取到的 GPU 数量
Product Name	GPU 名称
Serial Number	GPU 的 SN
GPU UUID	通用唯一识别码
Minor Number	设备的次要编号：dev/iluvatar[次要编号]
MultiGPU Board	是否为多 GPU 板卡
Board ID	板卡 ID，具有相同 ID 的 GPU 表示连接到同一个 PCIe Switch
GPU Position	GPU 在板卡上的位置
Board Part Number	板卡 PN 编码
GPU Virtualization Mode	GPU 虚拟化信息
Virtualization Mode	虚拟化类型

字段	说明
Host VGPU Mode	主机虚拟化模式
PCI	PCI 信息
Bus	PCIe 设备的 Bus Number
Device	PCIe 设备的 Device ID
Domain	PCIe 设备的 Domain ID
Device Id	PCIe 设备的厂商识别码和设备识别码
Bus Id	PCIe 设备的 Bus ID
Sub System Id	PCIe 设备的子厂商识别码和子系统识别码
GPU Link Info	PCIe 设备的连接信息
PCIe Generation Max	GPU 通信的最大 Generation
PCIe Generation Current	GPU 通信的当前 Generation
Link Width Max	GPU 通信的最大带宽
Link Width Current	GPU 通信的当前带宽
Tx Throughput	GPU 给 CPU 发送数据的吞吐率
Rx Throughput	GPU 从 CPU 接收数据的吞吐率
Fan Speed	风扇转速比
Memory Usage	内存使用情况
Total	GPU 总内存
Used	已经使用的 GPU 内存
Free	空闲的 GPU 内存
Utilization	利用率
Gpu	GPU 算力利用率
Memory	内存利用率
Encoder	VPU 编码利用率
Decoder	VPU 解码利用率
Ecc Mode	ECC 模式
Current	当前 ECC 模式
Pending	重启或者重新上电后的 ECC 模式
Single Bit ECC	单 bit ECC 错误统计，出错时为 1，无法统计实际错误数量

字段	说明
Double Bit ECC	双 bit ECC 统计错误，出错时为 1，暂无法统计实际错误数量
Temperature	温度信息
GPU Current Temp	当前芯片温度
GPU Shutdown Temp	关机的阈值温度
GPU Slowdown Temp	降频的阈值温度
GPU Max Operating Temp	可以运行的最高温度
Memory Current Temp	当前 GPU 显存温度
Power Readings	功率信息
GPU Power Draw	当前 GPU 功率
Current GPU Power Limit	当前 GPU 最大功率
Default GPU Power Limit	默认 GPU 最大功率
Board Power Draw	当前板卡功率
Current Board Power Limit	当前板卡最大功率
Default Board Power Limit	默认板卡最大功率
Clocks SM	当前 GPU 时钟频率
Clocks Memory	当前 DDR 频率
Max Clocks SM	最大 GPU 时钟频率
Max Clocks Memory	最大 DDR 频率
Compute Mode	计算模式
Processes	GPU 上运行的进程信息
Process ID	当前运行的进程 PID
Type	进程类型
Name	进程名称
Used GPU Memory	进程占用的 GPU 内存大小
Performance State	GPU 性能状态，固定为 P0，表示最大性能，不支持修改

3.2.3 执行 ixSMI -q -d HARDWARE_MONITOR 后回显的字段说明

字段	说明
Timestamp	查询信息时的时间戳
Driver Version	驱动的版本
Attached GPU device count	获取到的 GPU 数量
Hardware Monitor	硬件信息
Device ID	设备识别码
Vendor ID	厂商识别码
Sub Device ID	子系统识别码
Sub Vendor ID	子厂商识别码
Hardware Version	硬件版本
Firmware Version	固件版本
VCC12 Voltage	PCIe 设备 12 伏电压
GPU Voltage	GPU 工作电压
Memory Voltage	Memory 工作电压
GPU SN	GPU 的序列号
Board PN	板卡 PN 编码
Board SN	板卡序列号
Board Temperature	板卡温度
Date of Manufacture	板卡出厂日期
CE MASK	当前 CE Mask
CE GOLDEN MASK	最大可以设置的 CE Mask

3.2.4 执行 ixsmi dmon 后回显的字段说明

字段	说明
gpu Idx	GPU 索引编号
pwr	GPU 当前功耗
gtemp	GPU 的芯片温度
mtemp	GPU 的内存温度
sm	GPU 算力利用率

字段	说明
mem	内存利用率
enc	VPU 编码利用率
dec	VPU 解码利用率
mclk	DDR 频率
pclk	GPU 时钟频率

3.2.5 执行 ixsmi pmon 回显的字段说明

字段	说明
gpu Idx	GPU 的索引编号
pid	进程 PID
type	进程类型，C/G 分别表示计算和图形
sm	进程 GPU 利用率的平均值
mem	进程 Memory 利用率
enc	进程 VPU 编码利用率的平均值
dec	进程 VPU 解码利用率的平均值
command name	进程名称

3.3 General options

3.3.1 ixsmi

显示连接到系统的 GPU 摘要信息。

3.3.2 -h, --help

打印工具的帮助信息。

```
$ ixsmi -h
```

3.4 Summary options

3.4.1 -L, --list-gpus

显示连接到系统的 GPU。

```
$ ixsmi -L
```

3.4.2 -l [SEC], --loop [SEC]

按指定的时间间隔 (以秒为单位) 显示 GPU 信息，直到按下 Ctrl-C [默认 =5 秒]。

3.4.3 -lms [MS], --loop-ms [MS]

按指定的时间间隔 (以毫秒为单位) 显示 GPU 信息，直到按下 Ctrl-C [默认 =100 毫秒]。

3.4.4 -f [FILE], --filename [FILE]

将查询输出重定向到指定文件。

3.4.5 -i [IDENTIFIER], --id [IDENTIFIER]

指定要操作的 GPU，参数可以是 GPU 的 index，PCI BUS ID，或 UUID。

3.5 Query options

3.5.1 -q, --query

显示 GPU 详细信息。

```
$ ixsmi -q
```

3.5.2 -q -d MEMORY, --display MEMORY

显示当前 GPU 的内存使用情况。

```
$ ixsmi -q -d MEMORY
```

3.5.3 -q -d UTILIZATION, --display UTILIZATION

显示当前 GPU 的利用情况。

```
$ ixsmi -q -d UTILIZATION
```

3.5.4 -q -d ECC, --display ECC

显示当前 GPU 的 ECC 状态。

```
$ ixsmi -q -d ECC
```

3.5.5 -q -d TEMPERATURE, --display TEMPERATURE

显示当前 GPU 的温度和 GPU 温度极限。

如果是天垓 150 OAM 加速卡，会增加显示 VCORE 的电源温度。

```
$ ixsmi -q -d TEMPERATURE
```

3.5.6 -q -d POWER, --display POWER

显示当前 GPU 和板卡的功率以及功率极限。

```
$ ixsmi -q -d POWER
```

3.5.7 -q -d CLOCK, --display CLOCK

显示当前 GPU 的时钟频率。

```
$ ixsmi -q -d CLOCK
```

3.5.8 -q -d COMPUTE, --display COMPUTE

显示当前 GPU 的计算模式。

```
$ ixsmi -q -d COMPUTE
```

3.5.9 -q -d PIDS, --display PIDS

显示运行在当前 GPU 上的进程。

```
$ ixsmi -q -d PIDS
```

3.5.10 -q -d PERFORMANCE, --display PERFORMANCE

显示当前 GPU 的性能状态。

```
$ ixsmi -q -d PERFORMANCE
```

3.5.11 -q -x

以 XML 格式输出查询结果。

```
$ ixsmi -q -x
```

3.5.12 --dtd

输出 XML 配置文件 DTD。

```
$ ixsmi --dtd
```

3.5.13 --query-gpu

查询有关 GPU 的信息。

```
$ ixsmi --query-gpu=pci.bus_id,pci.device_id,temperature.gpu --format=csv,nounits
```

3.5.14 --query-compute-apps

查询当前活动的计算进程列表。

```
$ ixsmi --query-compute-apps=pid,name --format=csv
```

3.5.15 stats

以 CSV 格式生成 GPU 统计数据，包括功耗、利用率、温度、内存、Encoder、Decoder、时钟、热限制的持续时间和时钟降频事件。

```
$ ixsmi stats
```

3.5.16 stats -i [IDENTIFIER], --id [IDENTIFIER]

查询指定 GPU 的 stats 信息。

```
$ ixsmi stats -i 1
```

3.5.17 stats -f [FILE], --filename [FILE]

将查询输出重定向到指定文件。

```
$ ixsmi stats -f test.txt
```

3.5.18 stats -d [SUBOPTION], --display [SUBOPTION]

仅显示指定的信息，[SUBOPTION] 可选字段有：pwrDraw, temp, memUtil, gpuUtil, encUtil, decUtil, memClk, procClk, violPwr, violThm。

```
$ ixsmi stats -d pwrDraw
```

[SUBOPTION] 组合使用：

```
$ ixsmi stats -d pwrDraw,memUtil
```

3.5.19 stats -c [NUM], --count [NUM]

循环显示指定数量的样本。

```
$ ixsmi stats -c 1
```

3.6 Monitoring options

3.6.1 dmon

以滚动方式显示每次取样时的 device 状态，每行显示一次取样，显示信息包括功率、温度、GPU 占用率、内存占用率、VPU 编码利用率、VPU 解码利用率、GPU 时钟频率和内存时钟频率。

```
$ ixsmi dmon
```

3.6.2 dmon -i [IDENTIFIER], --id [IDENTIFIER]

查询指定 GPU 的 dmon 信息。

```
$ ixsmi dmon -i 0
```

3.6.3 dmon -d [SEC], --delay [SEC]

设置每次显示的间隔时间（单位：秒），缺省值是 1，取值范围 [1, 600]。

```
$ ixsmi dmon -d 5 -o T -i 0
```

3.6.4 dmon -c [NUM], --count [NUM]

循环显示指定数量的样本。

```
$ ixsmi dmon -c 2
```

3.6.5 dmon -o [D/T]

在滚动输出中包含日期/时间。

```
$ ixsmi dmon -o DT
```

3.6.6 dmon -f [FILE]

将查询输出重定向到指定文件。

```
$ ixsmi dmon -o DT -c 3 -f test.log
```

3.6.7 daemon

在后台运作并且作为守护进程监视设备，需要 root 权限执行，执行后没有任何输出，通过 ps 命令可以看到执行的守护进程。

```
$ ixsmi daemon
```

3.6.8 daemon -i [IDENTIFIER], --id [IDENTIFIER]

监控指定的 GPU。

```
$ ixsmi daemon -i 0
```

3.6.9 daemon -d [SEC], --delay [SEC]

设置每次记录的间隔时间 (单位：秒)，缺省值是 10，取值范围 [1, 600]。

```
$ ixsmi daemon -d 1
```

3.6.10 daemon -p [DIR], --path [DIR]

设置 daemon 记录数据的路径，默认是 /var/log/ixsmi。

```
$ ixsmi daemon -p ./
```

3.6.11 daemon -j, --join

添加到 log 文件名后的字符串。

```
$ ixsmi daemon -j test-join.log # 在日志默认目录/var/log/ixsmi/下，存在  
↪ dmon-{YYYY-MM-DD}-test-join.log 文件
```

3.6.12 daemon -t, --term

终止 daemon 守护进程，终止后通过 ps 命令查看，守护进程消失。

```
$ ixsmi daemon -t
```

3.6.13 replay -i [IDENTIFIER], --id [IDENTIFIER]

查询指定 GPU daemon 生成的信息。

```
$ ixsmi replay -i 0 -f /var/log/ixsmi/dmon-2023-03-22
```

3.6.14 replay -f [FILE], --filename [FILE]

指定查询日志文件的完整路径，replay 命令必须加上此参数。

```
$ ixsmi replay -f /var/log/ixsmi/dmon-2023-03-22
```

3.6.15 replay -b [HH:MM:SS], --begin [HH:MM:SS]

指定查询的开始时间，默认是 0，表示从文件的开始查询。

```
$ ixsmi replay -i 0 -f /var/log/ixsmi/dmon-2023-03-22 -b 18:00:00
```

3.6.16 replay -e [HH:MM:SS], --end [HH:MM:SS]

指定查询的结束时间，默认是 0，表示到文件的末尾结束。

```
$ ixsmi replay -i 0 -f /var/log/ixsmi/dmon-2023-03-22 -e 17:56:00
```

3.6.17 pmon

以滚动方式显示每次取样时的进程状态，每行显示一次取样，显示信息包括 pid、GPU 占用率、内存占用率、VPU 编码利用率、VPU 解码利用率和进程名称。

```
$ ixsmi pmon
```

3.6.18 pmon -i [IDENTIFIER], --id [IDENTIFIER]

查询指定 GPU 的 pmon 信息。

```
$ ixsmi pmon -i 0
```

3.6.19 pmon -d [SEC], --delay [SEC]

设置每次显示的间隔时间（单位：秒），缺省值是 1，取值范围 [1, 10]

```
$ ixsmi pmon -d 5 -i 0
```

3.6.20 pmon -c [NUM], --count [NUM]

循环显示指定数量的样本。

```
$ ixsmi pmon -c 2 -i 0
```

3.6.21 pmon -o [D/T]

在滚动输出中包含日期/时间。

```
$ ixsmi pmon -o DT
```

3.6.22 pmon -f [FILE], --filename [FILE]

将查询输出重定向到指定文件。

```
$ ixsmi pmon -c 2 -f test.txt
```

3.7 Device modification options

3.7.1 -r, --gpu-reset

重置 GPU，可以作为重启 GPU 硬件的替代方法。

注：

1. 使用该命令需要 root 权限，并确保 GPU 处于闲置状态。
2. 该命令执行过程中被 Ctrl+C 终止，可能造成 GPU 丢失。

```
$ ixsmi -r
```

3.7.2 --pl, --power-limit [POWER VALUE] (for BI-V150S)

指定最大功耗管理限值 (以瓦特为单位)，只支持设置 350W 或 450W，并且只在天核 150S 产品中生效。

```
$ ixsmi --pl 450
```

3.8 Topology options

3.8.1 topo -m, --matrix

显示系统的 GPU to GPU 的拓扑链接类型矩阵。

```
$ ixsmi topo -m
```

3.8.2 topo -mp, --matrix_pci

显示系统的 GPU to GPU 的拓扑链接类型矩阵 (PCI only)。

```
$ ixsmi topo --mp
```

3.8.3 topo -t, --tree

显示 GPU 到 CPU 的树状 topo 结构。

```
$ ixsmi topo -t
```

3.8.4 topo -R, --RoCE

在拓扑结构中显示 RoCE 网卡，必须与-t 参数结合使用。

```
$ ixsmi topo -t -R
```

3.8.5 topo -B, --infiniBand

在拓扑结构中显示 IB 网卡，必须与-t 参数结合使用。

```
ixsmi topo -t -B
```

3.8.6 topo -c [CPU IDX], --cpu [CPU IDX]

显示与指定 CPU 相关联的 GPU。

```
$ ixsmi topo -c 0
```

3.8.7 topo -i [GPU IDX] -n [TYPE]

显示给定遍历路径类型最近的 GPU。具体类型如下：

- 0 = a single PCIe switch on a dual GPU board
- 1 = a single PCIe switch
- 2 = multiple PCIe switches
- 3 = a PCIe host bridge
- 4 = an on-CPU interconnect link between PCIe host bridges
- 5 = an SMP interconnect link between NUMA nodes

```
$ ixsmi topo -i 0 -n 1
```

3.8.8 topo -p -i [GPU_A_IDX,GPU_B_IDX]

显示一对 GPU 的最直接路径遍历。

```
$ ixsmi topo -i 0,1 -p
```

3.8.9 topo --p2p [r/w], --p2pstatus [r/w]

显示给定 GPU 之间的 p2p 状态。

```
$ ixsmi topo --p2p r
```

3.9 GRID vGPU

3.9.1 vgpu

显示 GPU 上 vGPU 的基础信息。

```
$ ixsmi vgpu
```

- 当 GPU 不支持或驱动不支持 vGPU 时，提示 Not supported devices in vGPU mode。

3.9.2 vgpu -i,--id TEXT

查询指定 GPU 的 vGPU 信息。

```
$ ixsmi vgpu -i 0  
$ ixsmi vgpu -q -i 0
```

3.9.3 vgpu -q,--query

显示当前正在运行的 vGPU 的详细信息。

```
$ ixsmi vgpu -q
```

3.9.4 vgpu -u,--utilization

显示当前 GPU 上活跃 vGPU 的资源使用情况。

```
$ ixsmi vgpu -u
```

3.9.5 vgpu -p,--pmon

显示当前 GPU 上活跃的 vGPU 在 VM 中运行进程的资源使用情况。

```
$ ixsmi vgpu -p
```

3.9.6 vgpu -s,--supported

显示支持的 vGPU 类型。

```
$ ixsmi vgpu -s
```

3.9.7 vgpu -c,--creatable

显示当前可以创建的 vGPU 类型。

```
$ ixsmi vgpu -c
```

3.9.8 vgpu -v,--verbose

显示可支持的或可创建的 vGPU 类型的详细信息，需要和 **-c** 或 **-s** 参数一起使用。

```
$ ixsmi vgpu -s -v
```

3.9.9 vgpu -l,--loop

以指定的时间间隔 (以秒为单位) 显示信息，直到按下 Ctrl-C [默认值 =5 秒]，可以和参数 **-q**, **-c**, **-s**, **-v**, **-i** 等组合使用。

```
$ ixsmi vgpu -i 0 -l 3
$ ixsmi vgpu -q -i 0 -l
```

3.10 常见问题

1. 如何在 Docker 容器内查看进程对 GPU 的使用情况？

在启动 Docker 容器时加上 **--pid=host** 选项，则您可以在 Docker 容器内使用 ixSMI 工具查看进程信息。具体请参考《软件栈安装指南》。**--pid=host** 的作用是让容器的进程标识符 (PID) 与宿主机的 PID 相同，则可以查看到该容器的进程对 GPU 的使用情况。

4 Profiler 工具 ixPROF 使用指南

4.1 ixPROF 使用方法

ixPROF 工具可以收集和查看追踪数据。

您在命令行输入 **ixprof** 和所需命令选项，即可使用该工具了解天数智芯加速卡运行状态，并进行系统性能分析。

4.2 ixPROF 功能列表

该工具提供以下功能：

命令参数	命令描述和示例
-h, --help	打印工具的帮助信息
-V, --version	显示工具的版本信息
[cuda-app] [app-args]	显示 GPU 上的活动摘要和 API 调用的摘要信息
--print-api-summary [cuda-app] [app-args]	显示 API 调用的摘要信息
--print-gpu-summary [cuda-app] [app-args]	显示 GPU 上的活动摘要信息
--print-api-trace [cuda-app] [app-args]	打印 CUDA 执行和驱动的 API trace
--track-memory-allocations TEXT:{off,on} [cuda-app] [app-args]	显示 GPU 上 memory 的生存时期、地址、类型等信息
--print-gpu-trace [cuda-app] [app-args]	打印单个内核调用（包括 CUDA memcpy / memset），并按时间顺序对其进行排序
--demangling TEXT:{off,on} [cuda-app] [app-args]	在打印 kernel 调用时的名字时，可以选择 ixprof 输出优化后的 demangling 名字或原始的 symbol 名字
--profile-child-processes [cuda-app] [app-args]	对 CUDA 程序的子进程进行分析
--profile-from-start TEXT:{off,on}	选择是否从应用启动时进行 profiling，默认值为从启动时开始（ON）。设置为 OFF 时，可使用 {cu,cuda} Profiler{Start,Stop} 开启或停止 profiling
--query-events	列出当前设备所支持的所有 events
--query-metrics	列出当前设备所支持的所有 metrics
--quiet	不输出任何 profiling 的信息
-f, --force-overwrite	强制修改之前的输出文件
-e, --events TEXT [cuda-app] [app-args]	在特定的设备上收集 CUDA 应用中的特定的 events，使用参数 all 收集该设备支持的所有 events

命令参数	命令描述和示例
<code>-m, --metrics TEXT [cuda-app] [app-args]</code>	在特定的设备上收集 CUD A 应用中的特定的指标 (metrics)，使用参数 all 收集该设备支持的所有指标
<code>-o, --export-profile TEXT [cuda-app] [app-args]</code>	导出运行信息结果到特定的文件，用于稍后导入或者使用其他工具打开
<code>-i, --import-profile TEXT [cuda-app] [app-args]</code>	从文件导入之前的运行信息结果并输出
<code>--kernels TEXT [cuda-app] [app-args]</code>	在设备上收集 CUDA 应用中的 events 或者 metrics 时，只对特定的 kernel 进行数据进行收集
<code>--devices TEXT [cuda-app] [app-args]</code>	更改后续“--events”、“--metrics”、“--query-events”和“--query-metrics”执行的设备
<code>-c, --launch-count TEXT [cuda-app] [app-args]</code>	在收集 events 或者 metrics 时，限定需要分析的 kernel 数量，如果使用过滤器，则表示满足过滤器的 kernel 数量
<code>-s, --launch-skip TEXT [cuda-app] [app-args]</code>	在收集 events 或者 metrics 时，限定需要跳过分析的 kernel 数量，当“--launch-skip”和“--launch-count”同时使用时，优先跳过满足数量的 kernel，再对 kernel 进行计数
<code>--replay-mode TEXT: {application,kernel}</code>	如果需要进行 replay，该参数指定 replay 的模式为 application 或 kernel，默认是 kernel replay 模式。如果和“--profile-child-processes”同时使用，需要分析的程序每次执行的 kernel 是确定性的
<code>--target-processes-filter TEXT [cuda-app] [app-args]</code>	限定要分析的程序的名称，只对限定范围内的进程进行分析，未匹配上的进程不会进行分析，此功能默认支持正则表达式
<code>-t, --timeout TEXT [cuda-app] [app-args]</code>	限定要分析的程序执行时间。当时间计数完成后，会 kill 进程不再进行 profile

4.2.1 -h

打印工具的帮助信息。

```
$ ixprof -h
```

4.2.2 -V, --version

显示工具的版本信息。

```
$ ixprof -V
```

4.2.3 [cuda-app] [app-args]

显示 GPU 上的活动摘要和 API 调用的摘要信息。

```
$ ixprof ./vectorAdd
```

4.2.4 --print-api-summary [cuda-app] [app-args]

显示 API 调用的摘要信息。

```
$ ixprof --print-api-summary ./vectorAdd
```

4.2.5 --track-memory-allocations TEXT:{off,on} [cuda-app] [app-args]

显示 GPU 上 memory 的生存时期、地址、类型等信息。

```
$ ixprof --track-memory-allocations on ./vecadd
```

4.2.6 --print-gpu-summary [cuda-app] [app-args]

显示 GPU 上的活动摘要信息。

```
$ ixprof --print-gpu-summary ./vectorAdd
```

4.2.7 --print-api-trace [cuda-app] [app-args]

打印 CUDA 执行和驱动的 API trace。

```
$ ixprof --print-gpu-summary ./vectorAdd
```

4.2.8 --print-gpu-trace [cuda-app] [app-args]

打印单个内核调用（包括 CUDA memcpy / memset），并按时间顺序对其进行排序。

```
$ ixprof --print-gpu-trace ./vectorAdd
```

4.2.9 --demangling TEXT:{off,on} [cuda-app] [app-args]

在打印 kernel 调用时的名字时，可以选择 ixPROF 输出优化后的 demangling 名字或原始的 symbol 名字，默认选项为 demangling on。

```
$ ixprof --demangling on ./vectorAdd
$ ixprof --demangling off ./vectorAdd
```

4.2.10 --profile-child-processes [cuda-app] [app-args]

对 CUDA 程序的子进程进行分析。

```
$ ixprof --profile-child-processes bash resnet18_test.sh
```

4.2.11 --profile-from-start TEXT:{off,on}

选择是否从应用启动时进行 profiling，默认值为从启动时开始（ON）。设置为 OFF 时，可使用 {cu,cuda} Profiler{Start,Stop} 开启或停止 profiling。

1. 修改 device_scan 程序源码，在代码中使用 cudaProfilerStart() 和 cudaProfilerStop() 标记代码片段并编译，具体咨询天数应用工程师
2. 执行 **ixprof --profile-from-start off ./device_scan**

4.2.12 --query-events

列出当前设备所支持的所有 events。

```
$ ixprof --query-events
```

4.2.13 --query-metrics

列出当前设备所支持的所有 metrics。

```
$ ixprof --query-metrics
```

4.2.14 -f, --force-overwrite

强制修改之前的输出文件。

```
$ ixprof -f -o prof.log --print-gpu-summary ./vectorAdd
```

4.2.15 -e, --events TEXT [cuda-app] [app-args]

在特定的设备上收集 CUDA 应用中的特定的 events，使用参数 all 收集该设备支持的所有 events。

```
$ ixprof -e mem_load,mem_store,mem_atomic ./vectorAdd
```

4.2.16 -m, --metrics TEXT [cuda-app] [app-args]

在特定的设备上收集 CUDA 应用中的特定的指标（metrics），使用参数 all 收集该设备支持的所有指标。

```
$ ixprof -m inst_executed,inst_control,inst_compute_ld_st ./vectorAdd
```

4.2.17 -o, --export-profile TEXT [cuda-app] [app-args]

导出运行信息结果到特定的文件，用于稍后导入或者使用其他工具打开。

```
$ ixprof -o prof.log --print-gpu-summary ./vectorAdd
```

4.2.18 -i, --import-profile TEXT [cuda-app] [app-args]

从文件导入之前的运行信息结果并输出。

```
$ ixprof -i prof.log
```

4.2.19 --kernels TEXT [cuda-app] [app-args]

在设备上收集 CUDA 应用中的 events 或者 metrics 时，只对特定的 kernel 进行数据进行收集。

```
$ ixprof -e all --kernels vect ./vectorAdd
```

4.2.20 --query-events --devices TEXT [OR] -e all --devices TEXT [cuda-app] [app-args]

--query-events --devices TEXT 来指定需要执行查询 events 或者 metrics 的设备，或者 **-e [EVENT] -m [METRIC] --devices TEXT** 来指定 CUDA 设备进行收集 events 或者 metrics。默认是参数 all，在所有的设备上执行。

用法分别如以下两个 Sample 所示：

```
$ ixprof --devices all --query-events
```

4.2.21 --devices TEXT [cuda-app] [app-args]

更改后续"--events"、"--metrics"、"--query-events" 和"--query-metrics" 执行的设备。

```
$ ixprof --devices 0 ./vectorAdd
```

4.2.22 --replay-mode TEXT:{application, kernel}

选择 events、metrics 收集的 replay 方式。默认方式为 kernel，可选择为 application，多次重复执行 application 来获取所需要的 events 和 metrics 数据。

```
$ ixprof -m ipc --replay-mode application ./vecadd
```

4.2.23 -c,--launch-count TEXT [cuda-app] [app-args]

在收集 events 或者 metrics 时，限定需要分析的 kernel 的数量。如果使用过滤器功能，则限定满足过滤器要求的 kernel 数量。

Note

该功能分析多进程程序时，每个子进程都会被单独限定需要分析的 kernel 的数量。

```
$ ixprof -e mem_load -c 2 ./vectorAdd
```

4.2.24 -s,--launch-skip TEXT [cuda-app] [app-args]

在收集 events 或者 metrics 时，限定需要分析的 kernel 的数量。如果使用过滤器功能，则限定满足过滤器要求的 kernel 数量。

Note

该功能分析多进程程序时，每个子进程都会被单独限定需要跳过的 kernel 的数量。

```
$ ixprof -e mem_load -s 1 ./vectorAdd
```

4.2.25 --target-processes-filter TEXT [cuda-app] [app-args]

限定 ixPROF 进行分析的程序名称。

```
$ ixprof --target-processes-filter Add --profile-child-processes bash test.sh
```

4.2.26 -t,--timeout TEXT [cuda-app] [app-args]

限定需要分析的进程执行时间。当时间计数完成后，ixPROF 会 kill 正在分析的子进程，并展示已收集到的结果。

注：时间计数开始时刻是程序执行到 cuInit 初始化后。

```
$ ixprof -t 2 ./sdk/ixtools/ixkn/build/bin/vectorAdd
```

5 系统性能分析工具 ixSYS 使用指南

5.1 ixSYS 工具概述

Iluvatar System Analyzer 工具简称为 ixSYS, 是对运行在天数智芯加速卡上的 CUDA 应用程序进行 CUDA API, NVTX, cuDNN, OSRT, raw_syscalls, video, memory, power 的 trace 的性能分析, 帮助您对相关的应用程序进行优化。

ixSYS 工具由命令行工具和图形界面工具组成:

组件	描述
ixSYS 命令行工具	对于系统进行采样抓取 trace。 目前支持 x86 架构, 支持 Ubuntu 和 CentOS 操作系统。
ixSYS 图形界面工具	用 web 的图形化的方式将 ixSYS 命令行工具抓取的 trace 信息进行统计及展示。 目前支持 Linux 和 Windows 操作系统。

说明:

- ixSYS 工具抓取 trace 如果超过 2GB, 在文件加载和分析时会耗费较多时间。
- ixSYS 工具目前不支持超过 3GB 的 trace 文件。

5.2 1. 使用 ixSYS 命令行工具抓取 trace

5.2.1 系统要求

目前支持 x86 架构, 在天数智算软件栈安装要求基础上, 支持以下操作系统和 Linux kernel:

- Ubuntu:
 - 支持 Linux kernel 版本 4.15.0-20-generic 到 5.8.0-63-generic
 - 不支持 Linux kernel 版本 4.15.0-167-generic
- CentOS:
 - 支持 Linux kernel 版本 4.18.0-80.el8.x86_64 及以上
 - 不支持 Linux kernel 版本 3.x

5.2.2 开始之前

天数智算软件栈安装完成后, ixSYS 命令行工具也会一并安装好。在使用 ixSYS 命令行工具之前, 请确保已经设置好 PATH 和 LD_LIBRARY_PATH 两个环境变量。您可以通过如下命令确认 ixSYS 命令行工具是否正确安装:

```
$ ixsys --version
```

如果回显 ixSYS 的版本号，则表示安装正确。

ixSYS 支持在 Linux 环境、Docker 环境和云环境上使用。因为 ixSYS 部分功能，比如 OSRT，函数库 trace 依赖于内核的 ftrace 相关功能，在使用 ixSYS 抓取 trace 之前，需要确认系统是否已经正确挂载了 debugfs 和 tracefs，以及当前用户有访问 debugfs 的权限，且默认的 ftrace 处在关闭的状态。

callstack 的功能抓取需要 root 权限，如果您需要抓取 callstack 的信息，请确保已有 root 权限且环境变量已配置正确。

5.2.2.1 在 Linux 环境上准备

具体操作如下：

1. 执行以下命令挂载 debugfs 和 tracefs：

```
$ sudo mount -t debugfs nodev /sys/kernel/debug
$ sudo mount -t tracefs nodev /sys/kernel/tracing
```

2. 执行以下命令为当前用户获取 ftrace 的相关控制权限：

```
$ sudo chown -R "$USER" /sys/kernel/
```

3. 执行以下命令查询当前系统的 ftrace 是否被占用：

```
$ cat /sys/kernel/tracing/tracing_on
```

- 如果返回为 0，则 ftrace 处于关闭状态，环境设置完成。
- 如返回为 1，确认不存在使用冲突的情况下，可以使用以下命令来关闭 ftrace：

```
$ echo 0 > /sys/kernel/debug/tracing/tracing_on
```

5.2.2.2 在 Docker 环境/云环境上准备

ixSYS 在 Docker 环境下的使用方法和独立主机基本一致，区别在于 ftrace 的功能获取。首先，您需要确保宿主机可以开启相应功能，挂载相应的文件目录，以便在容器内使用。

云环境一般认为是 Docker 启动的容器，只是配置和启动容器过程相对复杂，涉及的软件和服务也更多，用户可能无法直接进行配置，需要运维人员的协助。因此，您也可以参考如下信息准备环境。

在创建容器时，

- 需要开启特权模式，即使用 **--privileged** 参数使能 ftrace 功能后方可使用 ixSYS。
- 需要加上 **--pid=host** 参数，使得容器可以访问宿主机的 /proc 文件系统，从而允许 ftrace 捕获宿主机的数据，即在使用 ixSYS 时可以抓取 osrt 数据。命令示例如下：

```
$ sudo docker run -it -d -v /dev:/dev -v /home:/home -v /lib/modules:/lib/modules -v
↪ /usr/src:/usr/src -v /store:/store --pid=host --privileged --cap-add=ALL
↪ --network=host --name xxx 192.168.1.1:80/sw_test/
↪ apps:latest-centos-7.8.2003-x86_64-10.2-python3.6 /bin/bash
```

Note

需要注意的是，在 Docker 环境或云环境下，如果宿主机的内核版本不高于 3.10，可能会遇到功能受限的情况。

5.2.3 抓取 trace 信息

您可参考如下使用示例：

使用场景	示例命令	说明
抓取 cuda 可执行程序日志	<code>ixsys -o matrixMul.pttrace ./matrixMul</code>	使用默认 trace 参数，即 cuda，nvtx，抓取 matrixMul 应用程序执行的相关日志，并将抓取到的日志保存在指定的名为 matrixMul.pttrace 文件之中。
	<code>ixsys -o matrixMul_cuda_nvtx.pttrace -t cuda,nvtx ./matrixMul</code>	使用指定的 trace 参数，当前指定为 cuda，nvtx，抓取 matrixMul 应用程序执行的日志，并将抓取到的日志保存在指定的名为 matrixMul_cuda_nvtx.pttrace 文件之中。
抓取 shell 脚本日志	<code>ixsys -o alexnet.pttrace bash run_alexnet_inference.sh</code>	使用默认 trace 参数，即 cuda，nvtx 抓取 run_alexnet_inference.sh 脚本执行的相关日志，并将抓取到的日志保存在指定的名为 alexnet.pttrace 文件之中。
抓取 python 脚本日志	<code>ixsys -o alexnet_py.pttrace python3 main.py</code>	使用默认 trace 参数，即 cuda，nvtx，抓取 python3 下运行 main.py 脚本执行的相关日志，并将抓取到的日志保存在指定的名为 alexnet_py.pttrace 文件之中。
抓取带参数的 python 脚本日志	<code>ixsys -o bert_param.pttrace -t cuda python3 -u train.py -b 128 --epochs 1 --lr 8e-1 --task_name mrpc --multiprocessing-distributed --world-size 1 --rank 0 -j 4</code>	使用指定的 trace 参数，当前指定参数为仅抓取 cuda，抓取 python3 运行脚本 train.py 在其特定参数下的相关日志，并将日志保存在指定的名为 bert_param.pttrace 的文件之中。
抓取带调用栈信息的 cuda 可执行程序日志	<code>ixsys -c -o matrixMul_c.pttrace ./matrixMul</code>	使用默认 trace 参数，即 cuda，nvtx，抓取 matrixMul 应用程序执行的相关日志，同时按照默认的调用栈抓取参数（不抓内核态调用栈，以 1000Hz 的默认采样频率）抓取调用栈信息，并将抓取到的日志保存在指定的名为 matrixMul_c.pttrace 文件之中。

使用场景	示例命令	说明
抓取包含内核态调用栈信息的 shell 脚本日志	<code>ixsys -c --kernel true -o alexnet_c_k.pttrace bash run_alexnet_inference.sh</code>	使用默认 trace 参数，即 cuda, nvtx，抓取 run_alexnet_inference.sh 脚本执行的相关日志，同时抓取调用栈信息，包括内核态调用栈信息，调用栈信息抓取的采样频率为默认频率即 1000Hz，并将抓取到的日志保存在指定的名为 alexnet_c_k.pttrace 文件之中。
抓取带参数同时带调用栈的 python 脚本日志	<code>ixsys -o alexnet_c100.pttrace -c --frequency 100 -t cuda,nvtx python3 main.py</code>	使用指定的 trace 参数，当前指定参数为仅抓取 cuda 和 nvtx，抓取 python3 运行脚本 main.py 执行的相关日志，同时以指定的 100Hz 的频率抓取调用栈相关信息（默认不抓取内核态调用栈），并将日志保存在指定的名为 alexnet_c100.pttrace 的文件之中。
抓取指定设备的 trace 信息	<code>ixsys --devices 0,1 -o output.pttrace ./my_app</code>	使用默认 trace 参数，即 cuda, nvtx，抓取 my_app 应用程序的相关 trace 信息，并根据 --devices 选项，限制只抓取指定设备上的 cuda，信息。--devices 选项只控制 cuda, mem, power 和 temperature 的信息抓取，对于其它信息不生效。
按照指定频率抓取指定设备的 metric 数据	<code>ixsys -o generate_metric_500.pttrace --gpu-metrics-device 0 --gpu-metrics-frequency 500 --gpu-metrics-set mr_generate_metrics python3 main.py</code>	使用指定的 500Hz 的采样频率，抓取应用在指定设备 deviceId=0 上的指定的 mr_generate_metrics set 数据，并将 metrics 数据保存在指定的名为 generate_metric_500.pttrace 的文件中。
按照代码中指定的代码段抓取 cuda 数据	<code>ixsys --profile-from-start off -t cuda -o output.pttrace ./my_app</code>	指定刚开始不抓取 cuda 数据，而是当代码中调用了 {cuda,cu}ProfilerStart 时才抓取 cuda 数据，当代码调用 {cuda,cu}ProfilerStop 时停止，并将数据保存在 output.pttrace 文件中。
抓取 IB 网卡流量	<code>ixsys --nic-metrics true -o output.pttrace ./my_app</code>	使用固定的 500Hz 的采样频率，抓取 IB 网卡所有端口的流量情况
抓取代码中使用 cudaEventRecord 和 cudaStreamWaitEvent 同步的同步点信息	<code>ixsys -o output.pttrace --event-sync true ./my_app</code>	抓取 my_app 代码中使用 cudaEventRecord 和 cudaStreamWaitEvent 同步的同步点信息，用于开发人员检查代码中的同步点信息是否正确

5.2.4 ixSYS 命令行工具功能列表

ixSYS 命令行工具提供以下功能：

简写, 命令全称	可选值	默认值	命令描述
-h, --help			打印 ixSYS 的帮助提示信息
-v, --version			显示 ixSYS 命令行工具版本信息
-o, --output	<filename>	none	指定 ixSYS 抓取的 trace 文件的文件名, 该参数必须指定
-t, --trace	cuda, osrt, nvtx, cudnn, cuinfer, raw_syscalls, video, mem, power, temperature, proc_kthread, ...	cuda, nvtx	选取需要抓取的 API 类型, 支持选择一个或多个。如选择多个类型, 则类型需要以逗号相连接且没有空格 注意, "raw_syscalls" 和 "proc_kthread" 使用时必须同时使用 "osrt"
-d, --deferred_trace	1 <= integer <= 2 ³² -1	none	以秒为单位延迟采集 trace
-l, --limited_trace	1 <= integer <= 2 ³² -1	none	以秒为单位限制 trace 采集时间
-f, --filter_fttrace_syscalls	1 <= integer <= 1000000	1000	过滤掉运行时长过于小的 ftrace syscalls, 这样可以减小 trace 文件的大小 -f 0 表示不过滤
-c, --callstack	--kernel on --frequency 100 --kernel off --frequency 500 ...	--kernel false --frequency 1000	使能 callstack 功能, 非必须参数, 使用该参数可抓取程序相关进程的 callstack 注意, 该功能需要使用 sudo 权限 抓取 callstack 时, 可设置是否采集内核态调用栈信息, 以及采样频率
--devices	all, 0, 1, ..., <id>	all	限制 cuda, mem, power, temperature, osrt, nvtx, cudnn, cuinfer, video, raw_syscalls 和 proc_kthread 选项的采集范围 取值范围为可选的设备物理 id 列表, 并以逗号相连且没有空格

简写, 命令全称	可选值	默认值	命令描述
-- profile-from-start	on, off	on	选择是否从应用启动时进行 profiling。设置为 off 时, 可在代码中使用 {cu,cuda}P rofiler{Start,Stop} 开启或停止 profiling 限制: 该功能主要是控制 cuda 数据的抓取, 因此只有 -t 中包含 cuda 才可以使用该功能
-- gpu-metrics-device	none, all, 0, 1, ..., <id>	none	ixSYS 抓取指定 GPU 的 metric 信息, 该参数可通过 - gpu-metrics-device help 来获取 GPU IDs
--gpu -metrics-frequency	10 <= integer <= 200000	500	指定 metrics 的采样频率 注意, 由于当前硬件的限制, metrics sampling 的性能较差, 推荐使用默认频率
--gpu-metrics-set	0, 1, ..., 9 or mr_gener ate_metrics, ...	0	指定抓取的 metric set, 详情可通过 --gpu-metrics-set help** 来查看
--nic-metrics	true, false	false	ixSYS 获取 IB 网卡流量
--event-sync	true, false	false	ixSYS 采集通过 cudaEventRecord 和 cudaStreamWaitEvent 同步的同步信息 注意, 使用该功能必须同时使能 cu da_api 和 cuda_device

ixSYS -t 选项详细说明如下:

API 类型	说明	附加条件
cuda	使能跟踪 cuda cuda 可分为 cuda_device 和 cuda_api 其中, cuda_device 又可分为 cuda_memcpy, cuda_memset, cuda_kernel 和 cuda_memcpy_p2p 对于 cuda 这些细分的 API 类型, 均可单独或组合进行抓取 同样, 在进行组合时, 需要以逗号直接相连接	
nvtx	使能跟踪 nvtx	
mem	使能跟踪 GPU memory 使用	
osrt	使能跟踪操作系统运行时库	
raw_syscalls	使能跟踪系统调用	必须使能 osrt
proc_kthread	使能跟踪线程调度	必须使能 osrt
cudnn	使能跟踪 cudnn 库	
cuinfer	使能跟踪 cuinfer 库	
video	使能跟踪 video 相关库	
power	使能跟踪 GPU 功耗	必须使能 cuda_memcpy, cuda_memset, cuda_kernel 或 cuda_memcpy_p2p 的一种或多种, 也可以直接使能 cuda_device 或 cuda
temperature	使能跟踪 GPU 温度	必须使能 cuda_memcpy, cuda_memset, cuda_kernel 或 cuda_memcpy_p2p 的一种或多种, 也可以直接使能 cuda_device 或 cuda

ixSYS -c 子参数详细说明如下:

参数	可选值	默认	描述
--kernel	true, false	false	指定采集 callstack 信息时是否带内核态调用栈信息
--frequency	1 <= integer <= 10000	1000	指定 callstack 的采样频率值, 单位为 Hz

5.2.5 使用命令行强制结束 trace 抓取

通常情况下，ixSYS 会在应用结束后整合并保存抓取的 trace 数据至文件中。然而，对于生成大量 trace 数据但仅需分析其中一小部分的应用来说，完整抓取所有 trace 数据会导致文件过大，并且需要额外等待应用结束的时间。ixSYS 支持在应用强制退出时保存 trace 数据，避免上述问题。

需要注意的是，直接使用 **Ctrl+C** 终止操作不可行，因为这将导致 ixSYS 进程一并结束，无法正确保存已抓取的数据。正确的做法是：仅终止应用进程，允许应用提前退出。ixSYS 具备周期性保存机制，能够确保即使在应用提前退出的情况下，已抓取的数据也能被正常保存至 trace 文件中，只需等待 ixSYS 正常结束即可完成整个过程。

具体步骤参考如下 (设定 ixSYS 抓取的应用为 myAPP)：

1. 获取应用进程号 pid。

```
$ ps -ef | grep myApp
```

2. kill 应用进程。

- a. 对于有信号机制的框架 (比如超时检测或心跳机制)，当主进程被 kill 后，各个 GPU 上的子进程也会自动结束，因此只需要 kill 主进程即可。

```
$ kill -9 pid
```

- b. 对于没有信号机制的框架，可以通过 **pstree -p pid** 方式检测出 pid 的所有子进程，通过 **kill** 命令结束主进程以及子进程：

```
$ pstree -p pid
$ kill -9 pid pid1 pid2 ...
```

5.2.6 支持抓取的 API List

5.2.6.1 NVTX 函数列表

支持获取以下 NVTX 函数的 activity trace：

```
nvtxMarkEx
nvtxMarkA
nvtxRangeStartEx
nvtxRangeStartA
nvtxRangeEnd
nvtxRangePushEx
nvtxRangePushA
nvtxRangePop
nvtxNameOsThreadA
nvtxDomainMarkEx
nvtxDomainRangeStartEx
nvtxDomainRangeEnd
nvtxDomainRangePushEx
nvtxDomainRangePop
```

```
nvtxDomainRegisterStringA  
nvtxDomainCreateA  
nvtxDomainDestroy
```

5.2.6.2 cuDNN 函数列表

支持获取以下 cuDNN 函数的 activity trace:

```
cudaDnnActivationBackward  
cudaDnnActivationForward  
cudaDnnAddTensor  
cudaDnnBatchNormalizationBackward  
cudaDnnBatchNormalizationForwardInference  
cudaDnnBatchNormalizationForwardTraining  
cudaDnnConvolutionBackwardBias  
cudaDnnConvolutionBackwardData  
cudaDnnConvolutionBackwardFilter  
cudaDnnConvolutionBiasActivationForward  
cudaDnnConvolutionForward  
cudaDnnCreateActivationDescriptor  
cudaDnnCreateConvolutionDescriptor  
cudaDnnCreateDropoutDescriptor  
cudaDnnCreateFilterDescriptor  
cudaDnnCreateLRNDescriptor  
cudaDnnCreatePersistentRNNPlan  
cudaDnnCreatePoolingDescriptor  
cudaDnnCreateRNNDescriptor  
cudaDnnCreateTensorDescriptor  
cudaDnnCreate  
cudaDnnDeriveBNTensorDescriptor  
cudaDnnDestroyActivationDescriptor  
cudaDnnDestroyConvolutionDescriptor  
cudaDnnDestroyDropoutDescriptor  
cudaDnnDestroyFilterDescriptor  
cudaDnnDestroyLRNDescriptor  
cudaDnnDestroyPersistentRNNPlan  
cudaDnnDestroyPoolingDescriptor  
cudaDnnDestroyRNNDescriptor  
cudaDnnDestroyTensorDescriptor  
cudaDnnDestroy  
cudaDnnDropoutBackward  
cudaDnnDropoutForward  
cudaDnnDropoutGetReserveSpaceSize  
cudaDnnDropoutGetStatesSize  
cudaDnnFindConvolutionBackwardDataAlgorithmEx  
cudaDnnFindConvolutionBackwardDataAlgorithm  
cudaDnnFindConvolutionBackwardFilterAlgorithmEx  
cudaDnnFindConvolutionBackwardFilterAlgorithm  
cudaDnnFindConvolutionForwardAlgorithmEx
```

cudaFindConvolutionForwardAlgorithm
cudaFindRNNBackwardDataAlgorithmEx
cudaFindRNNBackwardWeightsAlgorithmEx
cudaFindRNNForwardInferenceAlgorithmEx
cudaFindRNNForwardTrainingAlgorithmEx
cudaGetActivationDescriptor
cudaGetCallback
cudaGetConvolution2dDescriptor
cudaGetConvolution2dForwardOutputDim
cudaGetConvolutionBackwardDataAlgorithmMaxCount
cudaGetConvolutionBackwardDataAlgorithm
cudaGetConvolutionBackwardDataAlgorithm_v7
cudaGetConvolutionBackwardDataWorkspaceSize
cudaGetConvolutionBackwardFilterAlgorithmMaxCount
cudaGetConvolutionBackwardFilterAlgorithm
cudaGetConvolutionBackwardFilterAlgorithm_v7
cudaGetConvolutionBackwardFilterWorkspaceSize
cudaGetConvolutionForwardAlgorithmMaxCount
cudaGetConvolutionForwardAlgorithm
cudaGetConvolutionForwardAlgorithm_v7
cudaGetConvolutionForwardWorkspaceSize
cudaGetConvolutionGroupCount
cudaGetConvolutionMathType
cudaGetConvolutionNdDescriptor
cudaGetConvolutionNdForwardOutputDim
cudaGetDropoutDescriptor
cudaGetFilter4dDescriptor
cudaGetFilterNdDescriptor
cudaGetLRNDescriptor
cudaGetPooling2dDescriptor
cudaGetPooling2dForwardOutputDim
cudaGetPoolingNdDescriptor
cudaGetPoolingNdForwardOutputDim
cudaGetProperty
cudaGetRNNBackwardDataAlgorithmMaxCount
cudaGetRNNBackwardWeightsAlgorithmMaxCount
cudaGetRNNDescriptor
cudaGetRNNForwardInferenceAlgorithmMaxCount
cudaGetRNNForwardTrainingAlgorithmMaxCount
cudaGetRNNLinLayerBiasParams
cudaGetRNNLinLayerMatrixParams
cudaGetRNNMatrixMathType
cudaGetRNNParamsSize
cudaGetRNNProjectionLayers
cudaGetRNNTrainingReserveSize
cudaGetRNNWorkspaceSize
cudaGetStream
cudaGetTensor4dDescriptor
cudaGetTensorNdDescriptor
cudaGetTensorSizeInBytes

```

cudnnIm2Col
cudnnLRNCrossChannelBackward
cudnnLRNCrossChannelForward
cudnnPoolingBackward
cudnnPoolingForward
cudnnRestoreDropoutDescriptor
cudnnRNNBackwardData
cudnnRNNBackwardWeights
cudnnRNNForwardInference
cudnnRNNForwardTraining
cudnnSetActivationDescriptor
cudnnSetCallback
cudnnSetConvolution2dDescriptor
cudnnSetConvolutionGroupCount
cudnnSetConvolutionMathType
cudnnSetConvolutionNdDescriptor
cudnnSetDropoutDescriptor
cudnnSetFilter4dDescriptor
cudnnSetFilterNdDescriptor
cudnnSetLRNDescriptor
cudnnSetPersistentRNNPlan
cudnnSetPooling2dDescriptor
cudnnSetPoolingNdDescriptor
cudnnSetRNNAlgorithmDescriptor
cudnnSetRNNDescriptor
cudnnSetRNNDescriptor_v5
cudnnSetRNNDescriptor_v6
cudnnSetRNNMatrixMathType
cudnnSetRNNProjectionLayers
cudnnSetStream
cudnnSetTensor4dDescriptorEx
cudnnSetTensor4dDescriptor
cudnnSetTensorNdDescriptorEx
cudnnSetTensorNdDescriptor
cudnnSoftmaxBackward
cudnnSoftmaxForward
cudnnTransformTensor

```

5.2.6.3 OSRT 函数列表

5.2.6.3.1 libc syscalls API 函数列表

```

accept
accept4
acct
alarm
arch_prctl

```

bind
bpf
brk
chroot
clock_nanosleep
connect
copy_file_range
creat
dup
dup2
dup3
epoll_ctl
epoll_pwait
epoll_wait
fallocate
fcntl
fdatasync
flock
fork
fsync
ftruncate
futex
ioctl
ioperm
iopl
kill
listen
membarrier
mlock
mlock2
mlockall
mmap
mount
move_pages
mprotect
mq_notify
mq_open
mq_timedreceive
mq_timedsend
mremap
msgctl
msgget
msgrcv
msgsnd
msync
munmap
nanosleep
open
openat
pause

pipe
pipe2
pivot_root
poll
ppoll
prctl
pread64
preadv
preadv2
process_vm_readv
process_vm_writev
pselect6
ptrace
pwrite64
pwritev
pwritev2
read
readv
reboot
recvfrom
recvmsg
recvmsg
rt_sigaction
rt_sigqueueinfo
rt_sigsuspend
rt_sigtimedwait
sched_yield
seccomp
select
semctl
semget
semop
semtimedop
sendfile64
sendmsg
sendmsg
sendto
shmat
shmctl
shmdt
shmget
shutdown
socket
socketpair
splice
swapoff
swapon
sync
sync_file_range
syncfs

```
tee
tgkill
tkill
truncate
unshare
vfork
vhangup
vmsplice
wait4
waitid
write
writev
```

5.2.6.3.2 pthread API 函数列表

```
pthread_barrier_wait
pthread_cancel
pthread_cond_broadcast
pthread_cond_signal
pthread_cond_timedwait
pthread_cond_wait
pthread_create
pthread_join
pthread_kill
pthread_mutex_lock
pthread_mutex_timedlock
pthread_mutex_trylock
pthread_rwlock_rdlock
pthread_rwlock_timedrdlock
pthread_rwlock_timedwrlock
pthread_rwlock_tryrdlock
pthread_rwlock_trywrlock
pthread_rwlock_wrlock
pthread_spin_lock
pthread_spin_trylock
pthread_timedjoin_np
pthread_tryjoin_np
pthread_yield
sem_timedwait
sem_trywait
sem_wait
```

5.2.6.3.3 IO 函数列表

aio_fsync
aio_fsync64
aio_suspend
aio_suspend64
fclose
fcloseall
fflush
fflush_unlocked
fgetc
fgetc_unlocked
fgets
fgets_unlocked
fgetwc
fgetwc_unlocked
fgetws
fgetws_unlocked
flockfile
fopen
fopen64
fputc
fputc_unlocked
fputs
fputs_unlocked
fputwc
fputwc_unlocked
fputws
fputws_unlocked
fread
fread_unlocked
freopen
freopen64
ftrylockfile
fwrite
fwrite_unlocked
getc
getc_unlocked
getdelim
getline
getw
getwc
getwc_unlocked
lockf
lockf64
mkfifo
mkfifoat
posix_fallocate
posix_fallocate64
putc
putc_unlocked
putwc

putwc_unlocked

5.2.6.4 JPU 相关函数列表

支持抓取的函数接口列表如下：

```

cujpegDestroyDecoder
cujpegDecodeV1
cujpegCreateEncoderForMem
cujpegDecodeBatchedV1
cujpegEncodeBatchedForMem
cujpegDestroyEncoder
cujpegEncode
cujpegCreateDecoder
cujpegCreateEncoder
cujpegEncodeBatched
nvjpegGetPinnedMemoryPadd
nvjpegGetImageInfo
nvjpegSetDeviceMemoryPadd
nvjpegJpegStateCreate
nvjpegEncoderParamsSetOpt
nvjpegEncodeYUV
nvjpegDecodeParamsSetOutp
nvjpegEncoderParamsSetQua
nvjpegSetPinnedMemoryPadd
nvjpegDestroy
nvjpegDecodeBatchedInitia
nvjpegCreateSimple
nvjpegEncodeGetBufferSize
nvjpegEncodeRetrieveBitst
nvjpegEncoderParamsDestro
nvjpegDecodeBatched
nvjpegEncodeImage
nvjpegEncoderParamsSetEnc
nvjpegEncoderParamsSetSam
nvjpegDecodeParamsSetScal
nvjpegJpegStateDestroy
nvjpegDecodeParamsDestroy
nvjpegDecodeParamsCheck
nvjpegEncoderStateCreate
nvjpegDecodeParamsSetROI
nvjpegDecodeParamsSetAllo
nvjpegEncodeRetrieveBitst
nvjpegEncodeRetrieveBitst
nvjpegCreate
nvjpegDecode
nvjpegGetDeviceMemoryPadd
nvjpegEncodeRetrieveBitst
nvjpegDecodeParamsCreate

```

```
nvjpegCreateEx
nvjpegDecodeParamsSetExif
nvjpegEncoderStateDestroy
nvjpegEncodeYUVBatched
nvjpegEncoderParamsCreate
```

5.2.6.5 VPU 相关函数列表

支持抓取的函数接口列表如下：

```
cuvidDecCtxCreate
cuvidDecCtxDestroy
cuvidCreateDecoder
cuvidDestroyDecoder
cuvidDecodePicture
cuvidGetDecodeStatus
cuvidReconfigureDecoder
cuvidMapVideoFrame
cuvidUnmapVideoFrame
cuvidDecodeGetPictureInfo
cuvidCreateVideoParser
cuvidParseVideoData
cuvidDestroyVideoParser
```

5.2.6.6 cuINFER 函数列表

支持获取以下 cuINFER 函数的 activity trace：

```
cuinferActivationForward
cuinferAddTensor
cuinferArrangeAttenOutputI8II80
cuinferArrangeEncselfQkvI8II80
cuinferArrangeEncselfQkvSepI8II80
cuinferBatchNormalizationForwardInference
cuinferBiasGeluI8II80
cuinferBiasResidualLn
cuinferConcatenate
cuinferConvolutionForward
cuinferCorrelationSoftmaxEncselfI32II80
cuinferCorrelationSoftmaxEncselfI8II80
cuinferCreate
cuinferCreateActivationDescriptor
cuinferCreateConvolutionDescriptor
cuinferCreateCTCLossDescriptor
cuinferCreateDropoutDescriptor
cuinferCreateFilterDescriptor
cuinferCreateLRNDescriptor
```

cuinferCreatePoolingDescriptor
cuinferCreateReduceTensorDescriptor
cuinferCreateRNNDescriptor
cuinferCreateTensorDescriptor
cuinferCropAndResize
cuinferCTCLoss
cuinferCustomGemm
cuinferCustomGemmEx
cuinferDeQuantSoftmaxForwardQuant
cuinferDeriveBNTensorDescriptor
cuinferDestroy
cuinferDestroyActivationDescriptor
cuinferDestroyConvolutionDescriptor
cuinferDestroyCTCLossDescriptor
cuinferDestroyDropoutDescriptor
cuinferDestroyFilterDescriptor
cuinferDestroyLRNDescriptor
cuinferDestroyPoolingDescriptor
cuinferDestroyRNNDescriptor
cuinferDestroyTensorDescriptor
cuinferDropoutForward
cuinferDropoutGetReserveSpaceSize
cuinferDropoutGetStatesSize
cuinferEncEmbI8I
cuinferEncEmbI8I_M8I
cuinferFindConvolutionForwardAlgorithm
cuinferFindConvolutionForwardAlgorithmEx
cuinferFindConvolutionForwardAlgorithmFP16
cuinferFMHAFoward
cuinferFMHAFowardEx
cuinferFusedMultiHeadAttentionI8
cuinferGetActivationDescriptor
cuinferGetConvolution2dDescriptor
cuinferGetConvolution2dForwardOutputDim
cuinferGetConvolutionForwardAlgorithm
cuinferGetConvolutionForwardAlgorithm_v7
cuinferGetConvolutionForwardAlgorithmMaxCount
cuinferGetConvolutionForwardWorkspaceSize
cuinferGetConvolutionGroupCount
cuinferGetConvolutionMathType
cuinferGetConvolutionNdDescriptor
cuinferGetConvolutionNdForwardOutputDim
cuinferGetCTCLossDescriptor
cuinferGetCTCLossDescriptorEx
cuinferGetCTCLossWorkspaceSize
cuinferGetCudartVersion
cuinferGetCustomGemmExWorkspace
cuinferGetCustomGemmWorkspace
cuinferGetDropoutDescriptor
cuinferGetErrorString

cuinferGetFilter4dDescriptor
 cuinferGetFilterNdDescriptor
 cuinferGetFilterSizeInBytes
 cuinferGetHammingDistanceWorkspace
 cuinferGetLRNDescriptor
 cuinferGetNMSBatchedWorkspaceSize
 cuinferGetNMSBatchedYoloFusedWorkspaceSize
 cuinferGetNMSWorkspaceSize
 cuinferGetPooling2dDescriptor
 cuinferGetPooling2dForwardOutputDim
 cuinferGetPoolingForwardWorkspaceSize
 cuinferGetPoolingNdDescriptor
 cuinferGetPoolingNdForwardOutputDim
 cuinferGetProperty
 cuinferGetQDEConvolutionTransposedWorkspaceSize
 cuinferGetReduceWorkspace
 cuinferGetStream
 cuinferGetTensor4dDescriptor
 cuinferGetTensorNdDescriptor
 cuinferGetTensorSizeInBytes
 cuinferGetTopKBatchV2Workspace
 cuinferGetTopKBatchWorkspace
 cuinferGetTopKWorkspace
 cuinferGetVersion
 cuinferGPTFMHAFoward
 cuinferGPTFMHAQKVForward
 cuinferGroupNorm
 cuinferHalfConvolution2dForward
 cuinferHammingDistance
 cuinferIm2Col
 cuinferInstanceNorm
 cuinferLayerNorm
 cuinferLayerNormResidualI80F0
 cuinferLayerNormResualI80
 cuinferLRNCrossChannelForward
 cuinferLSTMForwardInference
 cuinferNMS
 cuinferNMSBatched
 cuinferNMSBatchedYoloFused
 cuinferPoolingForward
 cuinferPoolingForwardEx
 cuinferQConvolutionForward
 cuinferQDConvolutionForward
 cuinferQDEConvolutionForward
 cuinferQDEConvolutionTranspose
 cuinferReduce
 cuinferReduceTensor
 cuinferResidualBiasInI32I
 cuinferResidualBiasInI32II80
 cuinferResidualBiasInI8I

```
cuinferResidualBiasLnI8II80
cuinferResidualBiasLnI8II80F
cuinferResize2D
cuinferRestoreDropoutDescriptor
cuinferSetActivationDescriptor
cuinferSetConvolution2dDescriptor
cuinferSetConvolutionGroupCount
cuinferSetConvolutionMathType
cuinferSetConvolutionNdDescriptor
cuinferSetCTCLossDescriptor
cuinferSetCTCLossDescriptorEx
cuinferSetDropoutDescriptor
cuinferSetFilter4dDescriptor
cuinferSetFilterNdDescriptor
cuinferSetLRNDescriptor
cuinferSetPooling2dDescriptor
cuinferSetPoolingNdDescriptor
cuinferSetReduceTensorDescriptor
cuinferSetRNNDDescriptor
cuinferSetStream
cuinferSetTensor4dDescriptor
cuinferSetTensor4dDescriptorEx
cuinferSetTensorNdDescriptor
cuinferSetTensorNdDescriptorEx
cuinferSoftmaxForward
cuinferSplitForward
cuinferTopK
cuinferTopKBatch
cuinferTopKBatchV2
cuinferTransformTensor
cuinferTranspose
cuinferViterbiDecode
cuinferYoloV5Detect
```

5.2.7 常见问题

问题 1. 使用 ixSYS 提示 Cannot access ftrace control node under /sys/kernel/tracing or /sys/kernel/debug/tracing. Please check your environment! , 需要如何检查?

问题分析

因为 ixSYS 依赖于内核的 ftrace 相关功能, 在使用 ixSYS 抓取 trace 之前, 需要确认系统是否已经正确挂载了 debugfs 和 tracefs, 以及当前用户有访问 debugfs 的权限。

解决方法

1. 首先检查系统是否正确挂载了 debugfs 和 tracefs, 使用以下命令可以检查 debugfs 和 tracefs 是否被挂载。

```
$ mount | grep debugfs
$ mount | grep tracefs
```

2. 如未挂载，则可以参考本文[开始之前](#) 中的步骤 1 挂载 debugfs 和 tracefs。
3. 之后，请检查当前用户是否有 tracefs 的访问权限，可以使用以下命令来确认当前用户是否有 tracefs 的访问权限。

```
$ ls -al /sys/kernel/tracing/
```

4. 如检查发现当前用户没有权限控制 tracefs，则可以参考本文[开始之前](#) 中的步骤 2，为当前用户获取 ftrace 相关节点的控制权。

问题 2. 使用 ixSYS 提示 Ftrace is in use, please check your environment!，需要如何检查？

问题分析

ixSYS 依赖于内核的 ftrace 相关功能，在使用 ixSYS 抓取 trace 之前，应确保默认的 ftrace 处在关闭的状态。

解决方法

1. 确认当前系统没有其他用户或者应用程序正在使用 ftrace 相关的功能。
2. 确认后，可参考本文[开始之前](#) 中的步骤 3 关闭 ftrace。ixSYS 会在完成相关配置之后使能 ftrace 开始抓取相关日志，并在日志抓取结束之后关闭 ftrace 使能。

问题 3. callstack 使用 sudo 权限导致 cuda 等程序缺少环境变量找不到依赖库。

问题分析

由于 callstack 功能需要使用 sudo 权限，这时可能会导致执行的 cuda 程序或模型找不到依赖库。

解决方法

在当前用户的 .bashrc（如果是其他 shell 请找到对应的文件）文件中添加如下代码，使得 sudo 继承当前用户的环境变量：

```
$ alias sudo='sudo env PATH=$PATH'
$ alias sudo='sudo env PATH=$PATH LD_LIBRARY_PATH=$LD_LIBRARY_PATH'
```

问题 4. 使用 ixSYS 抓取网卡流量时，提示 [WARNING] libibmad.so: not found. [WARNING] libibumad.so: not found，同时输出 Log 中显示 nic_metrics = false。

问题分析

ixSYS 抓取网卡流量一开 ibmad 和 ibumad 两个库，因此需要独立安装这两个库。

解决方法

- Ubuntu 环境，执行以下命令安装 ibmad 和 ibumad 库：

```
$ apt install libibmad-dev
$ apt install libibumad-dev
```

- CentOS 环境，执行以下命令安装 ibmad 和 ibumad 库：

```
$ yum install libibmad-devel
$ yum install libibumad-devel
```

问题 5. 需要提前强制结束 trace 的抓取，如应用的 trace 数据全部抓取时，trace 文件太大，只需要一部分数据即可分析问题。

问题分析

不可以通过 **Ctrl+C** 来结束数据抓取，因为 **Ctrl+C** 会使得 ixSYS 进程也提前结束，而无法将抓取到的 trace 数据保存至最终的 trace 文件中。由于 ixSYS 抓取 trace 数据主要依赖应用进程是否还在运行，因此只需要将被 trace 的应用进程 kill 掉，即可使应用提前退出 (周期性保存 trace 数据的机制会保证数据被正常保存)，而不影响 ixSYS 将数据保存至最终的 trace 文件中。

解决方法

1. 通过 **ps -ef | grep myapp** 命令获取 myapp 进程 pid。
2. 大部分框架有信号机制，比如超时检测或心跳机制。当主进程被 kill 后，各个 GPU 上的子进程也会自动结束，因此不需要单独 kill 子进程，只需要执行 **kill -9 pid** 命令 kill 主进程即可。
3. 对于没有信号机制的框架，可以通过 **pstree -p pid** 方式检测出所有 pid 的子进程，并通过 **kill** 命令结束主进程以及子进程以提前结束 trace 数据的抓取。

5.3 2. 使用 ixSYS 图形界面工具分析 trace

ixSYS 图形界面工具用 web 的图形化的方式将 ixSYS 命令行工具抓取的 trace 信息进行统计及展示。您可使用该工具更直观地进行系统性能分析，了解应用程序在天数智芯加速卡、CPU 以及操作系统中的运行情况。

请使用 Firefox44、Chrome45 或更高版本的浏览器。

5.3.1 打开 ixSYS 图形界面工具

打开浏览器输入网址 <https://ui.ixsys.iluvatar.com> 进入 ixSYS 图形界面主页面：



图 1: ixSYS 图形界面主页面

界面说明：

1. 导入 trace 文件（文件小于 400MB 时可直接导入，更大文件请参考“导入大 trace 文件”）
2. 快捷键介绍（可能会存在与系统快捷键冲突的情况）
3. 基本功能介绍
4. 大 trace 文件导入辅助工具下载
5. 离线工作模式开关
6. WebAssembly 状态标识
7. 加载状态显示，error 提示，trace 名称搜索等
8. UI 简介

5.3.2 导入 trace 文件

5.3.2.1 导入 trace 文件（小于 400MB）

点击 **Open trace file** 导入文件后进入如下用户界面：

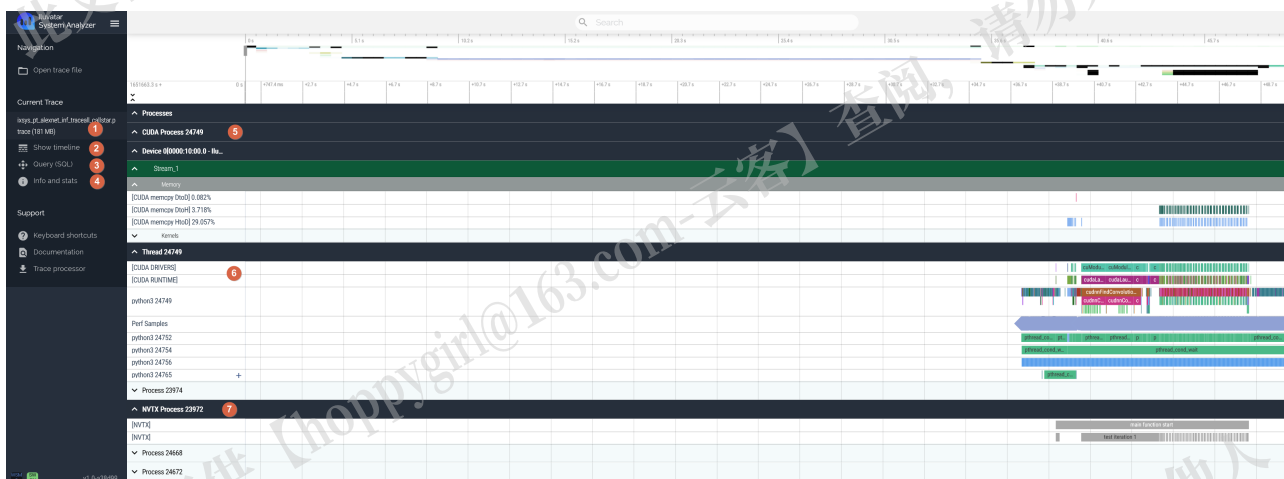


图 2: trace 文件导入后用户界面

界面说明：

1. 导入文件名称，文件大小显示
2. Trace View 主页面显示
3. 跳转至数据库查询页面
4. Trace 文件相关信息显示
5. 程序运行 trace 设备相关数据，如 stream，kernels 以及 memory
6. 线程调用 CUDA Driver、Runtime API 数据统计
7. CUDA NVTX trace 数据展示（如未抓取 NVTX 数据，则 group 内无数据展示），支持 nvtx domain 特性

5.3.2.2 导入大 trace 文件（大于 400MB）

一般超过 400MB 的 trace 文件是无法通过浏览器直接打开，需要借助 trace_processor_shell 辅助工具来实现数据的导入。

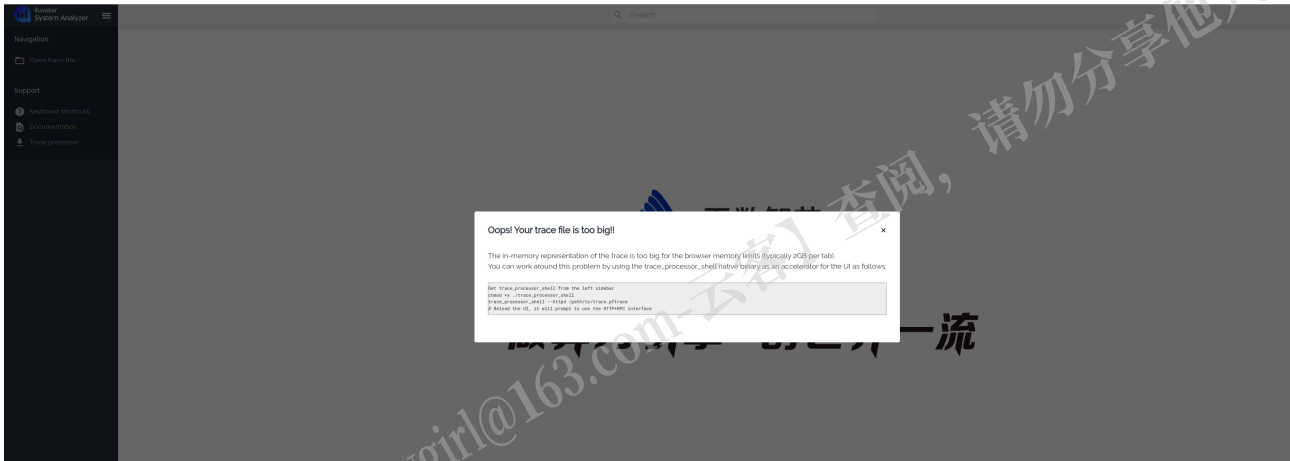


图 3: 页面提示用户文件太大无法导入

5.3.2.2.1 下载 trace_processor_shell 辅助工具

点击左侧导航栏 Trace processor 按钮，点击 **Download**，根据程序运行的系统架构选择对应的应用版本。当前仅支持 Windows 64 位系统，Linux x86 和 Linux arm 64 位系统，以及 Apple arm 架构系统。

5.3.2.2.2 安装 trace_processor_shell 辅助工具

- Linux：在文件所在目录执行以下命令使该文件具有可执行权限：

```
$ chmod +x trace_processor_shell
```

- Windows：下载后即可执行，无需安装。

5.3.2.2.3 导入大 trace 文件

场景一：在本地使用 trace_processor_shell 导入大 trace 文件

如果用户的浏览器以及 trace 文件在同一主机可使用 trace_processor_shell 导入大 trace 文件：

1. 切换到执行文件所在目录，在 Linux 或 Windows cmd 中执行如下命令：

```
$ ./trace_processor_shell --httpd ${your_trace_file.pftrace}
```

2. 等待 trace 文件解析成功提示，刷新 <https://ui.ixsys.iluvatar.com> 页面时，刷新页面，等待文件加载成功。由于文件的内容比较多，加载的过程会比较缓慢，需要耐心等待。

说明：

- 如果使用 **--httpd-port** 指定非默认端口 PORT，可以使用如下链接格式导入文件：https://ui.ixsys.iluvatar.com#!?rpc_port=PORT。
- 使用 **--httpd-port** 指定不同端口，可支持多页签导入多文件。

导入大 trace 文件示例：

```
@Tool-1:~/testdata/3.3$ ./trace_processor_shell --httpd ./resnet50_1epoch_4_devices
[748.890] e_processor_impl.cc:954 parsing file:./resnet50_1epoch_4_devices, please wait a while
[749.110] processor_shell.cc:1376 Trace loaded: ./resnet50_1epoch_4_devices, file size:210.04 MB
[749.111] httpd.cc:100 [HTTP] Starting RPC server on localhost:19001
[749.111] httpd.cc:104 [HTTP] This server can be used by reloading https://ui.ixsys.iluvatar.comclicking on Y
ES on the "Trace Processor native acceleration" dialog.
```

图 4: 在 Linux 系统通过辅助工具导入大 trace 文件示例

```
C:\Users\ \Desktop\release\3.3>trace_processor_shell.exe --httpd ./ixsys_trace_nvtx_nccl_pt_demon.sqlite
[501.231] e_processor_impl.cc:954 parsing file:./ixsys_trace_nvtx_nccl_pt_demon.sqlite, please wait a while
[501.253] processor_shell.cc:1376 Trace loaded: ./ixsys_trace_nvtx_nccl_pt_demon.sqlite, file size:1.09 MB
[501.254] httpd.cc:100 [HTTP] Starting RPC server on localhost:19001
[501.254] httpd.cc:104 [HTTP] This server can be used by reloading https://ui.ixsys.iluvatar.comclicking on Y
ES on the "Trace Processor native acceleration" dialog.
```

图 5: 在 window 系统通过辅助工具导入大 trace 文件示例

场景二：在远程 Linux 服务器使用 trace_processor_shell 导入大 trace 文件

在远程服务器导入大 trace 文件时，可以分为两种场景：1) 本地机器可以与远程机器 SSH 直连 2) 本地机器需要通过堡垒机或者跳转机才能 SSH 连接服务器。

一、SSH 直连远程机器

如果用户的浏览器和 trace 文件不在同一主机，需要远程加载文件。这里特指 trace 文件在远端 Linux 服务器打开，而用户在本地图浏览器分析 trace 文件。

1. 用户打开 Windows cmd 窗口或者 Linux terminal，输入 **ssh -L 19001:127.0.0.1:19001 {username@remote serve IP}**，将远程服务器端口映射到本地，例如：**ssh -L 19001:127.0.0.1:19001 san.zhang@192.168.1.1**，输入该用户的密码。在验证用户密码正确后，将进入远端服务器的 terminal，如下图所示：

```
C:\Users\ >ssh -L 19001:127.0.0.1:19001 @
@s password:
Welcome to Kylin V10 SP1 (GNU/Linux 5.4.0-174-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

196 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '22.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

18 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log
Last login: Thu Apr 11 15:23:28 2024 from 10.206.12.38
yufa.wang@Tool-1:~$
```

图 6: 进入远端服务器 terminal

2. 设置好端口映射后，进入远程主机 terminal 并使用 `./trace_processor_shell --httpd ./resnet50_epoch_4_devices` 命令打开对应的文件，如下图所示：

```
@Tool-1:~/testdata/3.35 ./trace_processor_shell --httpd ./resnet50_epoch_4_devices
[748.890] e_processor_impl.cc:954 parsing file:./resnet50_epoch_4_devices, please wait a while
[749.110] processor_shell.cc:1376 Trace loaded: ./resnet50_epoch_4_devices, file size:210.04 MB
[749.111] httpd.cc:100 [HTTP] Starting RPC server on localhost:19001
[749.111] httpd.cc:104 [HTTP] This server can be used by reloading https://ui.ixsys.iluvatar.com/clicking on Y
ES on the "Trace Processor native acceleration" dialog.
```

图 7: 打开远程主机上的 trace 文件

二、SSH 通过堡垒机/跳转机连接远程机器

该场景下是通过 Nginx 反向代理本地 `http://127.0.0.1:19001` 到远程服务器的 19001 端口来工作的。请确保工作机器与远程服务器之间的网络连接以及防火墙配置可以让工具正确运行。

1. 首先，请根据用户的工作环境下载并安装对应版本的 nginx-1.16.1 包：

- Linux 版本的下载链接：<https://nginx.org/download/nginx-1.16.1.tar.gz>
- Windows 版本的下载链接：<https://nginx.org/download/nginx-1.16.1.zip>

Linux 环境下使用源码编译安装 Nginx，可以参考如下命令：

```
$ ./configure --prefix=/usr/local/nginx --with-http_ssl_module --with-http_v2_module
↪ --without-http_rewrite_module
$ make
```

```
$ sudo make install
```

Linux 环境下，默认安装 bin 文件至 /usr/local/nginx/sbin 目录下，用户可以通过 **help** 命令查看 Nginx 的基本使用方法。

工作主机为 Windows 环境的操作指导如下：

2. Nginx 下载完成后，解压 Nginx 软件包文件，在 nginx-1.16.1/conf/nginx.conf 文件中添加反向代理配置。在 http 配置项中增加 server，如下图所示：

```
http {
    include        mime.types;
    default_type  application/octet-stream;

    #log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
    #                '$status $body_bytes_sent "$http_referer" '
    #                '"$http_user_agent" "$http_x_forwarded_for"';

    #access_log  logs/access.log  main;

    sendfile      on;
    #tcp_nopush   on;

    #keepalive_timeout  0;
    keepalive_timeout  65;

    #gzip  on;
    server {
        listen      19001;
        server_name 127.0.0.1;

        location / {
            proxy_pass http://remote host IP :19001;
        }
    }
}
```

图 8: 在 nginx.conf 文件中添加反向代理配置

Note

请根据实际情况替换上图中“remote host IP”为用户远程机器的 IP 地址。

3. Nginx 配置文件修改完成并保存后，选中 nginx.exe 并在右键菜单中选择 **以管理员身份运行**。其后在开始菜单中找到命令提示符，右键以管理员身份运行，在打开的窗口中切换到 nginx.exe 所在目录，输入 **nginx.exe -s reload** 命令重新加载反向代理。如下图所示：

```
D:\> cd \nginx-1.16.1\nginx-1.16.1 & nginx.exe -s reload
D:\> cd \nginx-1.16.1\nginx-1.16.1
```

图 9: 重新加载反向代理

如果需要停止 Nginx，可以在命令行窗口输入 **nginx.exe -s stop** 命令。

后续如果用户需要在本地使用 trace_processor_shell 导入 trace 文件，也需要检查 Nginx 是否已停止或者关闭。

4. 在 UI 下载 Linux 版本的 trace_processor_shell 并传送至远程服务器相应的工作目录下。在启动 httpd 加载文件时，指定远程主机的 IP 地址，如下图所示：

```
@s-113-2-2:~/code/LLMs$ ./trace_processor_shell --httpd --http-ip 10.0.2.150 --http-port 19001 --megatron-deepspeed/examples/llama2/profile.ptrace
[720.356] e_processor_tmpl.cc:954 parsing file: ./megatron-deepspeed/examples/llama2/profile.ptrace, please wait a while
[720.488] processor_shell.cc:1386 Trace loaded: ./megatron-deepspeed/examples/llama2/profile.ptrace, file size:186.22 MB
[720.488] httpd.cc:113 [HTTP] Starting RPC server on 10.0.2.150:19001
[720.488] httpd.cc:123 [HTTP] Starting RPC server on http://10.0.2.150:19001
[720.488] httpd.cc:126 [HTTP] Starting RPC server not find:http://10.0.2.19001 add to host conf
[720.488] httpd.cc:133 [HTTP] This server can be used by reloading https://ui.ixsys.iluvatar.com clicking on YES on the "Trace Processor native acceleration" dialog.
```

图 10: 指定远程主机的 IP 地址

总结

以上是当前支持的两种远程加载大文件进行分析的方式。

在 RPC 服务提示运行成功后，刷新本地浏览器网页 [https://ui.ixsys.iluvatar.com\(:8443\)](https://ui.ixsys.iluvatar.com(:8443))，在页面弹出的提示窗口内点击 **Yes, use loaded trace** 按钮：

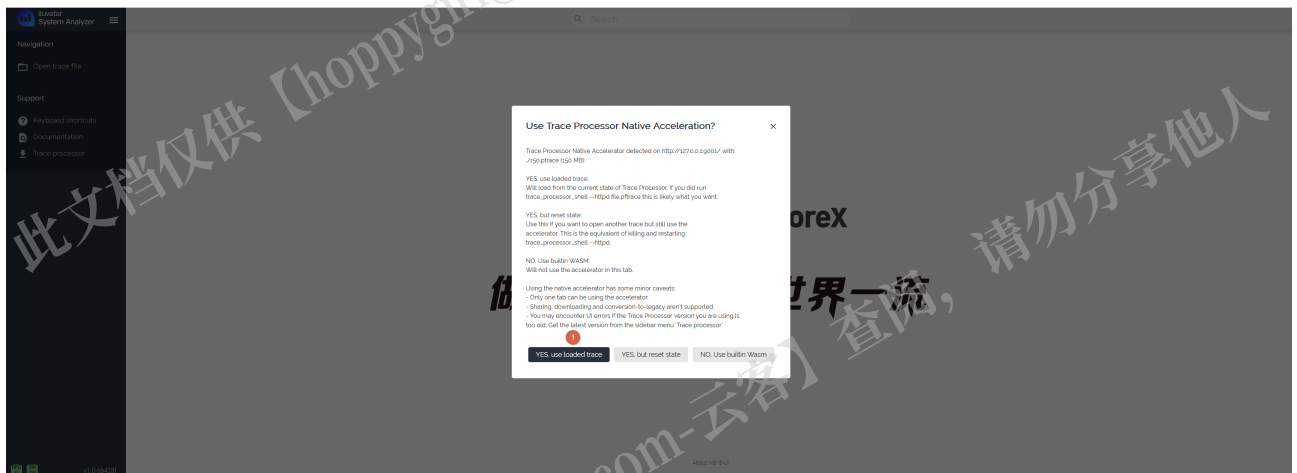


图 11: 确认使用 trace_processor_shell 辅助工具

5.3.3 查看 trace 信息

5.3.3.1 选择区域

对于您感兴趣的区域，按住鼠标左键进行区域选择，被选中的区域内 slice 的统计数据将出在屏幕的底端，分别可以有火焰图（callstack），轴聚合表格，聚合表，Top slice 表，以及 CUDA 统计数据页面。在抓取 memory counter 时，区域选择也会有 counter 页。

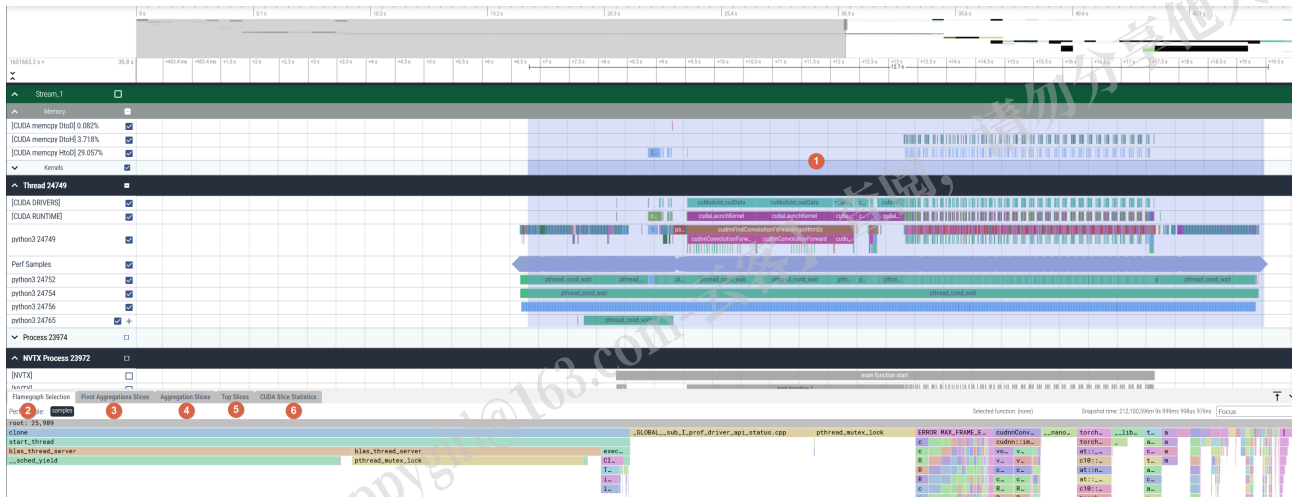


图 12: 被选中的区域内 slice 的统计数据

点击 **Pivot Aggregation Slices** 表头按钮 **Editor** 可以对表格显示内容进行定制：

1. 点击标记 1 处切换轴选项以及聚合选项
2. 点击标记 2 处选择添加对应的 column
3. 点击标记 3 处点击移除已添加的内容
4. 点击标识 4 处进行查询

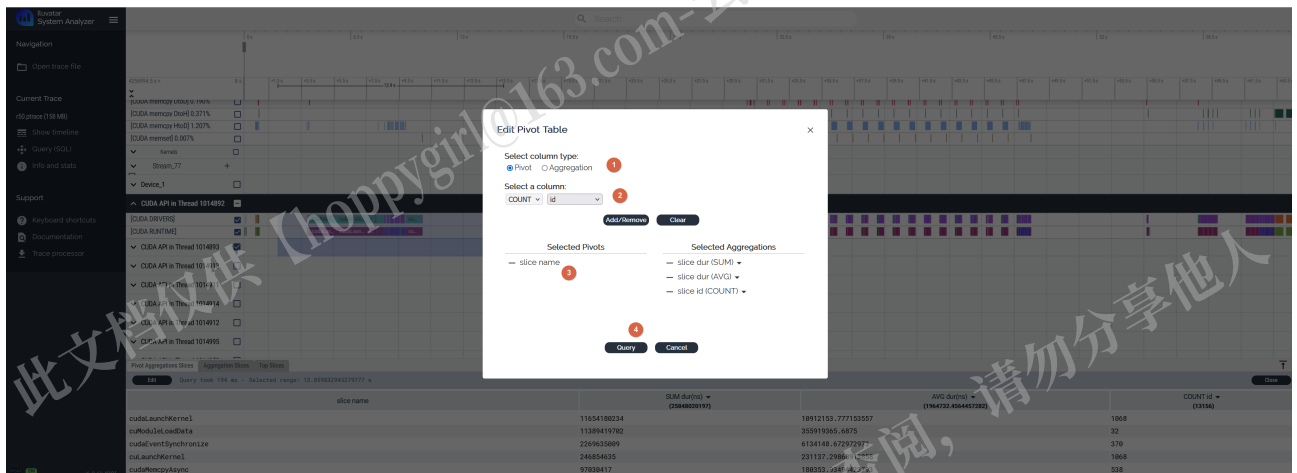


图 13: 修改表格显示内容

以 category 以及 name 作为 Pivot 轴聚合，查询结果如下图所示，首先以 category 作为第一级分类，name 作为第二级分类：

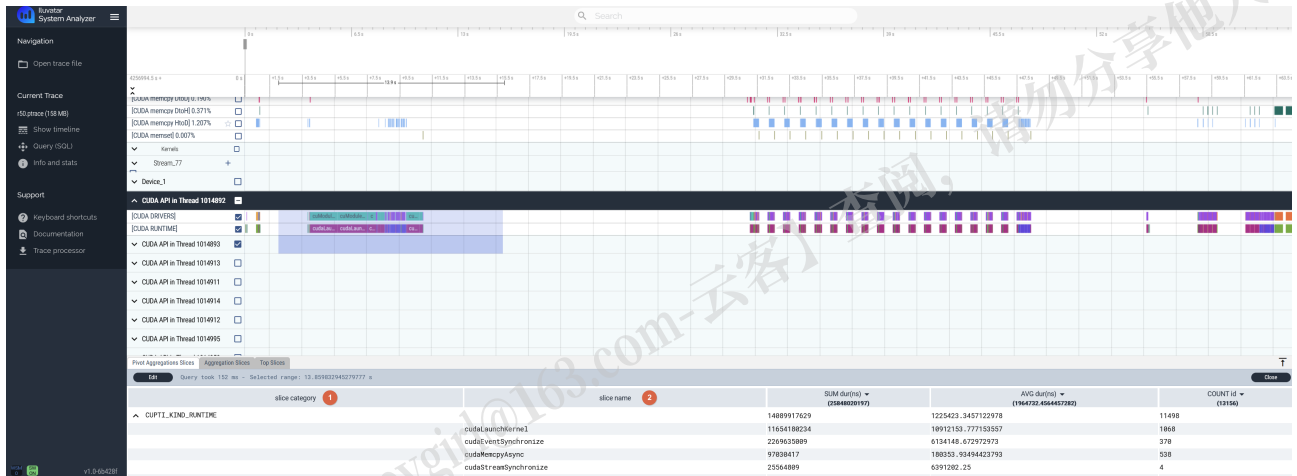


图 14: 以 category 以及 name 作为 Pivot 轴聚合

聚合表是统计同名称的 slice 的数据作为展示内容，包括总计的运行时间，平均运行时间，以及总共出现的次数。

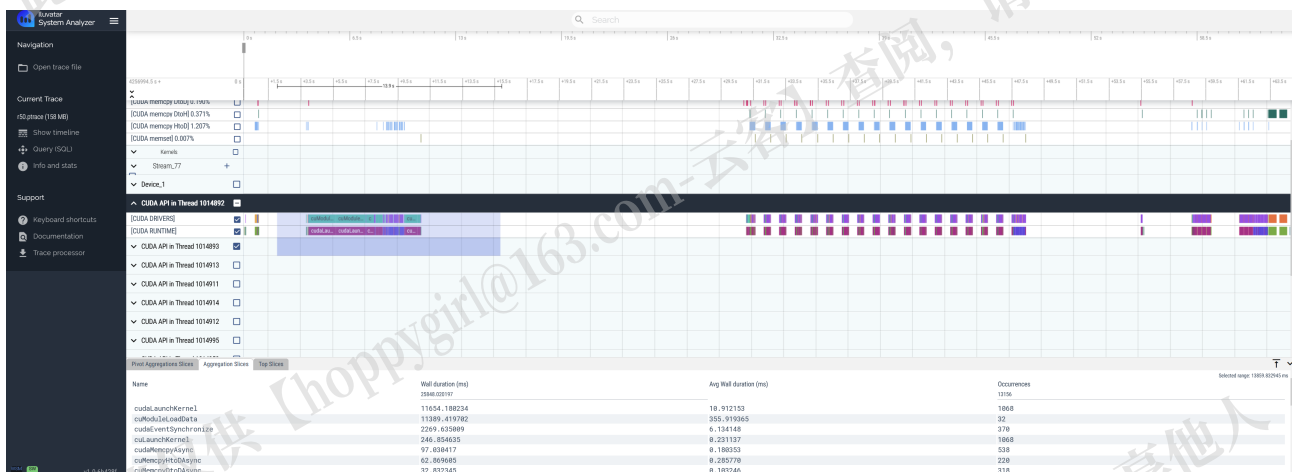


图 15: 聚合表统计同名称的 slice 的数据

Top slice 表格是耗时排序前 100 的 slice 数据统计表，排序为耗时降序排列。用户可以修改标注 3 以及标注 4，点击 submit 按钮来获取数据集。

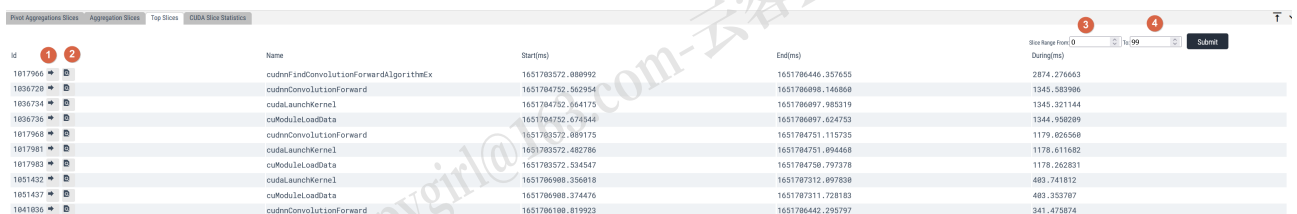


图 16: Top slice 表格显示耗时排序前 100 的 slice 数据

其中在 Top slice 页签表格中，如上图所示，点击跳转按钮（标注 1）可以跳转选中对应的 slice，如后续章节（选择单个 slice）描述。点击查看按钮（标注 2），可以查看该 slice 的附加属性信息。

如下图所示，点击按钮查看附加属性信息。

CUDA Slice Statistics					
Id	Name	Start(ms)	End(ms)	StartRange(ms)	Duration(ms)
1051432	cudaLaunchKernel	1651786988.356818	1651787312.897839	403.741812	
1051437	cudaMemcpyAsync	1651786988.374476	1651787011.720183	403.333707	
1657859	poll	1651718553.477475	1651718604.889453	51.331978	
1578282	poll	1651718124.159311	1651718174.281365	50.222654	
1525485	poll	1651789840.474132	1651789889.336446	48.862314	

图 17: 当前 slice 的附加属性信息

CUDA Slice Statistics 统计数据页面，在区域选中包含 cuda memory, kernel, driver 以及 runtime api 相关区域时出现，分别包含如下统计数据：

1. 区域内累计耗时最长的前十个 kernel 的饼图统计
2. 选中区域内 DtoD, DtoH, HtoD 的 memory 时长饼图统计
3. 区域内累计最耗时的前十个 runtime 的饼图统计
4. 区域内平均耗时最长的前十个 runtime 柱状图统计
5. 区域内全部 kernel, 全部 runtime 以及全部 memory 操作耗时的柱状图统计

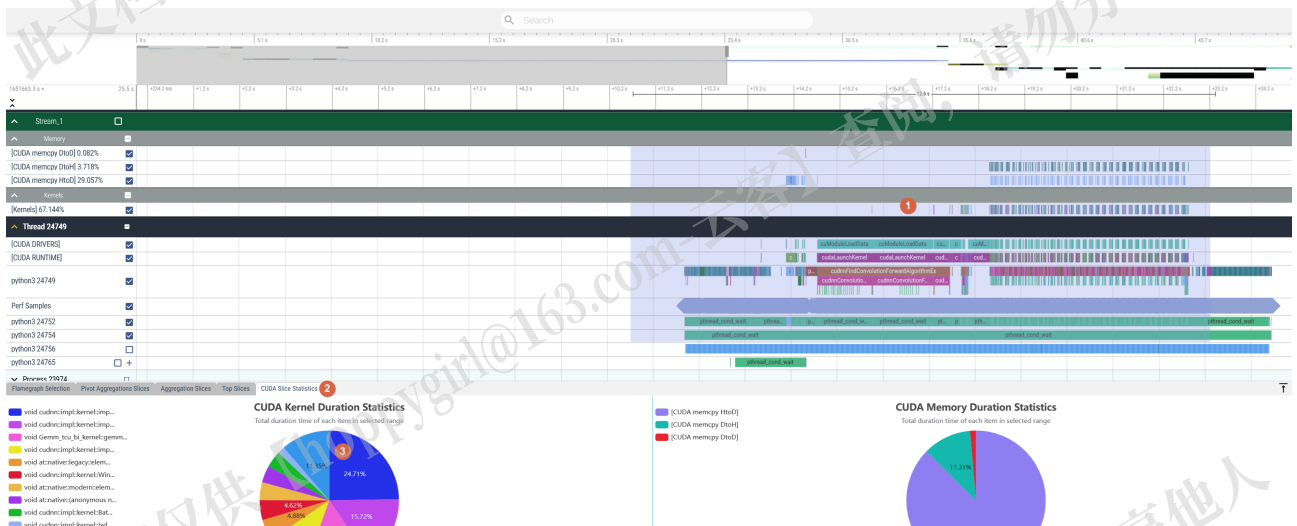


图 18: CUDA 数据统计

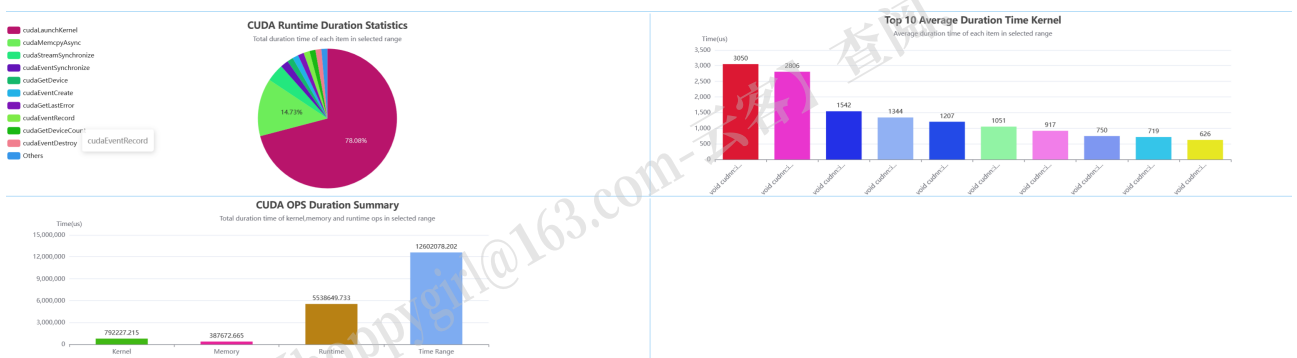


图 19: CUDA 数据统计

在 ixSYS 中抓取了 memory counter 的 trace 内容，在 ixSYS-UI 中将有 GPU 相关的 memory 统计数据。

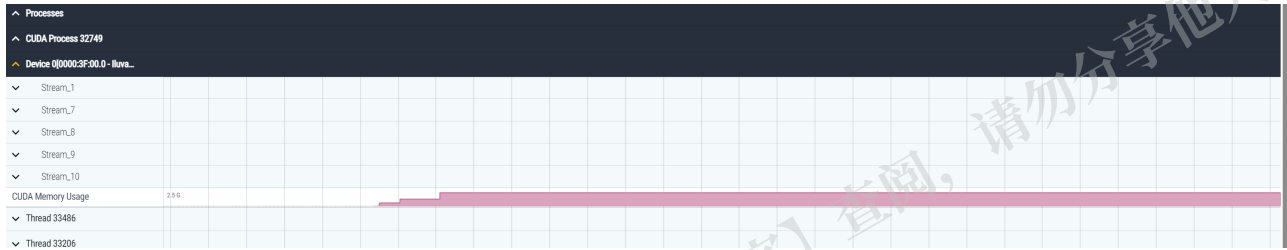


图 20: memory counter 数据统计

点击选中一段 counter 图，在下侧的 tab 页面 Counter Selection 中将出现该段数据的详情描述，Delta 数据为当前 counter 与前段 counter value 的差值。

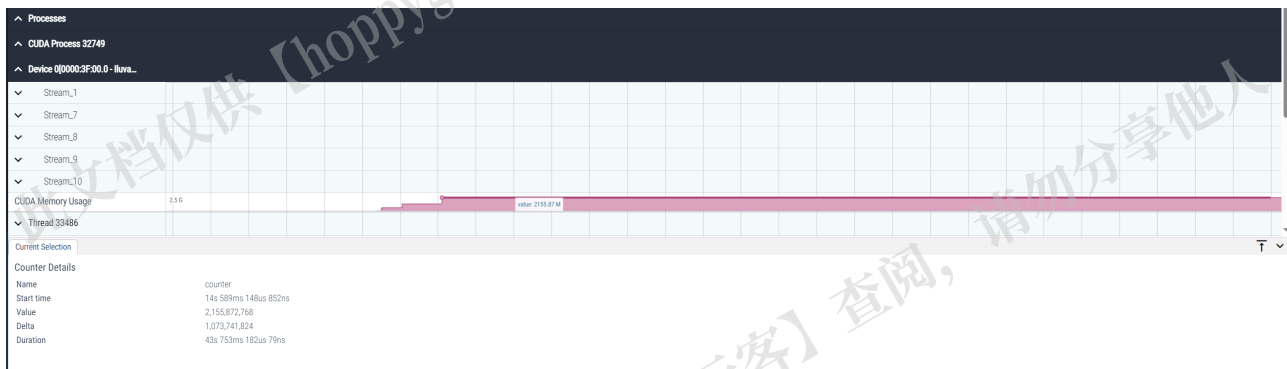


图 21: memory counter 详情

区域选择一段 counter 区域，在页面下方会出现 Counters 统计数据。表格中对每个 memory counter track 会有一条统计数据，描述该区域内的 track 内 counter 出现的次数，最大值，最小值，平均速率，加权值，Delta 值等数据。

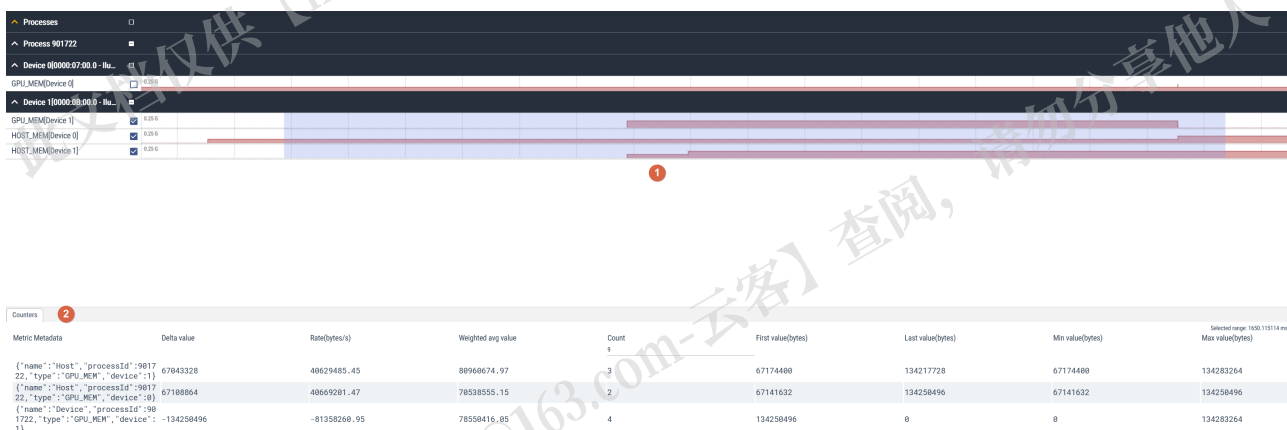


图 22: memory counter 详情

当用户使用 ixSYS 抓取 GPU 能耗数据时，ixSYS-UI 可以显示出相应 GPU 卡的能耗情况。当鼠标悬停时，可显示当前时间的功耗瓦数。

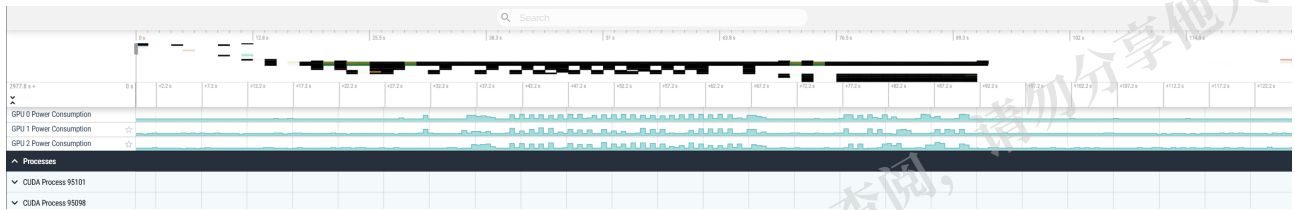


图 23: GPU 卡能耗显示图

Compute/Communication 图会自动出现在使用 NCCL 进行通信的 DDP 训练或者推理的 trace 数据中。点击选中一段感兴趣的区域，被选中的区域内 slice 的统计数据将出现在屏幕的底端，分别可以有轴聚合表格，聚合表，Top slice 表，以及通信/计算统计数据页面 (详情参考下图)。用户可以点击通信/计算统计数据页面来查看选择时间段的通信、计算、overlap 和闲置的占比，以及占比饼图。

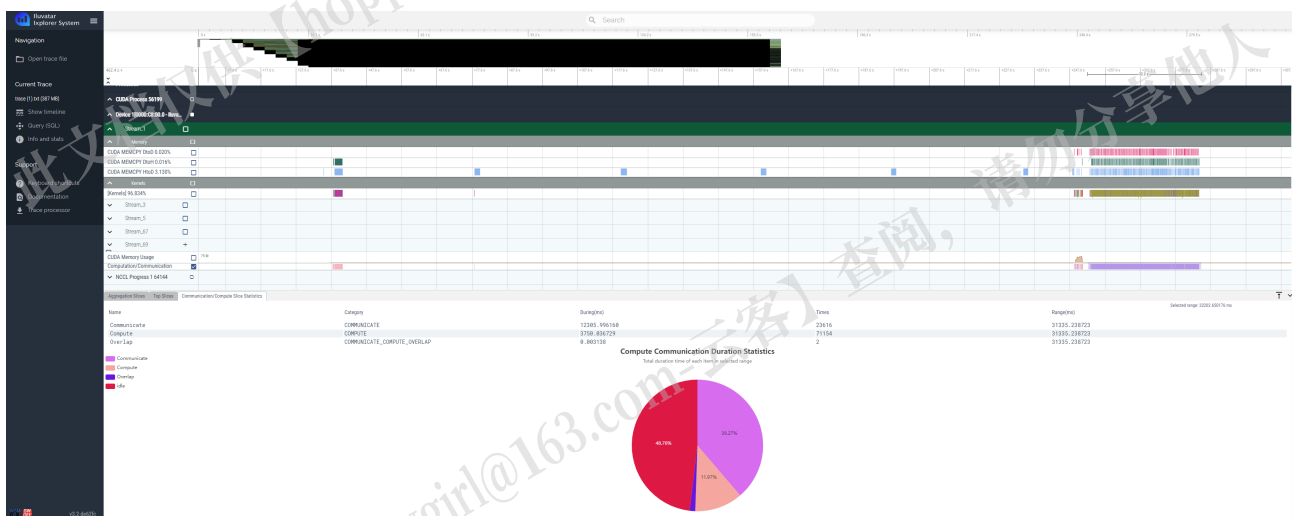


图 24: Compute/Communication 统计图

5.3.3.2 选择单个 slice

鼠标点击选中一个 slice，对应的 slice 边框加粗并显示白色三角号 Marker，在页面的底端会出现关于 slice 的详情描述。

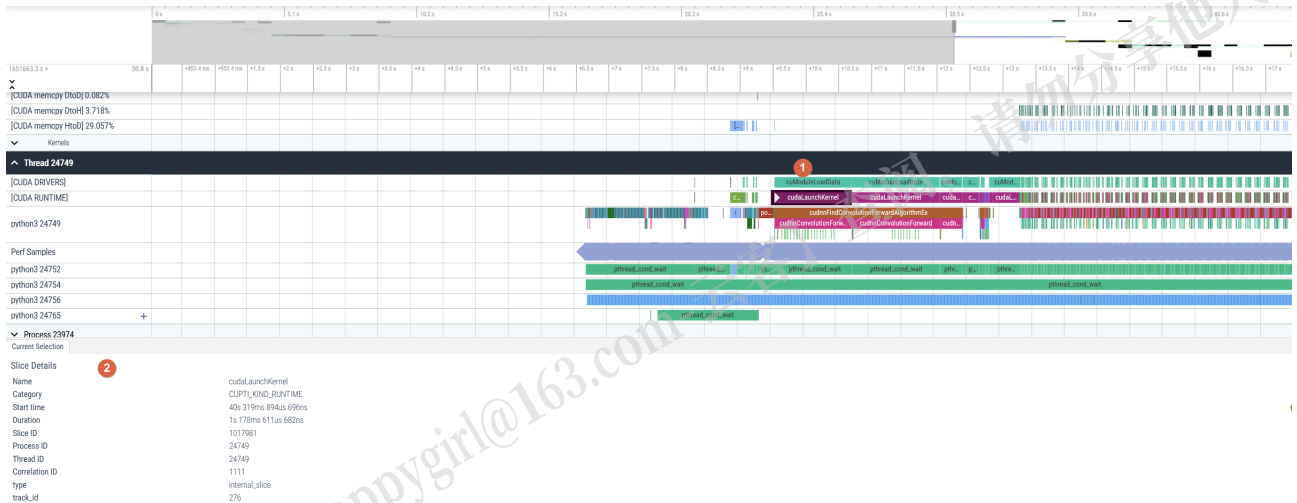


图 25: 选择单个 slice 查阅详情

5.3.3.3 跳转至 correlation slice

对于 Device 中的 slice, 选中 slice 后, 可以通过该 slice 的详情菜单跳转至 correlation slice, 点击 correlationId 后的按钮, 进行跳转, 页面将会找到对应的 slice, 并将其 group 展开选中 slice。在对应的两个 correlation slice 从 source slice 到 destination slice 会有连线进行标识。

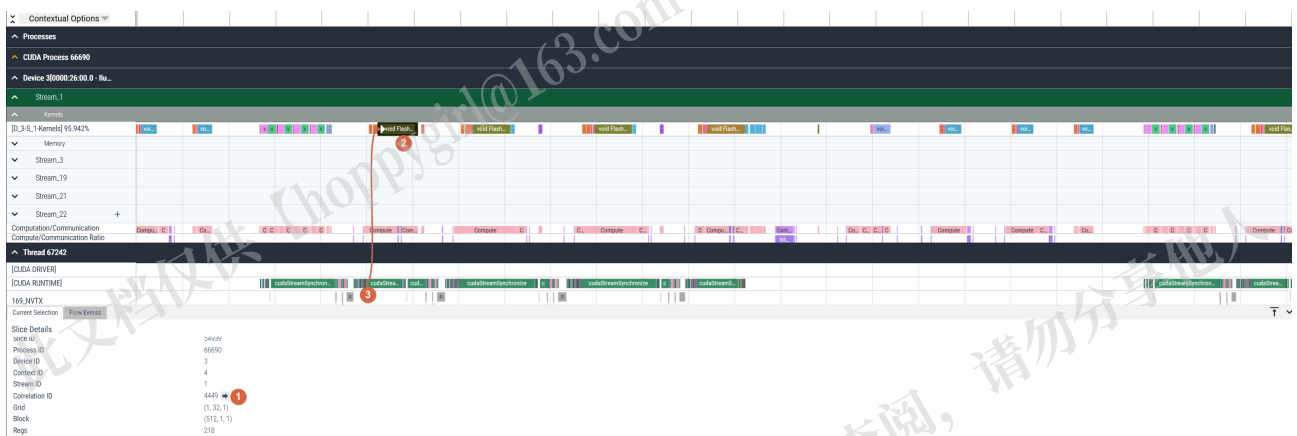


图 26: 跳转至 correlation slice

5.3.3.4 置顶 track

可以通过点击空心五星按钮, 将关心的 track 置顶至 time line 下方 (Pin to Top), 方便您对比查看, 通过点击实心五星按钮, 将 track 从上侧 pin 区域移除。具体操作步骤如下:

1. 点击标识 2 空心五星按钮, 点击后变成标识 1 所示的实心五星按钮
2. 置顶生效后, 选中的 track 数据会在标识 3 所示的区域显示
3. 如需取消置顶可点击标识 3 或者标识 2 的实心按钮

4. 当 pinned track 较多时将会出现可以支持上下滚动的滚动条，用户可以拖动区域修改显示区域

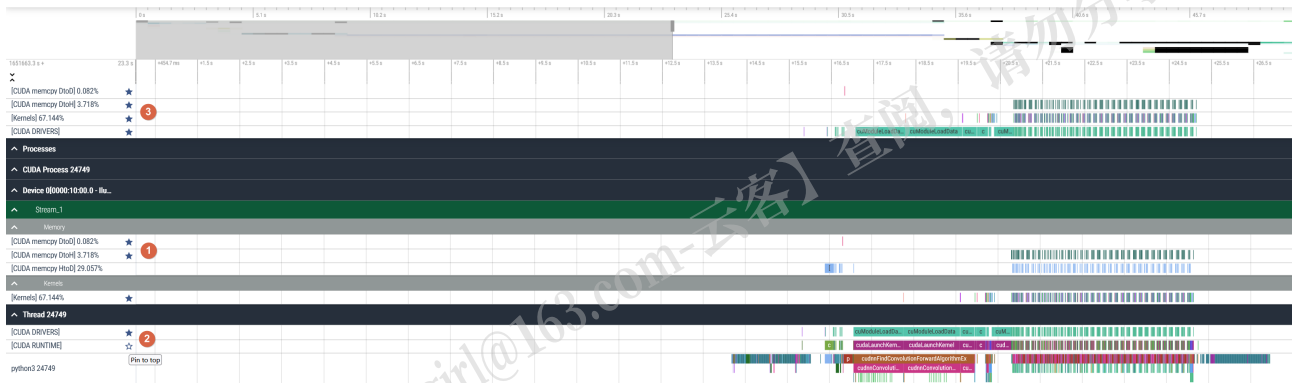


图 27: 将关心的 track 置顶

5.3.3.5 展开或折叠 track 和 group

1. 点击下图所示的加号按钮可以增加 5 个 track 或者 group
2. 点击下图所示的减号按钮可以折叠 5 个 track 或者 group

默认情况下，track 只显示前 5 个，group 只显示 2 个，您可以根据自己的需求进行展开或者折叠。

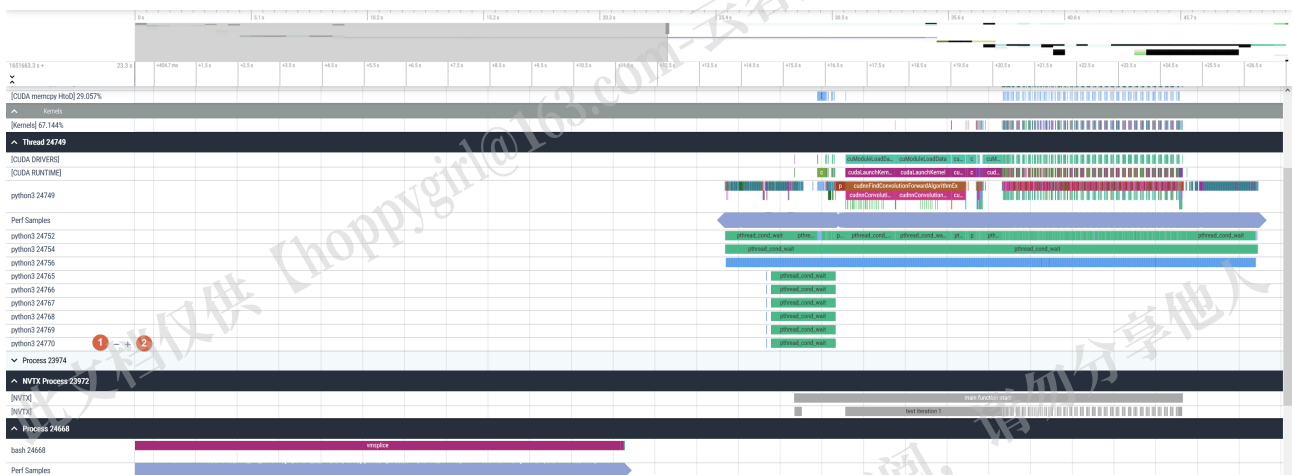


图 28: 展开或折叠 track 和 group

5.3.3.6 显示/隐藏无 kernel stream

在 Timeline 左上角，有下拉菜单支持隐藏无 kernel 的 stream 和显示全部 stream。用户可以根据自己的需要进行切换。

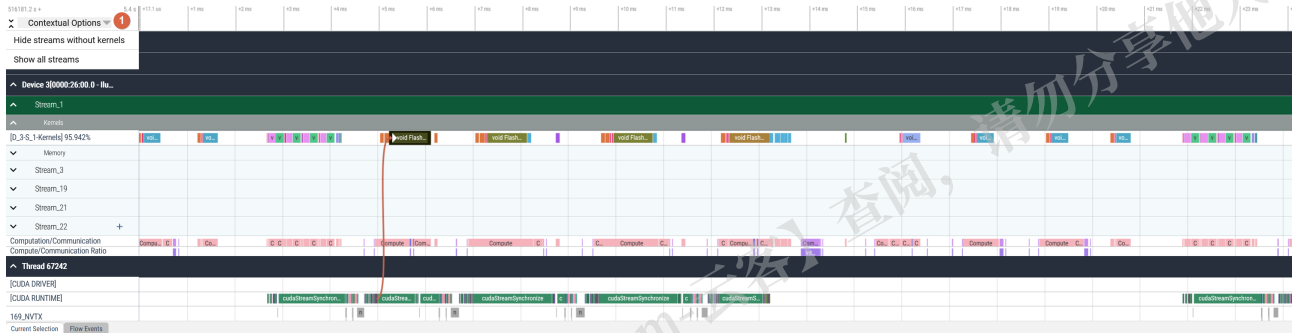


图 29: 显示或隐藏 stream

5.3.3.7 查看 callstack trace

打开对应 Process group，找到 Perf Sample，选择一个 sample 采样点，对应的 callstack 数据将会在页面的下端展示，如有 stack 显示为 unknown，可能是由于无法获取符号表造成。

具体操作如下：

1. 找到对应的 callstack track，选中目标点
2. 对应的 callstack 数据将显示在页面的底部
3. 如果您想要定位到某个函数名，可在标识 3 处的输入对应的函数名，进行检索

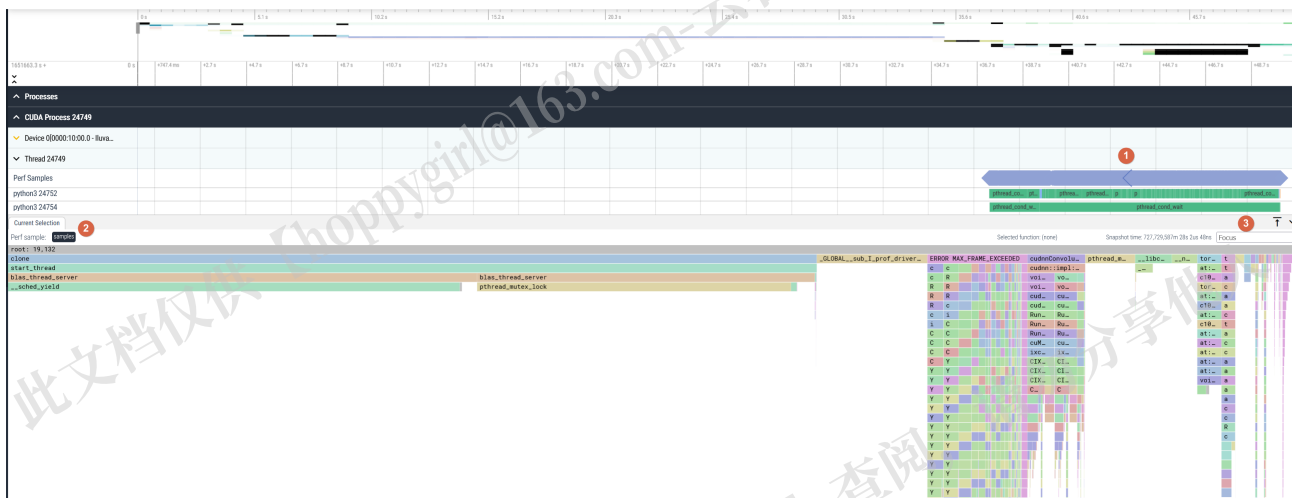


图 30: 查看 callstack trace 的步骤

5.3.3.8 分析 callstack 数据

随机选取一个点，详细的数据如下图所示：



图 31: 随机选取一个点查看详情

任意选取一个函数进行分析，以函数 `_PyFunction_Vectorcall` 为例分析其调用栈如下图所示：

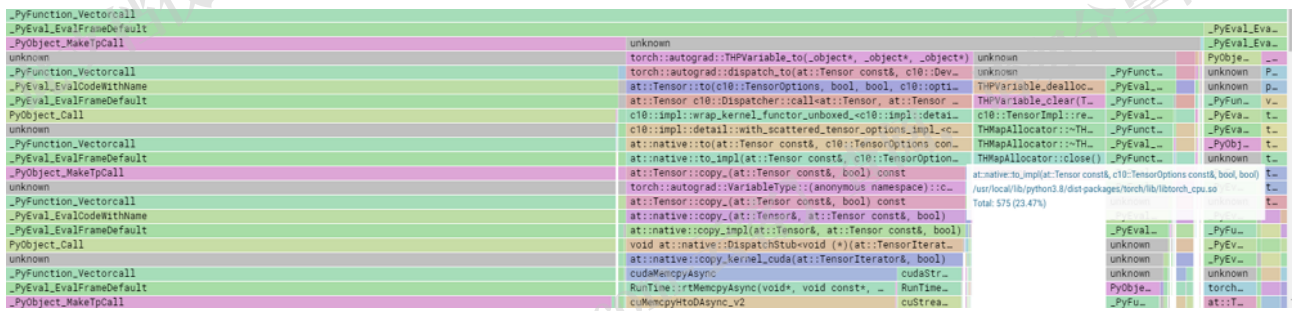


图 32: 以函数 `_PyFunction_Vectorcall` 为例分析其调用栈

每一个矩形代表一个函数，矩形宽度代表函数的开销，宽度越大，开销越大。将鼠标放到矩形框内，会显示函数所占用的百分比。

总结 `_PyFunction_Vectorcall` 调用栈关系如下：

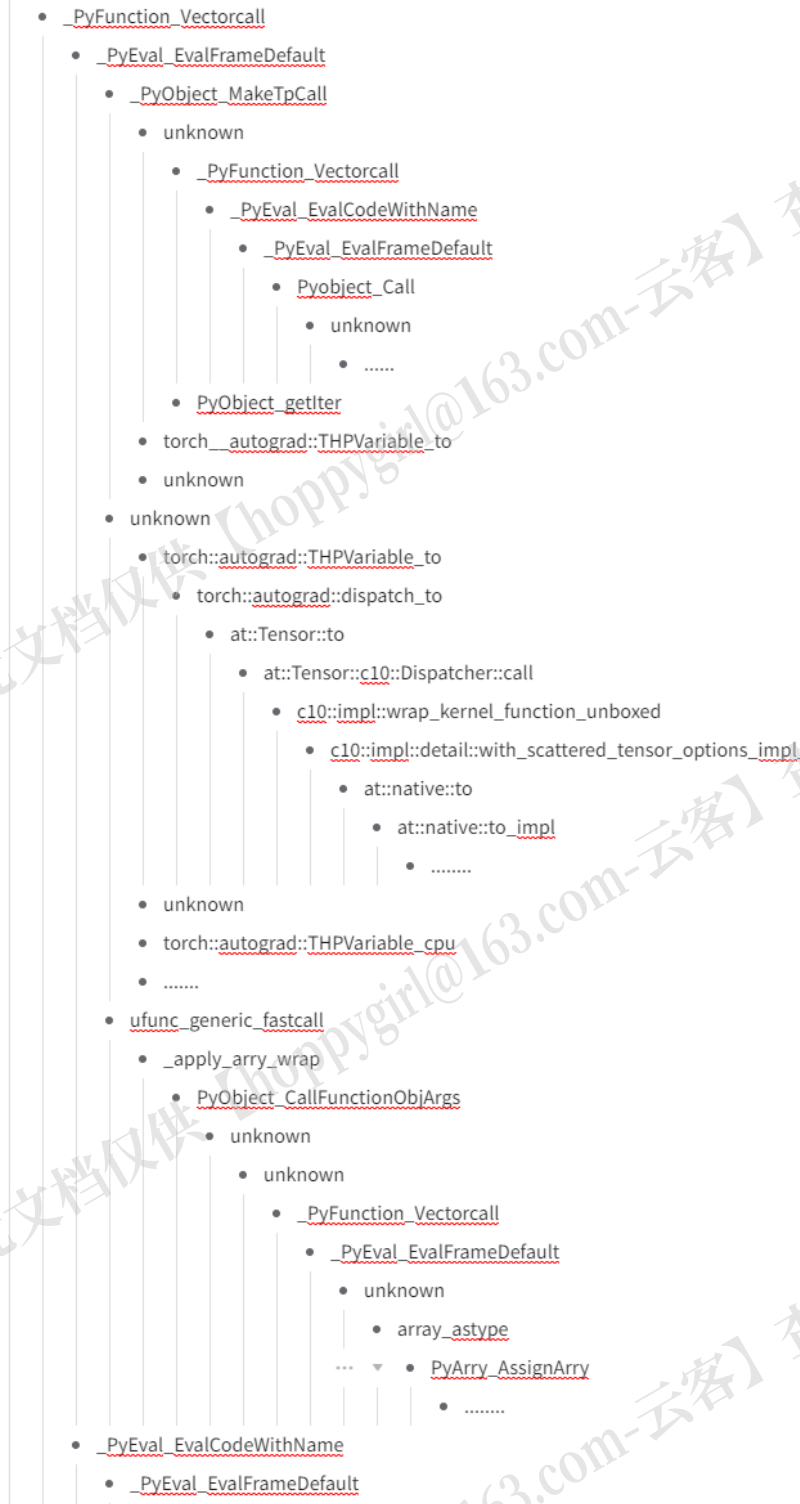


图 33: _PyFunction_Vectorcall 调用栈关系总结示例

5.3.3.9 置顶栏命令菜单

在主界面的搜索栏中，除了搜索功能外，SDK v4.2.0 版本还额外支持命令菜单和 SQL 模式功能。

Search or type '>' for commands or '?' for SQL mode

图 34: 搜索栏示意图

在用户输入">"时，搜索栏将出现阈值命令菜单，用户可以通过上下键来选中菜单，使用 **Enter** 键来触发菜单，使用 **Esc** 键来隐藏菜单。

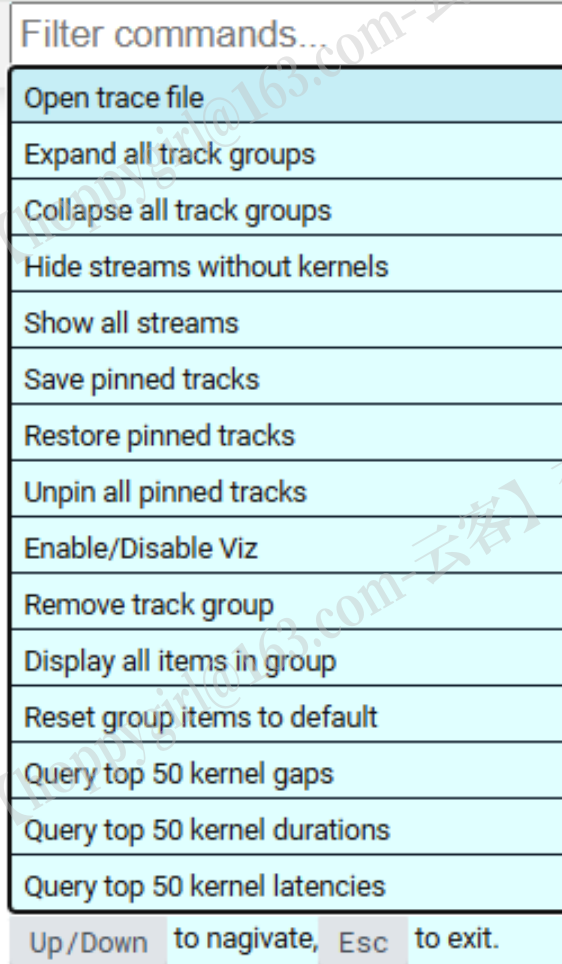


图 35: 搜索栏命令菜单示意图

命令菜单预置功能如下：

- Open trace file: 打开 trace 文件导入页面
- Expand all track groups: 展开全部群组
- Collapse all track groups: 折叠全部群组
- Hide streams without kernels: 隐藏没有 kernel 的 stream 群组
- Show all streams: 显示全部 stream 群组
- Save pinned tracks: 缓存当前 Pinned track
- Restore pinned tracks: 恢复缓存的 Pinned track
- Unpin all pinned tracks: 取消全部 Pinned track

- Enable/Disable viz: 打开或者关闭 Viz 功能，Viz 功能可以支持专家用户使用导入的源数据对柱状图或饼图进行简单的绘制
- Remove track group: 移除指定的群组
- Display all items in group: 显示群组的全部子集
- Reset group items in group: 仅显示默认数量子集
- Query top 50 kernel gaps: 显示指定设备前 50 大的 kernel gap
- Query top 50 kernel durations: 显示指定设备以及 stream 内的耗时最长的 50 个 kernel
- Query top 50 kernel latency: 显示指定设备以及 stream 内的延时最长的 50 个 kernel

5.3.3.10 Viz 功能

在专家用户使用 ixSYS-UI 时，可以通过开启 Viz 功能进行统计绘图，这要求用户非常了解 ixSYS 的源数据格式。目前仅支持绘制 bar 和 pie 两种类型的图标。

触发执行方式：在文本框中输入描述后按下 **Ctrl + Enter** 键。

```
{
  "data": {"url": "select name,dur from slice group by name order by dur DESC limit 5"},
  "title": "Pie chart for Test",
  "mark": "pie",
  "encoding": {
    "x": {"field": "name"},
    "y": {"field": "dur"}
  }
}
```

图 36: Viz 文本框输入示意图

上图中，

- url 数据为您待绘制数据的 SQL 查询语句
- mark 为支持的图表，pie 或 bar 类型
- encoding 中的 x 为属性名，y 为该属性对应的统计指标数值

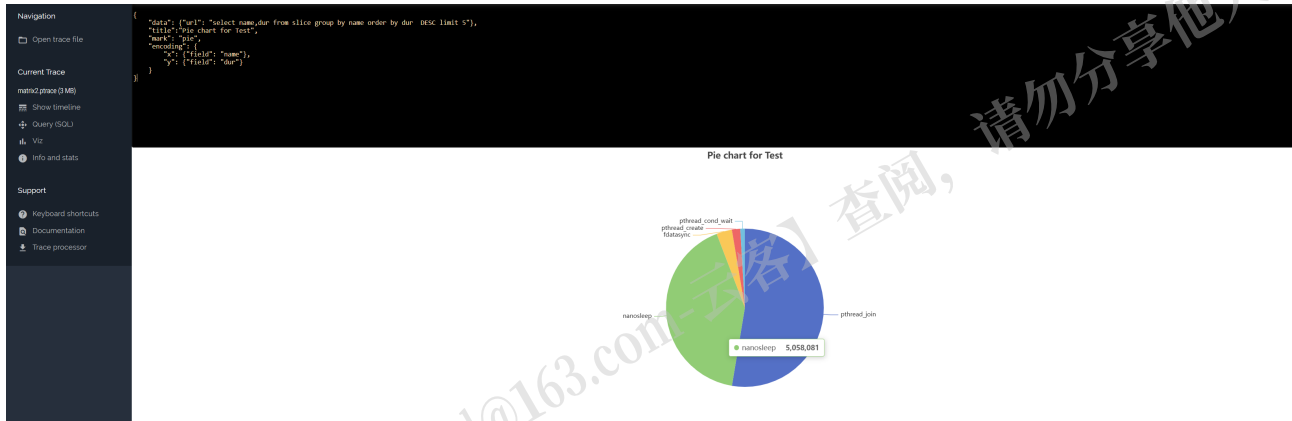


图 37: Viz 数据统计功能示意图

5.3.3.11 Event Sync 功能

在开发人员使用 Event Sync 功能时，通过点击对应的 cudaEventRecord slice 即可将同步的 cudaStreamWait-Event 关联起来，用户可以显式地观察到同步信息是否正确。

5.3.3.12 ixSYS 命令生成器

在 UI 左侧工具栏点击 **ixSYS CMD Generator**，进入生成器页面，用户可以在生成器的协助下生成可以正确运行的命令和参数。生成器会根据用户的选择自动处理命令参数之间的依赖情况。



图 38: ixSYS 命令生成器功能示意图

5.3.4 常见问题

问题 1. 导入文件时报错：大文件导入辅助工具 trace 没有关闭。

解决方法

大文件导入辅助工具 trace 没有关闭，如下图所示，标注 1 所示提示 table 已存在 Error。您需要关闭退出导入 trace_processor_shell 辅助工具，重新刷新页面并导入文件。

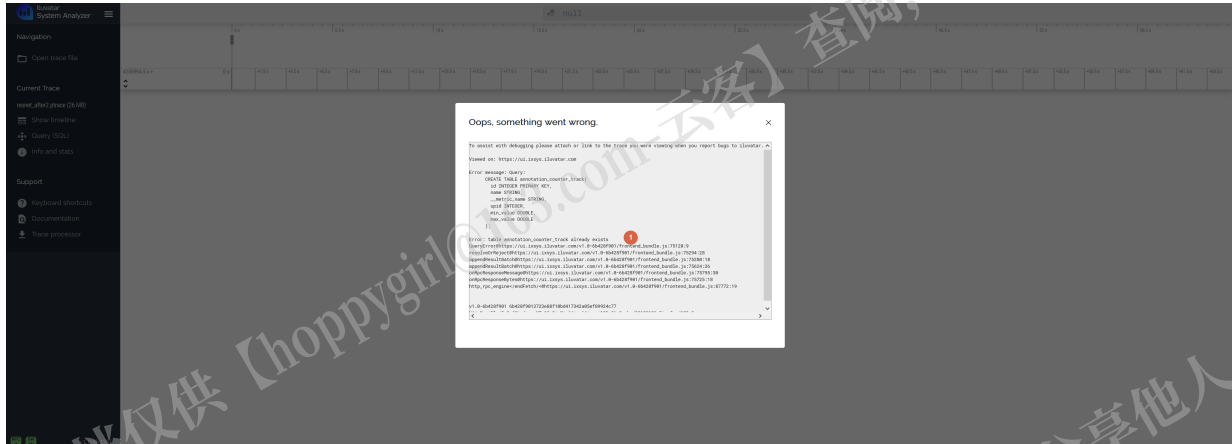


图 39: Table 已存在错误

问题 2. 导入文件时报错：ixSYS 图形界面工具处于离线工作模式。

解决方法

ixSYS 图形界面工具处于离线工作模式，如下图标注 1 所示 SW On 绿色状态，而 trace 文件是最新版本的 ixSYS 命令行工具抓取，版本不匹配。您需要双击 SW On 按钮，在弹出对话框选择 **Disable and reload**，然后重新双击红色 SW On 按钮，将该功能启用。

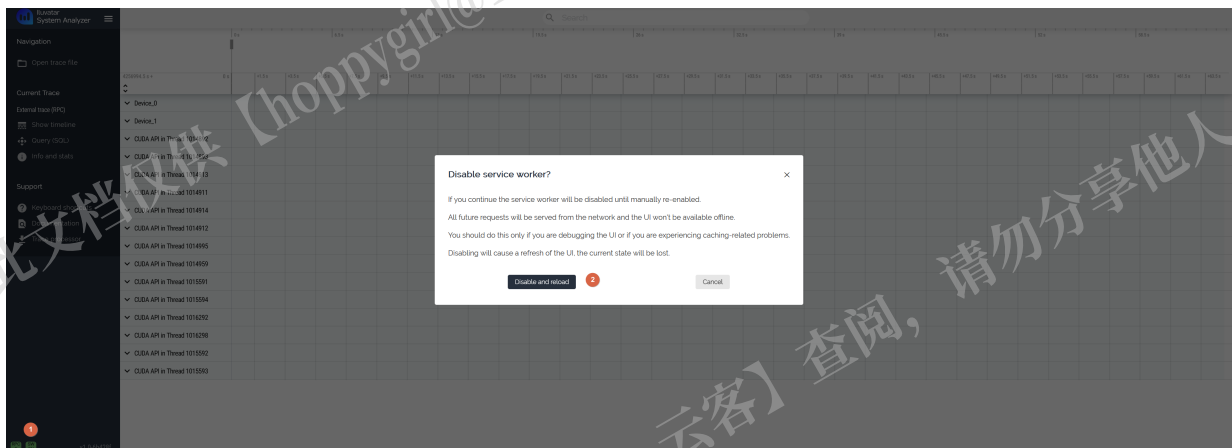


图 40: SW On 绿色状态

问题 3. 超大文件刷新 web 页面时报错。

问题分析

通过辅助工具导入大文件后，刷新 web 页面出现下图所示的错误，可能是由于刷新页面时 URL 包含不存在的 traceID 造成的，如 https://ui.ixsys.iluvatar.com/#!/viewer?trace_id=f8d3e298-0f17-c68c-b44e-152ecb7f4aed 而非在 URL 为 <https://ui.ixsys.iluvatar.com> 时进行的页面刷新。



图 41: 导入大文件后刷新 web 页面报错

解决方法

URL 更新为 <https://ui.ixsys.iluvatar.com> 再进行页面刷新

问题 4. 超大文件导入后 web 页面卡顿。

解决方法

一般而言，如果对于 1GB 的 trace 文件，建议在不小于 16GB 内存的 PC 上查看，2GB 或者更大文件，建议在性能较强的服务器环境中使用。

问题 5. 鼠标点击无法使用。

问题描述

鼠标表现为四方向箭头，无法选中 slice 或者进行区域选择。

解决方法

按 Ctrl 键。

问题 6. 在使用 trace_processor_shell 工具时，用户区域选中 slice 时报错。

问题描述

在用户使用 trace_processor_shell 工具打开较大的 trace 文件后，用户使用鼠标区域选择大量的 trace 内容时，有可能会出现如下错误：

```
A TraceProcessor RPC error occurred
The trace processor RPC sequence ID was broken
This can happen when using a HTTP trace processor instance and either accidentally sharing this between multiple tabs or restarting the trace processor while still in use by UI.
Please refresh this tab and ensure that trace processor is used at most one tab at a time.
```

图 42: 使用 trace_processor_shell 工具报错

解决方法

刷新页面后，尽可能选择精确的区域，减少选中的数据量。

5.4 3. ixSYS 工具最佳实践

5.4.1 GPU Profiling

GPU Profiling 主要是追踪 CUDA 相关的操作，支持 Host (CPU) 和 Device (GPU) 两侧的追踪。

CPU 侧追踪 Runtime 和 Driver 的 API (常见的有模块加载，内存分类，Kernel 加载，内存拷贝和设置，同步)，以及 **cuDNN 函数列表**。

GPU 侧追踪每个 Kernel 的运行耗时寄存器和 Shared Memory 等消耗，另外还会追踪 Memcpy 和 Memset。GPU 侧追踪 Memcpy，HtoD 和 DtoH 会关联 CPU 侧和 GPU 侧，DtoD 和 PToP 是 GPU 内或 GPU 间的操作。GPU 侧追踪 Memset，CPU 可以控制初始化成何种数据 (set 的值)。

通过不同的配置命令行参数可以选择需要获取 Host 侧数据或 Device 侧数据。如果关注的是 GPU 上的行为，则需要获取 Device 侧数据，这部分的数据获取带来的性能开销会较大，特别是多线程的场景。

使用如下命令分析应用：

```
$ ixsys -o ${trace文件名称} -t osrt ${运行的应用} # -t osrt 表示抓取 OSRT
```

下面以 ResNet50 训练程序 (使用 PyTorch) 为例，对 ixSYS 支持 GPU Profiling 进行说明：

```
# bash train_resnet50_torch.sh 表示训练程序使用该脚本启动
$ ixsys -o resnet50_cuda_cudnn_osrt_power_mem.trace -t cuda,cudnn,osrt,power,mem bash
↪ train_resnet50_torch.sh
```

其中，

- -o 后的参数是保存的 trace 名称，无后缀要求，名字和后缀能尽量便于区分即可
- -t 后的参数需要用逗号分隔，指的是待抓取的 trace 类型，包括：
 - cuda，包括 CUDA API 和 GPU Kernel 信息
 - cudnn，包括 cuDNN 库的 API
 - osrt，值 CPU 信息
 - power，值 GPU 功耗
 - mem，指分析程序的内存使用情况
- --profiler-from-start TEXT:{off,on}，选择是否从应用启动时进行 Profiling，默认值为从启动时开始 (ON)。设置为 OFF 时，可使用 {cu,cuda}Profiler{Start,Stop} 开启或停止 Profiling。
 1. 修改程序源码，在代码中使用 cudaProfilerStart() 和 cudaProfilerStop() 标记代码片段并编译，详情请咨询天数智芯应用工程师。
 2. 执行 **ixsys --profile-from-start off -t cuda -o output ./my_app** 命令。

程序运行开始时打印提示信息，如哪些 trace 是正要获取的。如下图所示，true 标识的都是正要获取的：

```
**** collection configuration ****
    trace_cuda = true
    trace_osrt = true
    trace_nvtx = false
    trace_cudnn = true
    trace_video = false
    trace_power = true
    trace_raw_syscalls = false
    trace_metric_sampling = false
    trace_callstack = false
    trace_mem = true
    application command = bash
    application arguments = train_resnet50_torch.sh
```

图 43: 将要获取的 trace

下图所示的 3 个“python3” 标签中，主要是 CPU 侧的 trace，可以看到 cuDNN API, ioctl syscall, pthread 等操作，以及 CUDA Driver 和 Runtime 的 API:

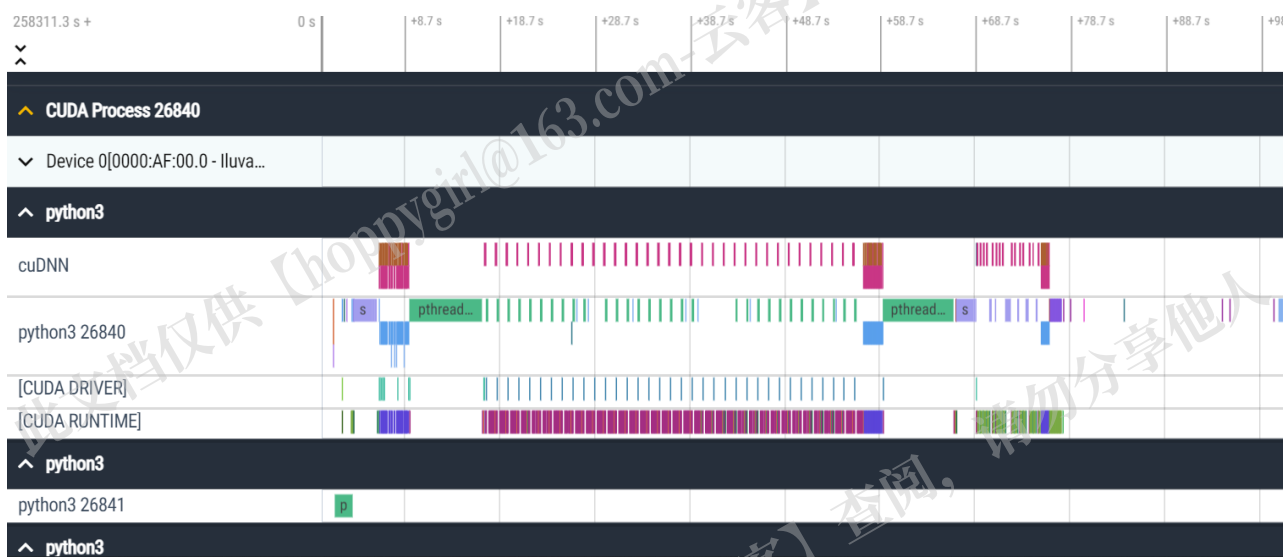


图 44: CPU 侧的 trace 信息

如果您需要更细致的观察，请使用 w, s, a, d 键缩放整个界面。放大之后，可以看到 cudnnFindConvolution-ForwardAlgorithmEx, cudaEventSynchronize, 且中间还有细小的 slice:

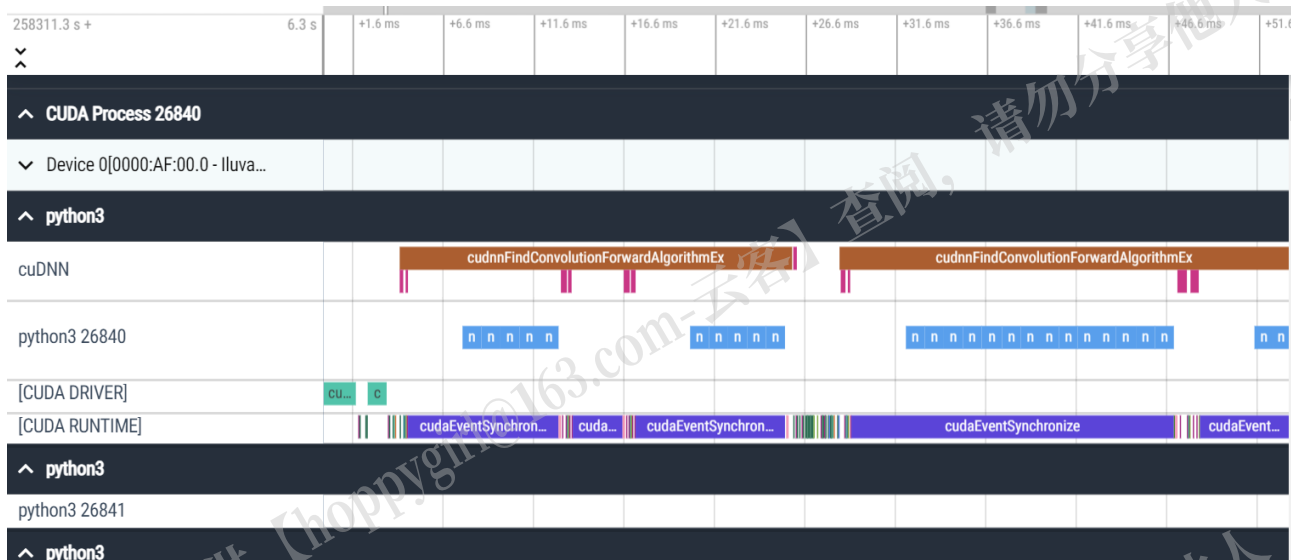


图 45: 放大 CPU 侧 trace 信息

当细小的 slice 过多，除了放大操作，您还可以宏观统计把控。框选住关心的 slices，点击右下角的向上箭头：

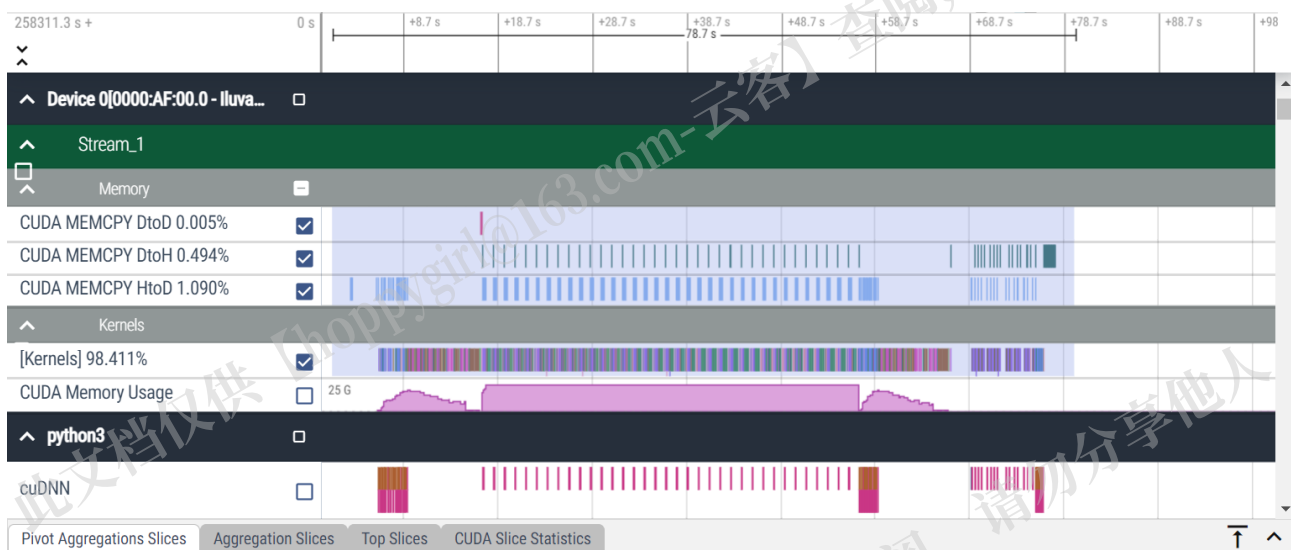


图 46: 宏观统计 CPU 侧 trace 信息

此时，您可以观察统计信息。特别是持续期 (Duration) 的统计，更容易看出一段时间内选中的 slices 的占比，能够看出是否充分使用了 CPU/GPU：

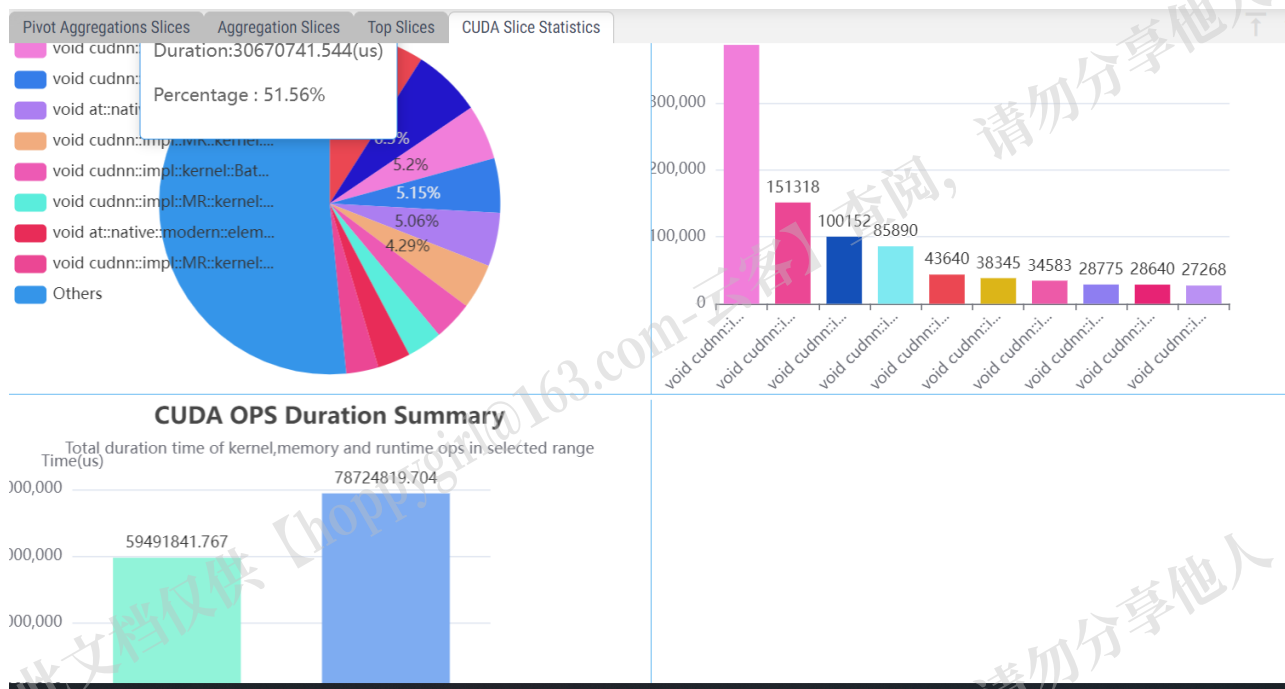


图 47: 详细统计信息

GPU 相关的信息一般跟随设备显示。总的会有 GPU 的消耗，还有下面设备的内存使用情况，不同 CUDA stream 的操作。

CPU 侧的 Memcpy 和 Kernel Launch 都是给 GPU 发命令，真正的执行是 GPU 的行为。在 Profiling 时，追踪的是分析的进程的行为，如果有其它的进程同时在使用 GPU，则不能被直接追踪到。在分析时，整体的功耗会有反应，命令的延迟也可能会增大。GPU 的功耗为采样所得，Memory Usage 是追踪到的被分析进程分配的 GPU 内存。

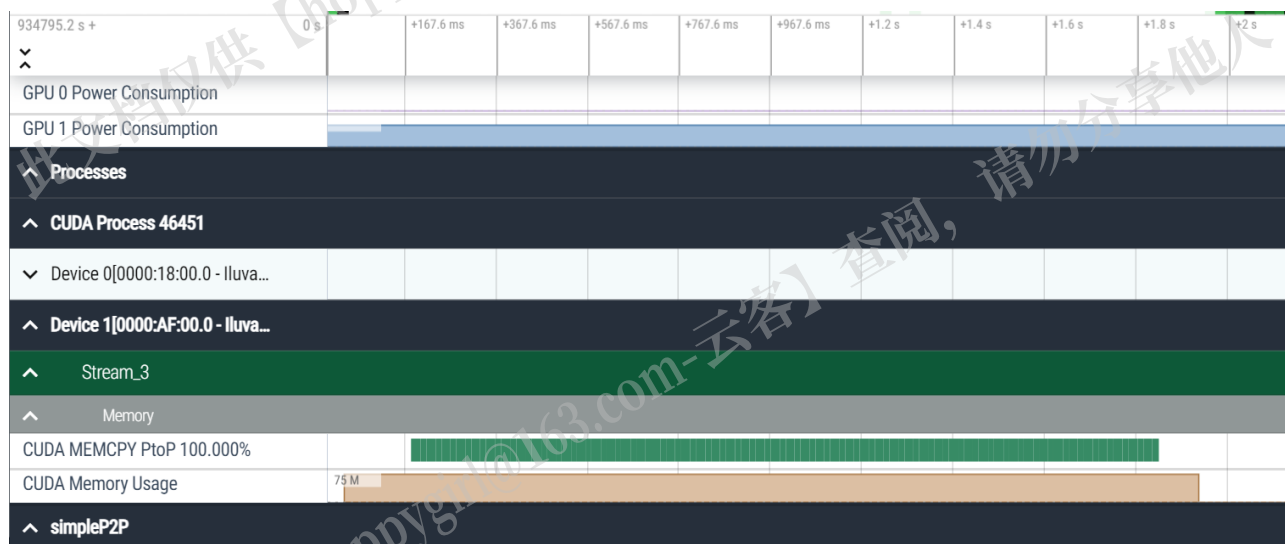


图 48: GPU 侧相关信息

5.4.2 CPU Profiling

ixSYS 工具在 CPU Profiling 方面主要支持 OSRT 和 callstack 两个功能。

5.4.2.1 OSRT

在 Linux 上，可以跟踪操作系统运行时库以收集有关低级用户空间 API 的信息。只是部分跟踪 C 运行时库和 POSIX 线程 (pthread) 库中的 API，如系统调用的 API 和线程同步的 API，关注执行时间长的 API，或可能导致线程进入等待状态的 API，并不会跟踪全部的 API。

操作系统运行时跟踪不适用于 Windows。

操作系统运行时跟踪通过以下方式补充和增强采样信息：

1. 可视化进程何时与硬件通信、控制资源、执行多线程同步或与内核调度程序交互。
2. 通过关联操作系统运行时库跟踪如何影响线程调度来添加额外的线程状态：
 - 等待：线程未在 CPU 上调度，它位于 OS 运行时库跟踪中，并且被认为正在等待固件完成请求。
 - 在 OS 运行时库函数中：线程在 CPU 上和 OS 运行时库跟踪内部调度。如果跟踪表示系统调用，则该进程很可能在内核模式下运行。
3. 收集长 OS 运行时库调用的回溯。这提供了一种收集阻塞状态回溯的方法，使您能够获得更多关于为什么线程被阻塞这么长时间的上下文，同时避免短事件的不必要开销。

使用如下命令获取 trace：

```
$ ixsys -o ${trace文件名称} -t osrt ${运行的应用} # -t osrt 表示抓取 OSRT
```

下面是分析一个 OSRT 相关的 trace 信息的例子，可以看到对应的函数调用的次数、执行时间等信息。

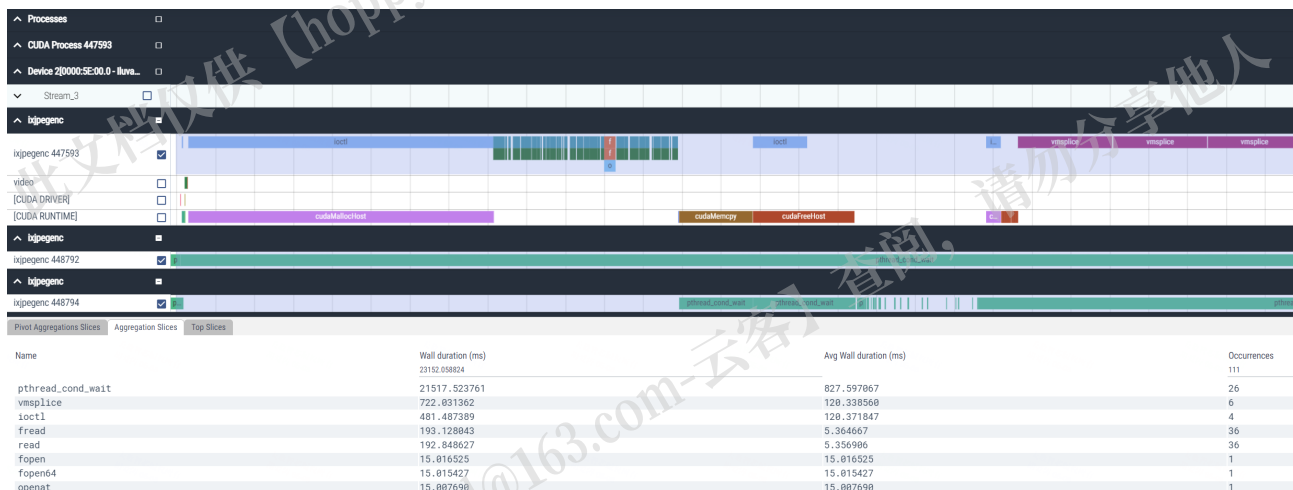


图 49: OSRT 相关的 trace 信息示例

5.4.2.2 callstack

分析函数的调用栈是常用的性能分析和调试手段之一。ixSYS 支持抓取函数的调用栈，用火焰图的方式来展示函数的调用关系与性能开销。

使用如下命令获取 trace：

```
$ ixsys -o ${trace文件名称} -c ${运行的应用} # -c 表示抓取 callstack
```

下面是一个调用栈信息显示的示例：

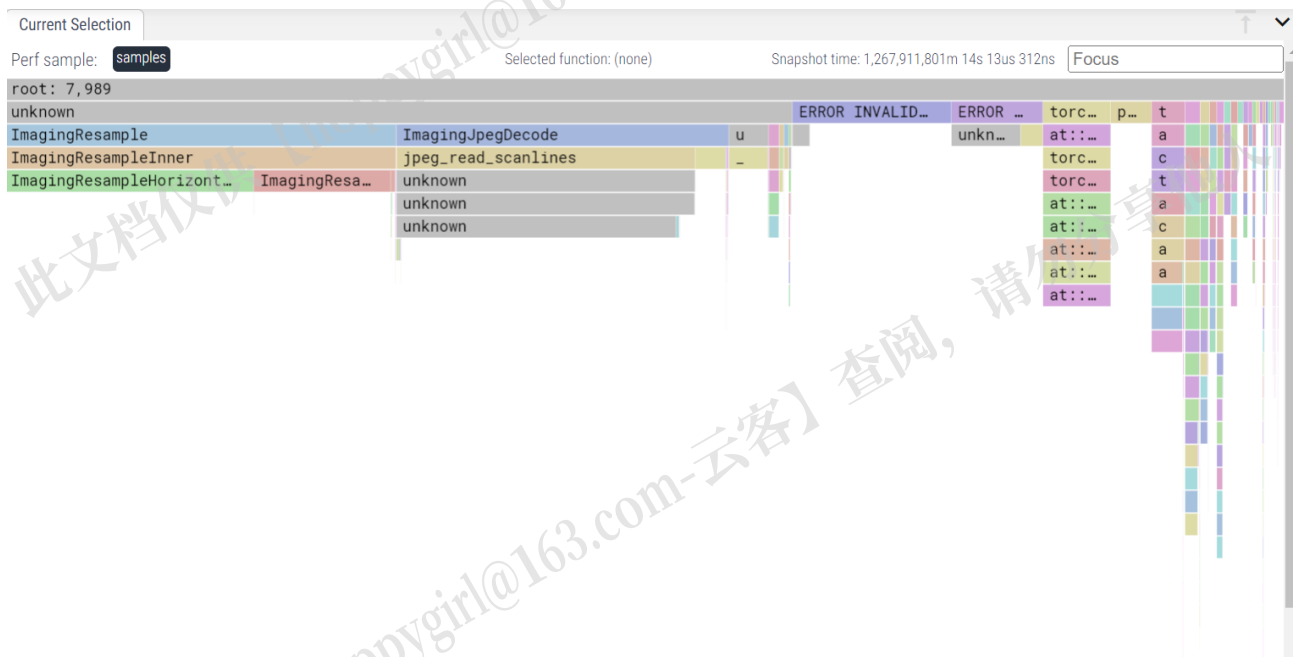


图 50: 调用栈信息显示示例

5.4.3 管理 ixTX 上报的数据

Iluvatar Explorer Tools Extension (ixTX) 是一个用于代码注释、为开发人员提供上下文信息的工具。

ixTX API 是用 C 语言编写的，兼容 NVTX，使用方法也与 NVTX 保持一致，具体使用方法请参考 [NVTX C API Reference](#)。

ixSYS 可以管理 ixTX 并收集 ixTX 上报的数据，同时保存这些数据到指定的输出文件中，输出的文件可以在 ixSYS 图形工具中进行展示。

5.4.3.1 ixTX 提供的注释类型

1. Markers:

用于注释程序执行中的特定点，可提供可选的额外字段，包括：类别，颜色和有效负载。

2. Ranges:

Ranges 用于标注程序执行中的某段代码，主要包括两种类型：

- Push/Pop Ranges，可以嵌套堆栈，Pop 自动与同一线程上先前的 Push 调用相关联
- Start/End Ranges，可以任意地与其它 Range 重叠，Start 返回一个句柄，该句柄必须传递给 End 调用，用于确定对应的 Start。Start 和 End 可以在不同的线程上。

3. Resource Naming/tracking:

资源命名是将一个字符串与一个对象关联起来。例如，命名 CPU 线程允许在时间轴上用字符串来显示该线程的活动，比显示线程 ID 更有意义。目前仅支持 CPU thread Naming。

5.4.3.2 ixTX Domain 介绍

Domain 的作用是将事件进行分组，将指定的事件分到用户定义的范围。开发人员可以利用 Domain 限定自己的事件范围，可以避免与其它组件的事件发生冲突，便于开发人员轻松区分和过滤自己关心的事件。用于可以通过 `nvtxDomainCreateA` 创建 Domain，通过 `nvtxDomainDestroy` 销毁创建的 Domain (当前版本并未真正销毁，销毁后的 Domain 仍可继续使用)。

以下接口使用默认的 Domain：

- `nvtxMarkEx`
- `nvtxMarkA`
- `nvtxRangeStartEx`
- `nvtxRangeStartA`
- `nvtxRangeEnd`
- `nvtxRangePushEx`
- `nvtxRangePushA`
- `nvtxRangePop`

以下接口使用指定的 Domain：

- `nvtxDomainMarkEx`
- `nvtxDomainRangeStartEx`
- `nvtxDomainRangeEnd`
- `nvtxDomainRangePushEx`
- `nvtxDomainRangePop`
- `nvtxDomainRegisterStringA`

当使用 ixSYS 显示数据时，只有相同 Domain 的事件才会显示在同一个 track 中。

5.4.3.3 ixTX 使用示例

1. ixTX 使用的示例如下：

```
#include <float.h>
#include <limits.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include "nvToolsExt.h"
#include "stdio.h"
#include "cuda.h"
#include "cuda_runtime.h"

#define DELAY_RANGE() usleep(100 * 1000); // sleep 100 ms
#define DELAY() usleep(10 * 1000); // sleep 10 ms

static const uint32_t COLOR_RED = 0xFFFF0000;
static const uint32_t COLOR_GREEN = 0xFF00FF00;
static const uint32_t COLOR_BLUE = 0xFF0000FF;
static const uint32_t COLOR_YELLOW = 0xFFFFFFFF00;
static const uint32_t COLOR_MAGENTA = 0xFFFF00FF;
static const uint32_t COLOR_CYAN = 0xFF00FFFF;

static void simple_nvtxMarker()
{
    // nvtxMark{A,W}

    nvtxMarkA("simple_nvtxMarker:nvtxMarkA");
    DELAY();
    // nvtxMarkEx

    // zero the structure
    nvtxEventAttributes_t eventAttrib = {0};

    // set the version and the size information
    eventAttrib.version = NVTX_VERSION;
    eventAttrib.size = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    // configure the attributes. 0 is the default for all attributes.
    eventAttrib.colorType = NVTX_COLOR_ARGB;
    eventAttrib.color = COLOR_RED;
    eventAttrib.messageType = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii = "simple_nvtxMarker:nvtxMarkEx";
    nvtxMarkEx(&eventAttrib);

    DELAY();
}

static void simple_nvtxDomainMarker()
{
    // nvtxDomainMarkEx
    nvtxDomainHandle_t domain =
    ↪ nvtxDomainCreateA("simple_nvtxDomainMarker:nvtxDomainMarkEx");
    // nvtxMarkEx

    // zero the structure
    nvtxEventAttributes_t eventAttrib = {0};

    // set the version and the size information
```

```

eventAttrib.version = NVTX_VERSION;
eventAttrib.size    = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

// configure the attributes. 0 is the default for all attributes.
eventAttrib.colorType = NVTX_COLOR_ARGB;
eventAttrib.color     = COLOR_RED;
eventAttrib.messageType = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib.message.ascii = "simple_nvtxMarker:nvtxDomainMarkEx";
nvtxDomainMarkEx(domain, &eventAttrib);

DELAY();
}
static void simple_nvtxDomainRangePushPop()
{
    // nvtxDomainRangePushEx
    nvtxDomainHandle_t domain =
    ↪ nvtxDomainCreateA("simple_nvtxDomainRangePushPop:nvtxDomainRangePushEx");

    // zero the structure
    nvtxEventAttributes_t eventAttrib = {0};

    // set the version and the size information
    eventAttrib.version = NVTX_VERSION;
    eventAttrib.size    = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    // configure the attributes. 0 is the default for all attributes.
    eventAttrib.colorType = NVTX_COLOR_ARGB;
    eventAttrib.color     = COLOR_GREEN;
    eventAttrib.messageType = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii =
    ↪ "simple_nvtxDomainRangePushPop:nvtxDomainRangePushEx";

    nvtxDomainRangePushEx(domain, &eventAttrib);
    DELAY_RANGE()
    nvtxDomainRangePop(domain);

    DELAY();

    // show nested ranges

    // re-use eventAttrib
    eventAttrib.message.ascii = "simple_nvtxDomainRangePushPop:Level 0";
    nvtxDomainRangePushEx(domain, &eventAttrib);
    DELAY();
    eventAttrib.message.ascii = "simple_nvtxDomainRangePushPop:Level 1";
    nvtxDomainRangePushEx(domain, &eventAttrib);
    DELAY_RANGE();
    nvtxDomainRangePop(domain); // Level 1
    DELAY();
    nvtxDomainRangePop(domain); // Level 0

```

```

}

static void simple_nvtxRangePushPop()
{
    // nvtxRangePush{A,W}

    nvtxRangePushA("simple_nvtxRangePushPop:nvtxRangePushA");
    DELAY_RANGE();
    nvtxRangePop();

    DELAY_RANGE();
    /*
        nvtxRangePushW(L"simple_nvtxRangePushPop:nvtxRangePushW");
        DELAY_RANGE();
        nvtxRangePop();
    */
    // nvtxRangePushEx
    DELAY_RANGE();

    // zero the structure
    nvtxEventAttributes_t eventAttrib = {0};

    // set the version and the size information
    eventAttrib.version = NVTX_VERSION;
    eventAttrib.size     = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    // configure the attributes. 0 is the default for all attributes.
    eventAttrib.colorType = NVTX_COLOR_ARGB;
    eventAttrib.color     = COLOR_GREEN;
    eventAttrib.messageType = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii = "simple_nvtxRangePushPop:nvtxRangePushEx";

    nvtxRangePushEx(&eventAttrib);
    DELAY_RANGE();
    nvtxRangePop();

    DELAY();

    // show nested ranges

    // re-use eventAttrib
    eventAttrib.message.ascii = "simple_nvtxRangePushPop:Level 0";
    nvtxRangePushEx(&eventAttrib);
    DELAY();
    eventAttrib.message.ascii = "simple_nvtxRangePushPop:Level 1";
    nvtxRangePushEx(&eventAttrib);
    DELAY_RANGE();
    nvtxRangePop(); // Level 1
    DELAY();
    nvtxRangePop(); // Level 0
}

```

```

}

static void simple_nvtxRangeStartEnd()
{
    // nvtxRangeStart{A,W}

    nvtxRangeId_t id1 = nvtxRangeStartA("simple_nvtxRangeStartEnd:nvtxRangeStartA");
    DELAY_RANGE();
    nvtxRangeEnd(id1);
    /*
        nvtxRangeId_t id2 =
↪ nvtxRangeStartW("simple_nvtxRangeStartEnd:nvtxRangeStartW");
        DELAY_RANGE();
        nvtxRangeEnd(id2);
    */
    // zero the structure
    nvtxEventAttributes_t eventAttrib = {0};

    // set the version and the size information
    eventAttrib.version = NVTX_VERSION;
    eventAttrib.size    = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    // configure the attributes. 0 is the default for all attributes.
    eventAttrib.colorType    = NVTX_COLOR_ARGB;
    eventAttrib.color        = COLOR_BLUE;
    eventAttrib.messageType  = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii = "simple_nvtxRangeStartEnd:nvtxRangeStartEx";

    nvtxRangeId_t id3 = nvtxRangeStartEx(&eventAttrib);
    DELAY_RANGE();
    nvtxRangeEnd(id3);

    // overlapping events
    // re-use eventAttrib
    eventAttrib.message.ascii = "simple_nvtxRangeStartEnd:Range 1";
    nvtxRangeId_t r1         = nvtxRangeStartEx(&eventAttrib);
    DELAY();
    eventAttrib.message.ascii = "simple_nvtxRangeStartEnd:Range 2";
    nvtxRangeId_t r2         = nvtxRangeStartEx(&eventAttrib);
    DELAY_RANGE();
    nvtxRangeEnd(r1);
    DELAY();
    nvtxRangeEnd(r2);

    DELAY();
}

static void simple_nvtxDomainRangeStartEnd()
{
    // nvtxDomainRangeStartEx

```

```

nvtxDomainHandle_t domain =
↪ nvtxDomainCreateA("simple_nvtxDomainRangeStartEnd:nvtxDomainRangeStartEx");
// zero the structure
nvtxEventAttributes_t eventAttrib = {0};

// set the version and the size information
eventAttrib.version = NVTX_VERSION;
eventAttrib.size    = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

// configure the attributes. 0 is the default for all attributes.
eventAttrib.colorType    = NVTX_COLOR_ARGB;
eventAttrib.color        = COLOR_BLUE;
eventAttrib.messageType  = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib.message.ascii =
↪ "simple_nvtxDomainRangeStartEnd:nvtxDomainRangeStartEx";

nvtxRangeId_t id3 = nvtxDomainRangeStartEx(domain, &eventAttrib);
DELAY_RANGE();
nvtxDomainRangeEnd(domain, id3);

// overlapping events

// re-use eventAttrib
eventAttrib.message.ascii = "simple_nvtxDomainRangeStartEnd:Range 1";
nvtxRangeId_t r1          = nvtxDomainRangeStartEx(domain, &eventAttrib);
DELAY();
eventAttrib.message.ascii = "simple_nvtxDomainRangeStartEnd:Range 2";
nvtxRangeId_t r2          = nvtxDomainRangeStartEx(domain, &eventAttrib);
DELAY_RANGE();
nvtxDomainRangeEnd(domain, r1);
DELAY();
nvtxDomainRangeEnd(domain, r2);
DELAY();
}

static void attributes_unversioned()
{
    // The version and size field are used by the Tools Extension implementation
    // to handle multiple versions of the attributes structures. This example
    // shows how to initialize the structure for forwards compatibility.

    // 1. Initialize all fields to 0. 0 is the default value for all fields.
    // This should be done using
    // a. = {0} or
    // b. memset(&eventAttrib, 0, NVTX_EVENT_ATTRIB_STRUCT_SIZE);
    // Do not use ordered structure initialization as future version may
    // change the order of fields but maintain source level compatibility.
    //
    // 2. Configure the version field to NVTX_VERSION which is defined in

```

```
// nvToolsExt.h. The value of this macro will be increased whenever
// a change is made that break binary compatibility.
//
// 3. Configure the size field to NVTX_EVENT_ATTRIB_STRUCT_SIZE. The value
// of this macro will be increased whenever the struct tag is changed.

nvtxEvtAttributes_t eventAttrib = {0};
eventAttrib.version              = NVTX_VERSION;
eventAttrib.size                 = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

eventAttrib.colorType            = NVTX_COLOR_ARGB;
eventAttrib.color                = ::COLOR_YELLOW;
eventAttrib.messageType          = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib.message.ascii        = "attributes_unversioned";

nvtxMarkEx(&eventAttrib);

DELAY();

// This can be done using C99 designated initializers.
#ifdef SUPPORTS_C99_DESIGNATED_INITIALIZERS
nvtxEvtAttributes_t eventAttrib2 = {.version = NVTX_VERSION,
↪ // version
                                .size = NVTX_EVENT_ATTRIB_STRUCT_SIZE,
↪ // size
                                .colorType = NVTX_COLOR_ARGB,
↪ // colorType
                                .color = COLOR_YELLOW,
↪ // color
                                .messageType = NVTX_MESSAGE_TYPE_ASCII,
↪ // messageType
                                .message =
↪ "attributes_unversioned:Designated
↪ Initializer"};

nvtxMarkEx(eventAttrib2);

DELAY();
#endif // SUPPORTS_C99_DESIGNATED_INITIALIZERS
}

static void attributes_versioned()
{
    // The version and size field are used by the Tools Extension implementation
    // to handle multiple versions of the attributes structures. This example
    // shows how to initialize the structure to a specific version of the library.

    // 1. Initialize all fields to 0. 0 is the default value for all fields.
    // This should be done using
    // a. = {0} or
```

```
// b. memset(&eventAttrib, 0, sizeof(nvtxEventAttributes_v1);
// If you hardcode to a specific version of the structure then ordered
// structure initialization can be used.
//
// 2. Configure the version field to a numeric value that matches version
// number in the struct tag (nvtxEventAttributes_v#).
//
// 3. Configure the size field to sizeof(nvtxEventAttributes_v#).
//
// Do not use NVTX_VERSION or NVTX_EVENT_ATTRIB_STRUCT_SIZE.

nvtxEventAttributes_v2 eventAttrib = {0};
eventAttrib.version                = 1;
eventAttrib.size                   = (uint16_t)(sizeof(nvtxEventAttributes_v2));

eventAttrib.colorType = NVTX_COLOR_ARGB;
eventAttrib.color     = COLOR_MAGENTA;

eventAttrib.messageType = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib.message.ascii = "attributes_versioned";

nvtxMarkEx(&eventAttrib);

DELAY();

// This can be done using ordered initialization. C89/99 only supports
// initialization of the first member of a union. This limits payload to
// ullValue and message to ascii. See attributes_unversioned() above
// for an example of C99 designated initializer for alternative syntax
// option. Do not use the typedef nvtxEventAttributes_t with ordered
// initialization, because the typedef is only guaranteed to preserve
// the named members across versions, not necessarily the order or number
// of members.

nvtxEventAttributes_v2 eventAttrib2 = {
    1, // version
    (uint16_t)(sizeof(nvtxEventAttributes_v2)), // size
    0, // category - no limitations
    NVTX_COLOR_ARGB, // colorType - no limitations
    COLOR_CYAN, // color - no limitations
    NVTX_PAYLOAD_TYPE_UNSIGNED_INT64, // payloadType - REQUIRED, can
    only initialize payload.ullValue
    0, // reserved
    1ULL, // payload - type must be
    assignable to uint64_t
    NVTX_MESSAGE_TYPE_ASCII, // messageType - REQUIRED, can
    only initialize message.ascii
    "attributes_versioned:Ordered Initialization" // message - type must be
    assignable to char*
};
```

```

    nvtxMarkEx(&eventAttrib2);

    DELAY();
}

static void attributes_category()
{
    // The category field is used to group events. The default category is 0.
    // The caller is responsible for managing categories numbers.

    nvtxEventAttributes_t eventAttrib = {0};
    eventAttrib.version                = NVTX_VERSION;
    eventAttrib.size                   = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
    eventAttrib.category               = 1;
    // specifying message to help identify this event in the tool.
    eventAttrib.messageType            = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii          = "attributes_category";

    nvtxMarkEx(&eventAttrib);

    // Categories can be named using nvtxNameCategory{A,W}().
    nvtxNameCategoryA(1, "attributes_category");

    DELAY();
}

static void attributes_color()
{
    // The color field is used to help visually identify events in the tool.
    // The caller must specify valid values for both colorType and color.

    // valid specification of color

    nvtxEventAttributes_t eventAttrib1 = {0};
    eventAttrib1.version                = NVTX_VERSION;
    eventAttrib1.size                   = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
    eventAttrib1.colorType              = NVTX_COLOR_ARGB;
    eventAttrib1.color                  = COLOR_RED;
    // specifying message to help identify this event in the tool.
    eventAttrib1.messageType            = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib1.message.ascii          = "attributes_color:valid color";
    nvtxMarkEx(&eventAttrib1);

    DELAY();

    // default color

    nvtxEventAttributes_t eventAttrib2 = {0};
    eventAttrib2.version                = NVTX_VERSION;
    eventAttrib2.size                   = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

```

```
// specifying message to help identify this event in the tool.
eventAttrib2.messageType = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib2.message.ascii = "attributes_color:default color";
nvtxMarkEx(&eventAttrib2);

DELAY();

// invalid specification of color -- missing colorType

nvtxEventAttributes_t eventAttrib3 = {0};
eventAttrib3.version = NVTX_VERSION;
eventAttrib3.size = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
eventAttrib3.color = COLOR_GREEN;
// specifying message to help identify this event in the tool.
eventAttrib3.messageType = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib3.message.ascii = "attributes_color:colorType undefined";
nvtxMarkEx(&eventAttrib3);

DELAY();
}

static void attributes_payload()
{
    // The payload field can be used to provide additional data for markers
    // and ranges. Range events can only specify values at the beginning of a
    // range.
    //
    // The caller must specify valid values for both payloadType and payload.

    // INVALID - payloadType must be defined to a type

    nvtxEventAttributes_t eventAttrib = {0};
    eventAttrib.version = NVTX_VERSION;
    eventAttrib.size = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
    // fail to set eventAttrib.payloadType = NVTX_PAYLOAD_TYPE_UNSIGNED_INT64;
    eventAttrib.payload ullValue = 1;
    // specifying message to help identify this event in the tool.
    eventAttrib.messageType = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii = "attributes_payload:Invalid payloadType";
    nvtxMarkEx(&eventAttrib);

    DELAY();

    // VALID NVTX_PAYLOAD_TYPE_UNSIGNED_INT64
    {
        // NOTE: eventAttrib can be re-used
        nvtxEventAttributes_t eventAttrib = {0};
        eventAttrib.version = NVTX_VERSION;
        eventAttrib.size = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
```

```

eventAttrib.payloadType      = NVTX_PAYLOAD_TYPE_UNSIGNED_INT64;
eventAttrib.payload.llValue  = 0;
eventAttrib.messageType      = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib.message.ascii    = "attributes_payload:UNSIGNED_INT64 = 0";
nvtxMarkEx(&eventAttrib);

DELAY();

eventAttrib.payloadType      = NVTX_PAYLOAD_TYPE_UNSIGNED_INT64;
eventAttrib.payload.llValue  = ULLONG_MAX;
eventAttrib.messageType      = NVTX_MESSAGE_TYPE_ASCII;
eventAttrib.message.ascii    = "attributes_payload:UNSIGNED_INT64 = ULLONG_MAX";
nvtxMarkEx(&eventAttrib);

DELAY();
}

// VALID NVTX_PAYLOAD_TYPE_INT64
{
    nvtxEventAttributes_t eventAttrib = {0};
    eventAttrib.version              = NVTX_VERSION;
    eventAttrib.size                 = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    eventAttrib.payloadType          = NVTX_PAYLOAD_TYPE_INT64;
    eventAttrib.payload.llValue      = 0LL;
    eventAttrib.messageType          = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii        = "attributes_payload:INT64 = 0";
    nvtxMarkEx(&eventAttrib);

    DELAY();

    eventAttrib.payloadType          = NVTX_PAYLOAD_TYPE_INT64;
    eventAttrib.payload.llValue      = LLONG_MIN;
    eventAttrib.messageType          = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii        = "attributes_payload:INT64 = LLONG_MIN";
    nvtxMarkEx(&eventAttrib);

    DELAY();

    eventAttrib.payloadType          = NVTX_PAYLOAD_TYPE_INT64;
    eventAttrib.payload.llValue      = LLONG_MAX;
    eventAttrib.messageType          = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii        = "attributes_payload:INT64 = LLONG_MAX";
    nvtxMarkEx(&eventAttrib);

    DELAY();
}

// VALID NVTX_PAYLOAD_TYPE_UNSIGNED_DOUBLE

```

```

{
    nvtxEventAttributes_t eventAttrib = {0};
    eventAttrib.version                = NVTX_VERSION;
    eventAttrib.size                   = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    eventAttrib.payloadType            = NVTX_PAYLOAD_TYPE_DOUBLE;
    eventAttrib.payload.dValue         = 0.0;
    eventAttrib.messageType            = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii          = "attributes_payload:DOUBLE = 0.0";
    nvtxMarkEx(&eventAttrib);

    DELAY();

    eventAttrib.payloadType            = NVTX_PAYLOAD_TYPE_DOUBLE;
    eventAttrib.payload.dValue         = DBL_MIN;
    eventAttrib.messageType            = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii          = "attributes_payload:DOUBLE = DBL_MIN";
    nvtxMarkEx(&eventAttrib);

    DELAY();

    eventAttrib.payloadType            = NVTX_PAYLOAD_TYPE_DOUBLE;
    eventAttrib.payload.dValue         = DBL_MAX;
    eventAttrib.messageType            = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib.message.ascii          = "attributes_payload:DOUBLE = DBL_MAX";
    nvtxMarkEx(&eventAttrib);

    DELAY();
}

static void attributes_message()
{
    // The message field can be used to specify an optional string.
    // The caller must specify a messageType and a message.

    // VALID NVTX_MESSAGE_TYPE_ASCII

    nvtxEventAttributes_t eventAttrib1 = {0};
    eventAttrib1.version                = NVTX_VERSION;
    eventAttrib1.size                   = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
    eventAttrib1.messageType            = NVTX_MESSAGE_TYPE_ASCII;
    eventAttrib1.message.ascii          = "attributes_message:ascii";
    nvtxMarkEx(&eventAttrib1);

    DELAY();

    // VALID NVTX_MESSAGE_TYPE_UNICODE

    nvtxEventAttributes_t eventAttrib2 = {0};

```

```

    eventAttrib2.version          = NVTX_VERSION;
    eventAttrib2.size             = NVTX_EVENT_ATTRIB_STRUCT_SIZE;
    eventAttrib2.messageType      = NVTX_MESSAGE_TYPE_UNICODE;
    eventAttrib2.message.unicode  = L"attributes_message:unicode \u2603 snowman";
    nvtxMarkEx(&eventAttrib2);

    DELAY();
}

static void sample_nvtxDomainRegisterStringA()
{ // Create custom Domain
    nvtxDomainHandle_t domain =
    ⇨ nvtxDomainCreateA("sample_nvtxDomainRegisterStringA:nvtxDomainRangeStartEx");

    // zero the structure
    nvtxEventAttributes_t eventAttrib = {0};
    nvtxStringHandle_t strHandle =
        nvtxDomainRegisterStringA(domain,
    ⇨ "sample_nvtxDomainRegisterStringA:nvtxRegisterString");
    // set the version and the size information
    eventAttrib.version = NVTX_VERSION;
    eventAttrib.size    = NVTX_EVENT_ATTRIB_STRUCT_SIZE;

    // configure the attributes. 0 is the default for all attributes.
    eventAttrib.colorType      = NVTX_COLOR_ARGB;
    eventAttrib.color          = COLOR_BLUE;
    eventAttrib.messageType    = NVTX_MESSAGE_TYPE_REGISTERED;
    eventAttrib.message.registered = strHandle;

    nvtxRangeId_t id3 = nvtxDomainRangeStartEx(domain, &eventAttrib);
    DELAY_RANGE();
    nvtxDomainRangeEnd(domain, id3);
    DELAY();
}

static void name_thread()
{
    // The caller can name threads in the current process.
    nvtxNameOsThreadA(getpid(), "main");
};

int main(int argc, char* argv[])
{
    // Events

    simple_nvtxMarker();
    simple_nvtxRangeStartEnd();
    simple_nvtxRangePushPop();

    simple_nvtxDomainMarker();

```

```

simple_nvtxDomainRangeStartEnd();
simple_nvtxDomainRangePushPop();
sample_nvtxDomainRegisterStringA();

// Events with Attributes

attributes_versioned();
attributes_unversioned();

attributes_category();
attributes_color();
attributes_payload();
attributes_message();

// Resource Naming

name_thread();
// see attributes_category for example of naming a category

return 0;
}

```

5.4.3.4 使用 ixSYS 抓取 ixTX 上报的数据并分析

您可以通过如下任意一条命令，使用 ixSYS 命令行工具抓取 ixTX 上报的数据：

```

# my_app 是默认的输出文件名
$ ixsys -t nvtx -o output ./my_app
# 或者
$ ixsys -o output ./my_app # ixSYS 默认抓 nvtx, cuda 数据

```

使用 ixSYS 图形工具抓取到的 NVTX 数据如下显示：

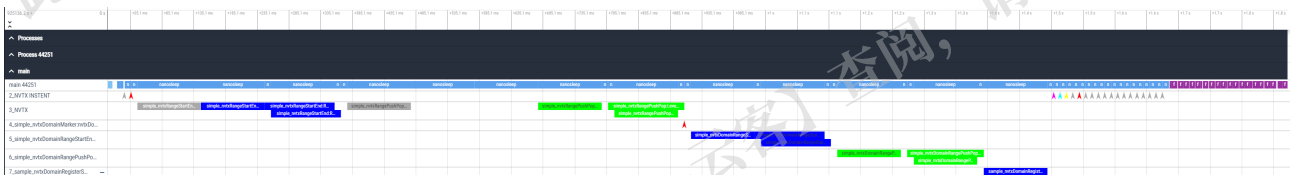


图 51: ixTX 数据抓取示例

从上图可以看到，线程名已经被命名为“main”。ixTX 的 API 默认使用 NVTX INSTENT (Marker 事件) 和 NVTX (Range 事件) Domain。如果通过 nvtxDomainCreateA 创建自定义的 Domain，并使用对应的 API，那么该事件会使用自定义的 Domain。

多机多卡 Profiling 示例

ixTX 可以提供多机多卡的 Profiling 信息。当通信后端选为 NCCL (ixCCL) 时, NCCL 的主要通信接口均嵌入了 ixTX API, 因此可以通过抓取 ixTX 上报的数据来获取 NCCL API 的时长信息 (通过抓取 Kernel 信息获取通信 Kernel 的详细信息), 并通过 ixSYS 图形化工具进行展示。

当前的 NCCL 代码中已经添加 ixTX 注释, 无需您修改代码, 只需要指定 ixSYS 抓取 ixTX 上报的数据即可。以输出文件名为 my_dpp_trian.py 为例, 执行以下任意一条命令抓取 NVTX 的数据:

```
$ ixsys -t nvtx -o output python3 "my_dpp_trian.py"
# 或者
$ ixsys -o output python3 "my_dpp_trian.py"
```

下面是 ResNet50 模型在 8 个 GPU 上使用 NCCL 为通信后端, 训练了 10 个 steps 的训练数据 (实例代码路径: /opt/apps/pytorch/third_party/kineto/tb_plugin/examples/resnet50_ddp_profiler.py)。其中, NCCL Domain 是自定义的 Domain, 主要记录 NCCL API 的信息, 根据这些信息可以分析卡间通信是否为性能瓶颈。由于 8 个设备的数据太多, 无法在一个图片中展示, 因此下面以一个设备为例进行展示。

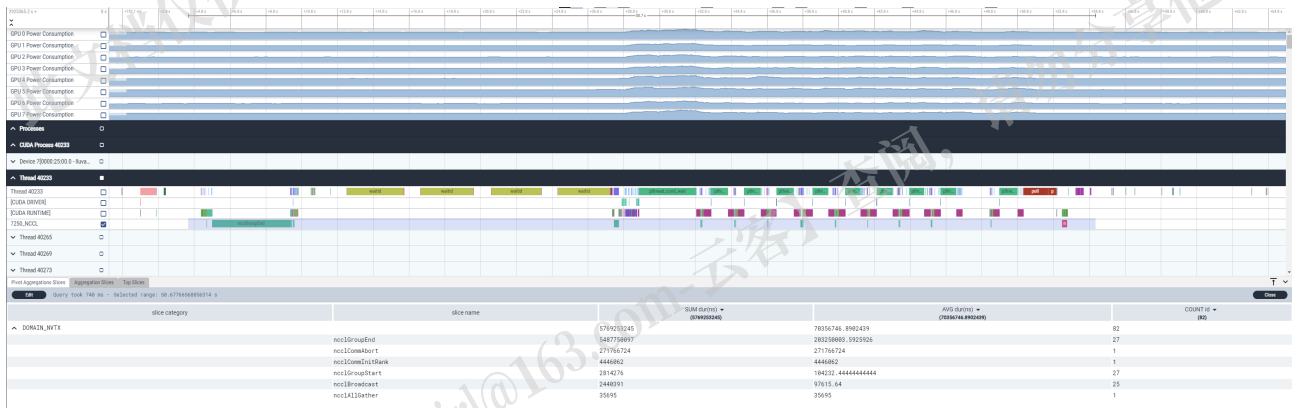


图 52: 多机多卡 Profiling 示例

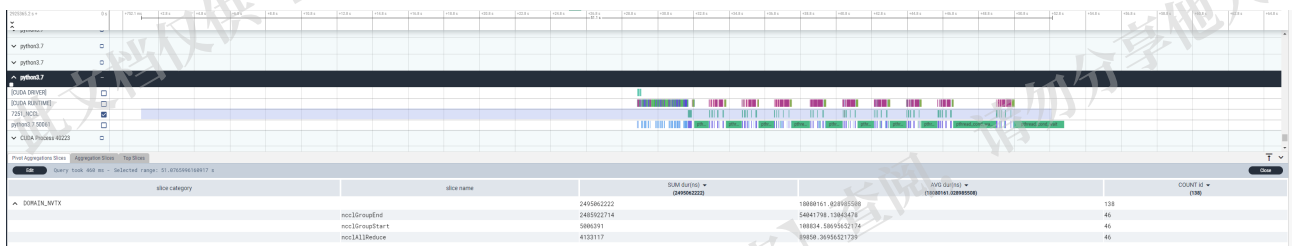


图 53: 多机多卡 Profiling 示例

从上图可以看到, 进程 40233 中有两个自定义 Domain 位于不同的线程中, 分别是 7250_NCCL 和 7251_NCCL。该自定义 Domain 主要用于区分过滤 NCCL 通信事件, 便于开发人员查找。从上面两张图片分析可知, 每个设备均有两个线程涉及 NCCL 通信, 一个用于 NCCL 的初始化以及广播操作, 一个用于训练过程中数据的 AllReduce 操作。在当前示例中, 40233 线程的通信事件花费了 5.7 秒, 50061 线程的通信事件花费了 2.5 秒, 该设备的 NCCL 共花费了 8.2 秒, 应用的总时间是 64.8 秒, 通信占了 1/8 的时间。但其中 NCCL API, 部分操作是在 CPU 侧进行, 并且两个线程可以并行进行, 可能存在部分时间是重叠的。因此可以从 Kernel 的角度来看 NCCL 的通信。

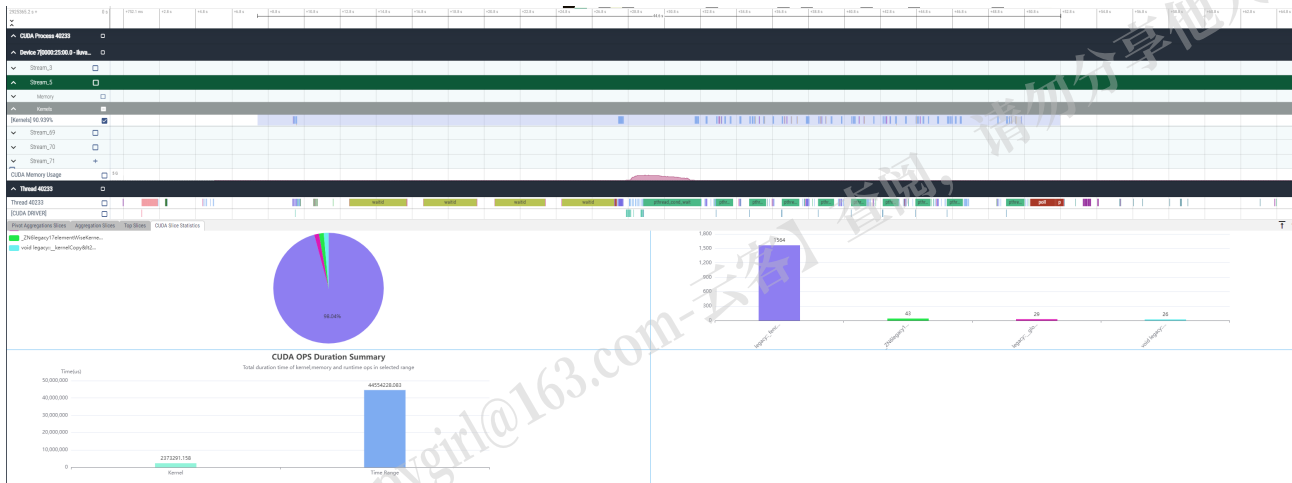


图 54: 多机多卡 Profiling 示例 - 从 Kernel 角度查看 NCCL 的通信

从 Kernel 角度来看，NCCL 的实现是通过 `__kernelCopy` 来实现的，通过上图可以看到 `__kernelCopy` 总共使用了 26 微妙，在整个应用运行期间占用的比例很少。因此，针对该 GPU 设备来说，通信占用设备的时间比例还是较少的。而上面统计的 8.2 秒，主要是在 CPU 侧消耗的时间和 Launch Kernel 的时间较长。使用 SQL 语句统计 `__kernelCopy` 的延迟，发现 `__kernelCopy` 的延迟花费的时间为 6.6 秒。

```
select sum(latency) from cuda_kernel where deviceId=7 and name="__kernelCopy<2, 2>(int*, int const*, unsigned long, legacy::syncDevicesHelper)"
Query result: 23 ms select sum(latency) from cuda_kernel where deviceId=7 and name="__kernelCopy<2, 2>(int*, int const*, unsigned long, legacy::syncDevicesHelper)"
sum(latency)
6625982191
```

图 55: 多机多卡 Profiling 示例 - 从 Kernel 角度查看 NCCL 通信的延迟时间

5.4.4 GPU metrics

GPU metrics 功能是用用于分析应用在进行 GPU 相关计算时的性能限制。GPU metrics 使用周期性采样来收集与不同 GPU 硬件单元相关的性能指标和详细的定时统计数据。由于硬件不支持周期性采样，因此 GPU metrics 是通过软件支持的，建议采样频率使用默认值。

Note

GPU metrics 可以提供精确的设备级别的信息，但无法获取进程、上下文相关信息。

5.4.4.1 使用说明

GPU metrics 包含以下 3 个选项：

- **--gpu-metrics-device [all, none, <index>]**: 用于指定抓取 metrics 的设备，默认是 none，表示不抓取 metrics 数据
- **--gpu-metrics-set [<alias>]**: 用于指定 metrics set，默认是第一个适合该 GPU 设备的 metrics set
- **--gpu-metrics-frequency [10Hz, 200000Hz]**: 用于指定采样频率，默认为 500Hz

以下是具体的命令使用说明：

- 查询可提供的设备：

```
$ ixsys --gpu-metrics-device help
Possible --gpu-metrics-device values are:
    0: Iluvatar MR-V50
    all: Select all supported GPU
    none: Disable GPU Metrics [Default]
```

- 查询所有的 metrics set：

```
$ ixsys --gpu-metrics-set help
Possible --gpu-metrics-set values are:
    [0] [mr_generate_metrics]           The generate metrics for MR
    [1] [mr_throughput_metrics]        Throughput metrics for MR
```

- 使用默认选项来采样 GPU 0 上的 metrics：

```
# my_app 是默认的输出文件名
$ ixsys --gpu-metrics-device 0 -o output ./my_app
```

- 指定选项参数来采样 GPU 0 上的 metrics：

```
# my_app 是默认的输出文件名
$ ixsys --gpu-metrics-device 0 --gpu-metrics-set mr_generate_metrics
  ↪ --gpu-metrics-frequency 400 -o output ./my_app
```

5.4.4.2 示例

下面是以一个推理模型 (示例所在路径：framework-tests/pt/alexnet/run_alexnet_inference.sh) 为例，抓取的 GPU metrics 数据在 ixSYS 图形化工具显示的信息：

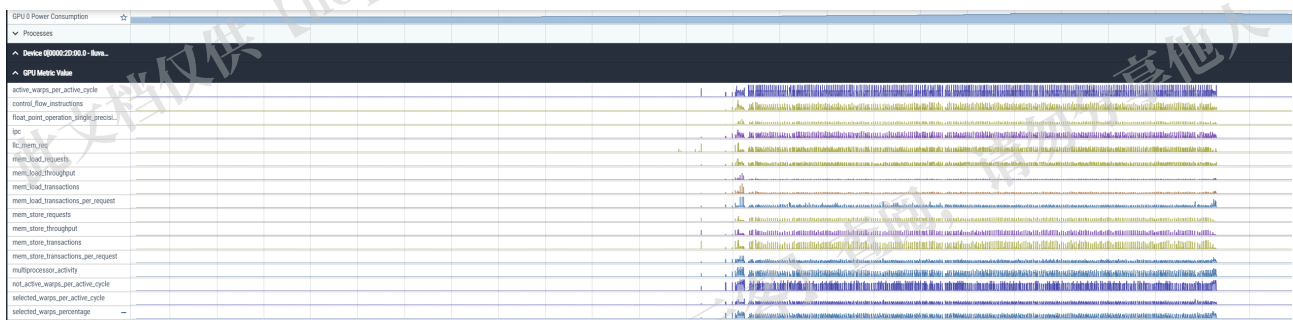


图 56: ixSYS 显示 GPU metrics 示例

由于 metrics sampling 的数据采集是在应用的初始化阶段开始采集，析构时停止采集，因此开始采集数据的时间要比应用真正运行的时间略早。您可以通过放大和圈选，只看自己关注的的数据段。

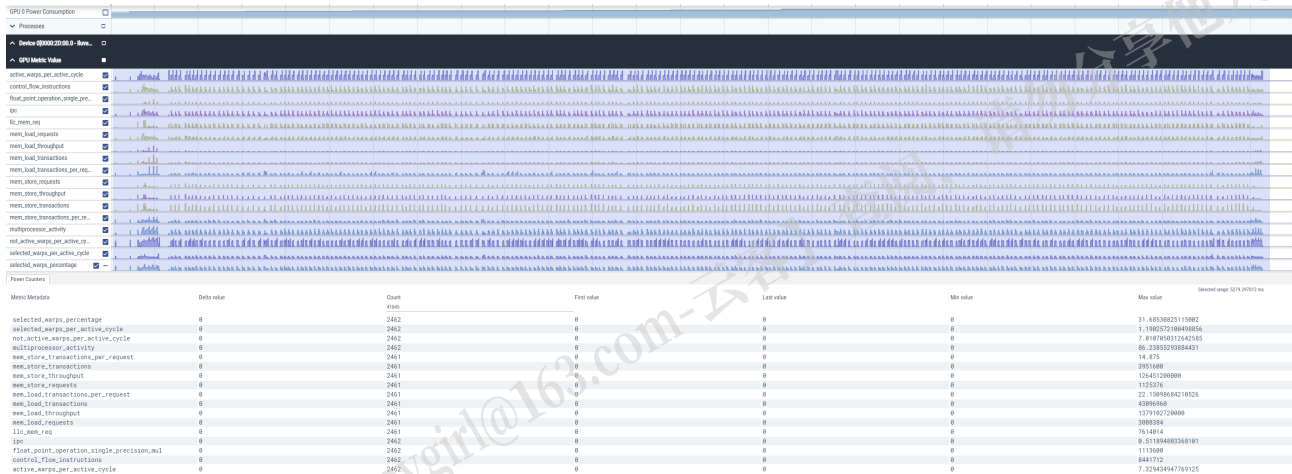


图 57: ixSYS 显示 GPU metrics 示例

您可以根据上面采集的数据分析整个应用运行期间各个硬件单元的资源是否被充分利用，从而来确认优化方向。

5.4.5 Event Sync

Event Sync 功能用于应用开发阶段，跟踪应用中使用 cudaEventRecord 和 cudaStreamWaitEvent 同步的同步信息，用户可以通过抓取到的同步信息来确认应用的同步是否与期望的一致。

5.4.5.1 使用说明

Event Sync 的 Profiling 默认关闭 (false)，需要设置为 true 来进行 Profiling，即：

- **--event-sync true**，打开 Event Sync Profiling
- (默认) **--event-sync false**，关闭 Event Sync Profiling

具体的命令使用方法如下：

```
$ ixsys -o output --event-sync true ./my_app
```

5.4.5.2 示例

下面以 ixTX 的应用为例，抓取 Event Sync 数据。

从下图中可以看到，stream_1 和 stream_4 之间的 GPU 操作存在 Event Sync。选取其中两个并放大，可以看到 kernel1 和 kernel2 之间存在同步，kernel2 依赖 kernel1；kernel3 和 kernel4 之间也存在同步。

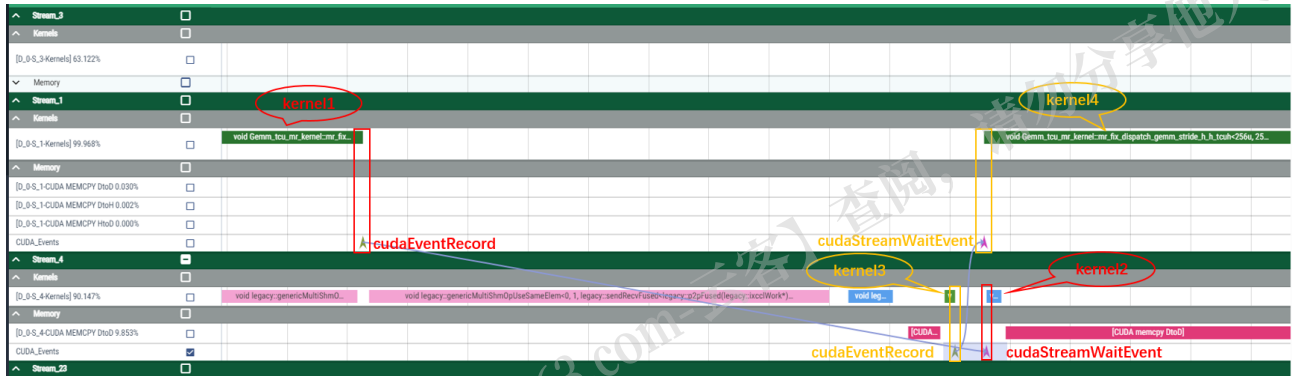


图 58: ixTX 应用的 Event Sync 数据

Note

统计 Event Sync 数据是通过 CUDA API 和 CUDA Device 数据统计得到。因此使用该功能，需要同时使能 cuda_api 和 cuda_device 数据。

6 核函数性能分析工具 ixKN 使用指南

6.1 ixKN 工具概述

Iluvatar Kernel Analyzer 工具简称 ixKN，是对运行在天数智芯加速卡上的 CUDA 应用程序进行性能分析的工具。该工具可根据按照工具预先定义的性能分析指标（称作 section），对 CUDA 应用程序的核函数抓取指定的性能数据，方便您基于该性能数据对 CUDA 核函数进行优化。

ixKN 工具由命令行工具合图形界面工具组成：

组件	描述
ixKN CLI 命令行工具	在 CUDA 核函数 (kernel) 执行期间抓取性能分析指标的数据 注：当前版本暂不支持在 svGPU 上使用 ixKN CLI
ixKN-UI 图形界面工具	对运行在天数智芯加速卡上的 CUDA 应用程序进行性能分析，分析数据可用该工具做可视化展示。 目前支持 Linux 和 Windows 操作系统。

6.2 1. 使用 ixKN CLI 工具抓取数据

6.2.1 使用方法

前置条件：ixKN CLI 支持在已经安装 Python3.6 及以上 3.9 及以下的环境上运行。

您在命令行输入 **ixkn-cli** 和所需命令选项，即获得 CUDA 核函数（kernel）相关性能数据。

命令参数	命令描述及示例
-h, --help	打印工具的帮助信息
-v, --version	显示工具的版本信息
[cuda-app] [app-args]	显示用户应用程序中 kernel 运行状态的默认性能分析指标数据
--list-sets	列出当前定义的所有 set
--set TEXT	指定一个 set 进行抓取 kernel 性能分析指标数据
--list-sections	列出当前定义的所有 section
--section TEXT	指定一个或多个 section 进行抓取 kernel 性能分析指标数据
--list-rules	列出当前定义的所有用于性能分析的 rule
--quiet	不输出除了用户应用程序之外的任何显示内容
--p rofile-child-processes	指定是否对程序的子进行分析
-k, --kernel-name TEXT	指定需要分析的 kernels

命令参数	命令描述及示例
--kernel-id TEXT	设定过滤器来指定需要进行分析的 kernel
-o, --export-profile TEXT [cuda-app] [app-args]	导出运行信息结果到特定的文件，用于稍后导入或者使用其他工具打开
-i, --import-profile TEXT	从文件导入之前的运行信息结果并输出
-f, --force-overwrite	强制修改之前的输出文件
-c, --launch-count TEXT [cuda-app] [app-args]	限定需要分析 kernel 信息的数量
-s, --launch-skip TEXT [cuda-app] [app-args]	限定需要跳过分析 kernel 的数量 注：此功能和 --launch-count 功能同时使用时，优先需要跳过满足限定数量的 kernel 之后，再对 kernel 进行计数
--profile-from-start TEXT {off, on}	设置分析是否从头开始，如果设置为 off，profile 只会从 cuda/cuProfilerStart() 调用处开始
--devices TEXT	限定需要 device 进行分析
--disable-profiler-start-stop	此设置会使得需要分析程序中的 cudaProfilerStart/Stop 和 cuProfilerStart/Stop 调用失效
--filter-mode TEXT:{global, per-gpu}	设置 kernel 过滤器的模式，global 是每个需要分析的程序共同使用同一个过滤器，per-gpu 是每个进程中根据设备数量的不同为每个设备都注册一个过滤器
--replay-mode TEXT: {kernel,application}	设置收集 section 的 replay 方式。当一次 kernel 执行需要收集的指标过多时，需要重复执行 kernel 来完成 section 的收集。提供两种方式来对 kernel 进行重放，默认方式是 kernel
--target-processes-filter TEXT	在分析多进程程序时，可以限定需要进行分析进程的名字进行分析
--kill {on, off}	此特性需要和 --launch-counter 共同使用。当 counter 计数完成后，profile 的进程提前退出，而不会执行完
--page TEXT	指定最终打印数据的界面，目前提供三种选择： details：显示 section 的分析结果 raw：显示所有 section 的原始数据 source：显示 kernel 的代码
--print-source TEXT	在指定 page 为 source 的情况下，选择希望展示的代码形式，目前提供四种模式供选择：IR，assembly，cuda 和 cuda,assembly
--resolve-source-file	指定解析源码的文件，多个文件可以用分号分开
--import-source TEXT	指定采集到的代码是否存储到导出的文件中，默认情况不导入 cuda 和 IR 源码

6.2.2 功能列表

该工具提供以下功能：

6.2.2.0.1 -h, --help

打印工具的帮助信息。

```
$ ixkn-cli -h
```

6.2.2.0.2 -v, --version

显示工具的版本信息。例如：

```
$ ixkn-cli -v
```

6.2.2.0.3 [cuda-app] [app-args]

显示反映用户应用程序当中 Kernel 运行状态的默认性能分析指标数据。例如：

```
$ ixkn-cli ./vectorAdd
```

6.2.2.0.4 --list-sets

列出当前定义的所有 set。set 被定义为一组 section 的集合。用户指定一个 set，即可以对属于该 set 的所有 section 进行抓取 kernel 性能分析指标数据。例如：

```
$ ixkn-cli --list-sets
```

6.2.2.0.5 --set TEXT

指定一个 set 进行抓取 kernel 性能分析指标数据。

```
$ ixkn-cli --set default ./vectorAdd
```

6.2.2.0.6 --list-sections

列出当前定义的所有 section。

```
$ ixkn-cli --list-sections
```

6.2.2.0.7 --section TEXT

指定一个或多个 section 进行抓取 kernel 性能分析指标数据。使用--section all”可以对全部 section 进行抓取 kernel 性能分析指标数据。例如：

```
$ ixkn-cli --section Instruction,LaunchStats ./vectorAdd
```

6.2.2.0.8 --list-rules

列出当前定义的所有用于性能分析的 rule。为每个 section 需要定义一个 rule 来计算 kernel 性能分析指标数据。

```
$ ixkn-cli --list-rules
```

6.2.2.0.9 --quiet

不输出除了用户应用程序之外的任何显示内容。

```
$ ixkn-cli --quiet ./vectorAdd
```

6.2.2.0.10 --profile-child-processes

指定是否对程序的子进行分析，不加该选项则不对子进程进行分析。

```
$ ixkn-cli --profile-child-processes ./test_ixkn.sh
```

6.2.2.0.11 -k,--kernel-name TEXT

指定需要分析的 kernels，如果有多个 kernel 需要分析可以使用逗号分开和使用正则表达式指定。例如：

```
$ ixkn-cli --profile-child-processes --kernels WinogradFusedKernelF2x2_3x3 ./
↪ run_alexnet_inference.sh
```

6.2.2.0.12 --kernel-id TEXT

设定过滤器来指定需要进行分析的 kernel，过滤格式为 context-id:stream-id:[name-operator:]kernel-name:correlation_id。例如，::foobar:2、':7:foo:'。

```
$ ixkn-cli --kernel-id :::44 ./device_scan
```

6.2.2.0.13 -o, --export-profile TEXT [cuda-app] [app-args]

导出运行信息结果到特定的文件，用于稍后导入或者使用其他工具打开。

```
$ ixkn-cli -o vecAdd.log ./vectorAdd
```

6.2.2.0.14 -i, --import-profile TEXT

从文件导入之前的运行信息结果并输出。例如：

```
$ ixkn-cli -i vecAdd.log
```

6.2.2.0.15 -f, --force-overwrite

强制修改之前的输出文件。例如：

```
$ ixkn-cli -f -o vecAdd.log ./vectorAdd
```

6.2.2.0.16 -c, --launch-count

限定需要分析 kernel 信息的数量。

```
$ ixkn-cli -c 1 ./vectorAdd
```

6.2.2.0.17 -s, --launch-skip

限定需要跳过分析 kernel 的数量。注意：此功能可以和 -c 参数共同使用，但是会优先跳过设定数量的 kernel，才会对 kernel 进行计数。

```
$ ixkn-cli -s 1 ./vectorAdd
```

6.2.2.0.18 --profile-from-start TEXT {off, on}

设置 profile 是否从头开始。如果设置为 off，profile 只会记录 cuda/cuProfilerStart() 到 cuda/cuProfilerStop() 之间的 kernel 信息。该参数的默认值为 on。

```
$ ixkn-cli --profile-from-start off ./vectorAdd
```

6.2.2.0.19 --devices TEXT

设置需要分析的设备 ID。profile 只会记录所设定设备上的 kernel 信息，其默认值为所有的设备。

```
$ ixkn-cli --devices 0,1 ./vectorAdd
```

6.2.2.0.20 --disable-profiler-start-stop

此设置会使得需要分析程序中的 cudaProfilerStart/Stop 和 cuProfilerStart/Stop 调用失效。

```
$ ixkn-cli --disable-profiler-start-stop ./vectorAdd
```

6.2.2.0.21 --filter-mode TEXT:{global, per-gpu}

设定 kernel 过滤器的模式。默认是 global，此模式会在每个需要分析的程序中使用此进程独有的一个过滤器。per-gpu 模式，会在每个需要分析的程序中针对多设备的情况下，给每个设备设定一个 kernel 过滤器。

```
$ ixkn-cli --filter-mode per-gpu -c 1 -s 1 ./vectorAdd
```

6.2.2.0.22 --replay-mode TEXT: {application,kernel}

指定重放收集指标的形式。默认是重放 kernel 来收集指标或者 section，也可以使用 application 重放来完成指标收集。

```
$ ixkn-cli --replay-mode application ./vectorAdd
```

6.2.2.0.23 --target-processes-filter TEXT

限定需要分析的进程名字。

Note

该功能默认支持正则表达匹配名字。

```
$ ixkn-cli --target-processes-filter Add,Mul ./vectorAdd
```

6.2.2.0.24 --kill {on, off}

此功能配合 --launch-counter 共同使用。当 launch-counter 记完数后，kill 设置为 on，会使得进程被 kill，而提前退出。

```
$ ixkn-cli --kill on -c 1 ./matrixMul
```

6.3 2. 使用 ixKN-UI 工具分析性能

6.3.1 在 Windows 环境上安装 ixKN-UI 工具

ixKN-UI 工具的 Windows 版本安装包名称为 Iluvatar Explorer Kernel-{ixkn-ui-version}-win64.exe，双击该安装包后即可进行安装。

您可以通过以下方式获取 ixKN-UI 工具的安装包：

- 登录 [天数智芯官网](#)，进入 **客户支持 > 资源中心** 页面进行下载
- 联系您的应用工程师获取

第 1 步选择安装路径

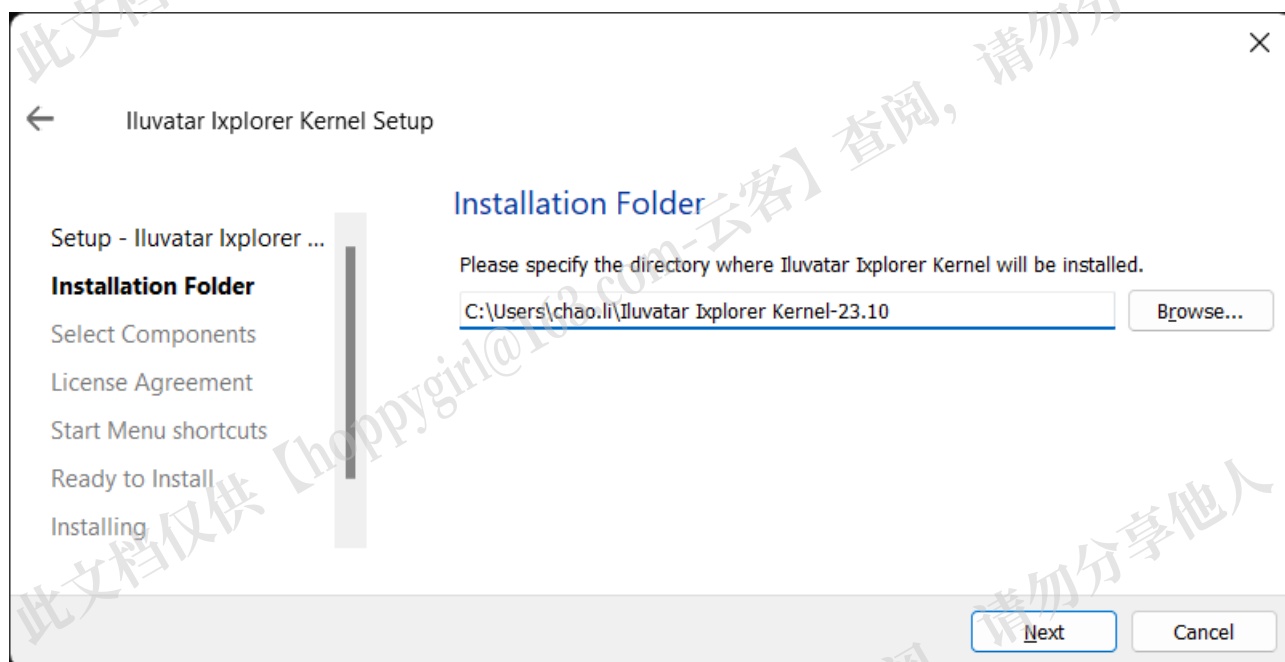


图 59: 选择安装路径

第 2 步接受安装许可

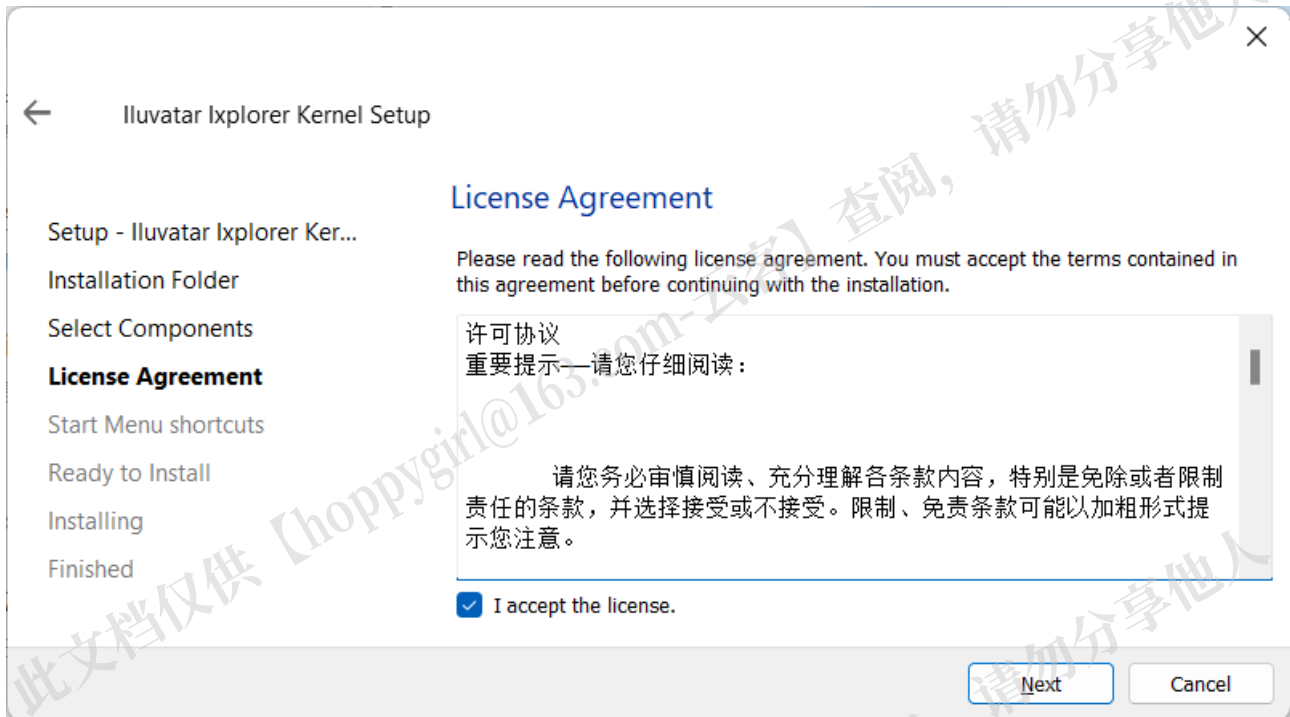


图 60: 接受安装许可

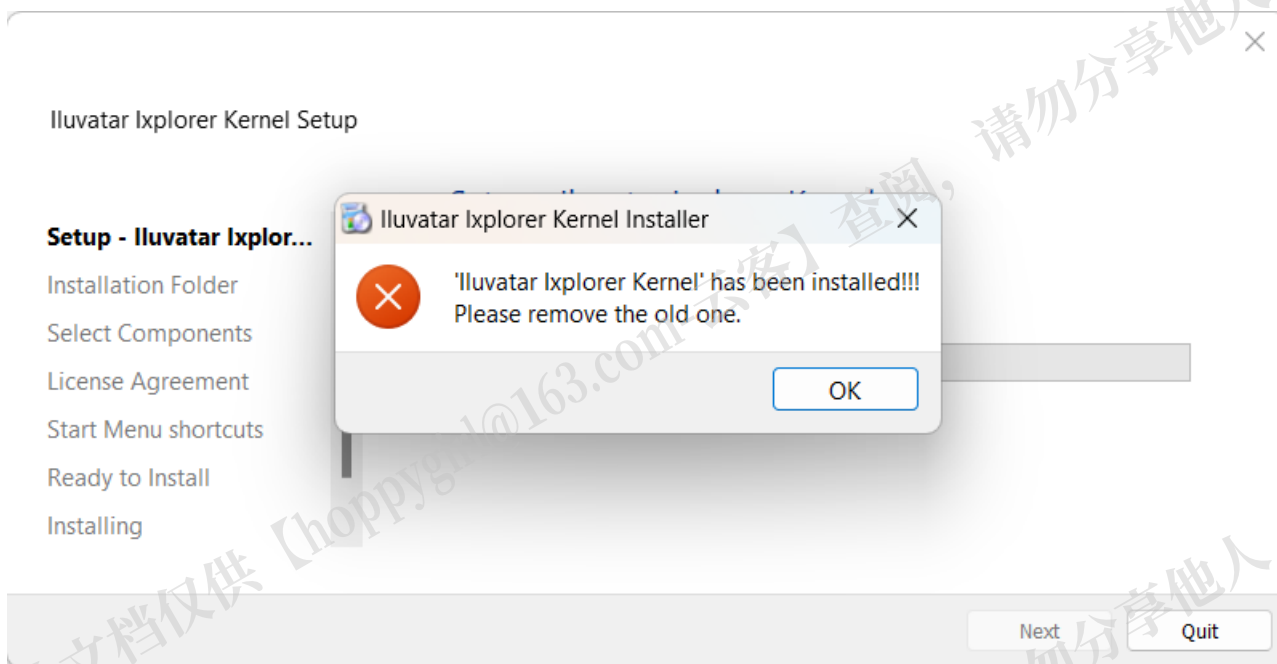
第 3 步可以在系统应用程序中找到启动图标



图 61: 找到启动图标

第 4 步重新安装 ixKN-UI 工具

再次安装工具时，您需要先卸载原有版本，否则会出现以下错误提示：



6.3.2 在 Linux 环境上安装 ixKN-UI 工具

ixKN-UI 工具的 Linux 版本安装包名称为 Iluvatar-Ixplorer-Kernel-{ixkn-ui-version}-Linux.run，目前仅支持在 Ubuntu 18 和 Ubuntu 20 的系统上使用，暂未适配其它 Linux 操作系统，双击该安装包后即可进行安装。

您可以通过以下方式获取 ixKN-UI 工具的安装包：

- 登录 [天数智芯官网](#)，进入 **客户支持 > 资源中心** 页面进行下载
- 联系您的应用工程师获取

第 1 步选择安装路径

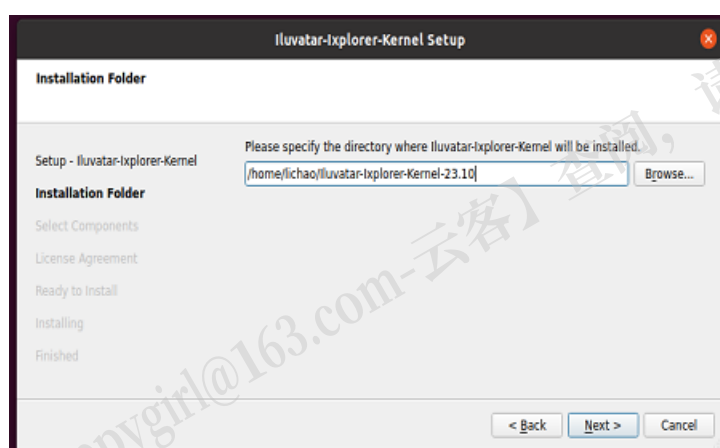


图 62: 选择安装路径

第 2 步接受安装许可



图 63: 接受安装许可

第 3 步在安装路径的文件夹种可以找到启动图标

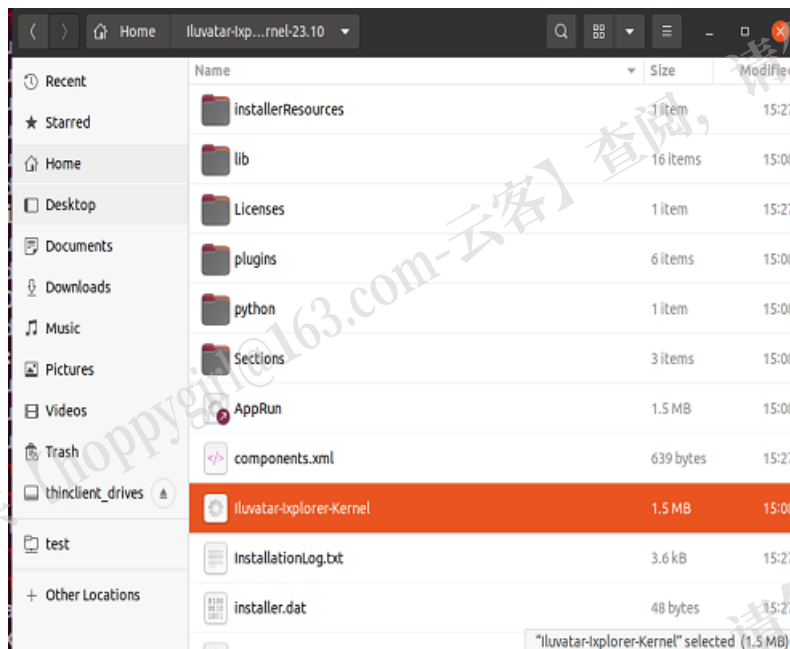


图 64: 找到启动图标

第 4 步重新安装 ixKN-UI 工具

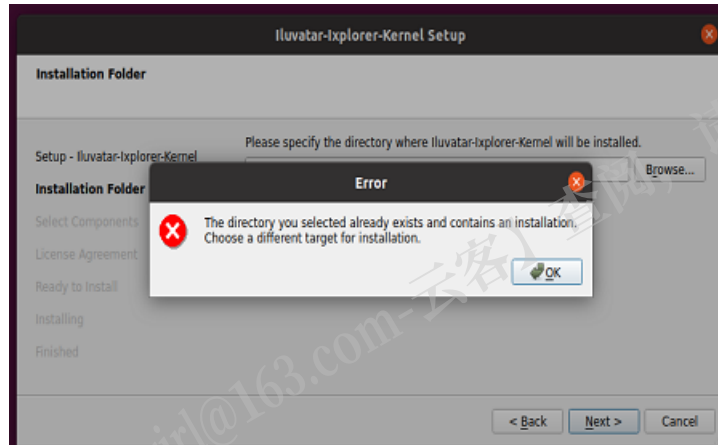


图 65: 重新安装 ixKN-UI 工具

6.3.3 使用 ixKN-UI 工具

ixKN-UI 工具的 Windows 版本与 Linux 版本的使用方法完全一致。

目前，工具的使用分为两个步骤：

1. 首先使用工具 ixKN CLI 在目标机器上抓取需要分析的应用程序的分析数据，使用子命令 **-o** 将数据库文件导出，相关内容请参考**功能列表**。
2. 其次，使用 ixKN-UI 工具打开导出的文件即可查看具体需要了解性能的 kernel 相关的数据。

ixKN-UI 的数据内容界面分为：

- Session
- Summary
- Detail

如下图所示：

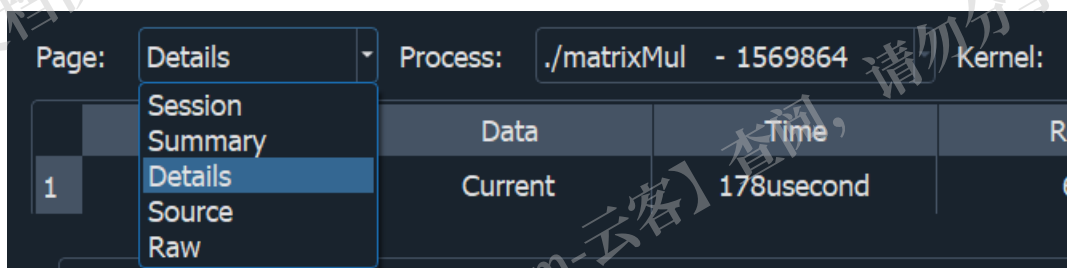


图 66: ixKN-UI 数据内容界面

以下是 ixKN-UI 的数据内容界面的三个页面的具体说明。

6.3.3.0.1 查看 Session 页面

Session 页面包含了三个部分，分别是 Session Info、Processes 与 Device Attributes。

Session Info 部分主要展示抓取分析文件的目标机器的概况，包括 CUDA version、文件创建时间、目标机器系统信息等。

Processes 部分展示分析的程序包含了多少个子进程，显示 Process ID 与 ProcessName 信息。

Device Attributes 显示了目标机器上所有天数智芯加速卡的属性。

具体内容如下图所示：

Session Info	
CUDA Version	10.02
Created	2023-10-23 15:52:43.427
Target Architectre	x86_64
Target Name	sw-10-113-1-83
Target OS	Linux-#92~20.04.1-Ubuntu SMP Mon Aug 21 14:00:49 UTC 2023
Target Processor	Intel(R) Xeon(R) Silver 4216 CPU @ 2.10GHz
Processes	
Process ID	Process Name
1569864	./matrixMul
Device Attributes	
Attribute	Iluvatar MR-V100(0)
ECCEEnabled	1
asyncEngineCount	1
canMapHostMemory	1
canUseHostPointerForRegisteredMem	0
clockRate	1500000
computeMode	0
computePreemptionSupported	0
concurrentKernels	1
concurrentManagedAccess	0
cooperativeLaunch	0
cooperativeMultiDeviceLaunch	0
devName	Iluvatar MR-V100
deviceOverlap	1
directManagedMemAccessFromHost	0

图 67: Session 页面内容

6.3.3.0.2 查看 Summary 页面

Summary 页面展示数据库中所有 kernel 的基本信息，包含 kernel 的 correlation Id、function name、demangled name、隶属的 process 与运行的 device name。如下图所示：

API Call ID	Fuction Name	Demangled Name	Process	Device Name
11	matrixMulCUDA<32>	void matrixMulCUDA<32>(float*, float*, float*, int, int)	[1569864] ./matrixMul	Iluvatar MR-V100
20	matrixMulCUDA<32>	void matrixMulCUDA<32>(float*, float*, float*, int, int)	[1569864] ./matrixMul	Iluvatar MR-V100

图 68: Summary 页面内容

6.3.3.0.3 查看 Detail 页面

Detail 页面分为两个部分，第一部分用于选择需要展示的 kernel 和 baseline 的相关操作以及该 kernel 的一些基本信息，第二部分为展示该 kernel 的分析指标。

第一部分用于选择需要分析的 process。如果应用中包含多个 process，可以通过下拉选项框来选择所需要分析的 process；在选定了需要的 process 后，第二部分的 kernel 选项框会列出该 process 下所有的 kernel，然后选择需要进行分析的 kernel。如下图所示：

Page:

Details

 Process:

./matrixMul - 1569864

 Kernel:

- 11 - matrixMulCUDA<32>

 Baseline:

Add Baseline

ScreenShot

	Data	Time	Regs	GPU	CC	Process
1	Current	178usecond	63	Iluvatar MR-V100	7.1	[1569864]

图 69: Detail 页面内容

默认界面会显示第一个 process 的第一个 kernel 的数据，在选择需要分析的 kernel 后，具体分析的数据会重新刷新。对于天数智芯加速卡来说，目前支持的 section 有 Instruction、LaunchStats、ComputeWorkload-Analysis、SpeedOfLight、SchedulerStats、Memory、Occupancy 与 WarpStateStats。展示的 section 数量会根据 ixKN CLI 命令行工具运行的参数来确定，也就是抓取数据时已经确定了可以显示的 section 的数量。section 显示的顺序是根据 section 文件中的 Order 字段来确定的。

以下是支持的 section 的具体说明。

SpeedOfLight 页面

该页面展示了 GPU 计算和内存资源使用情况的高级概述。表格展示了和 GPU 计算、内存相关的参数；柱状图展示了计算和内存的使用率。

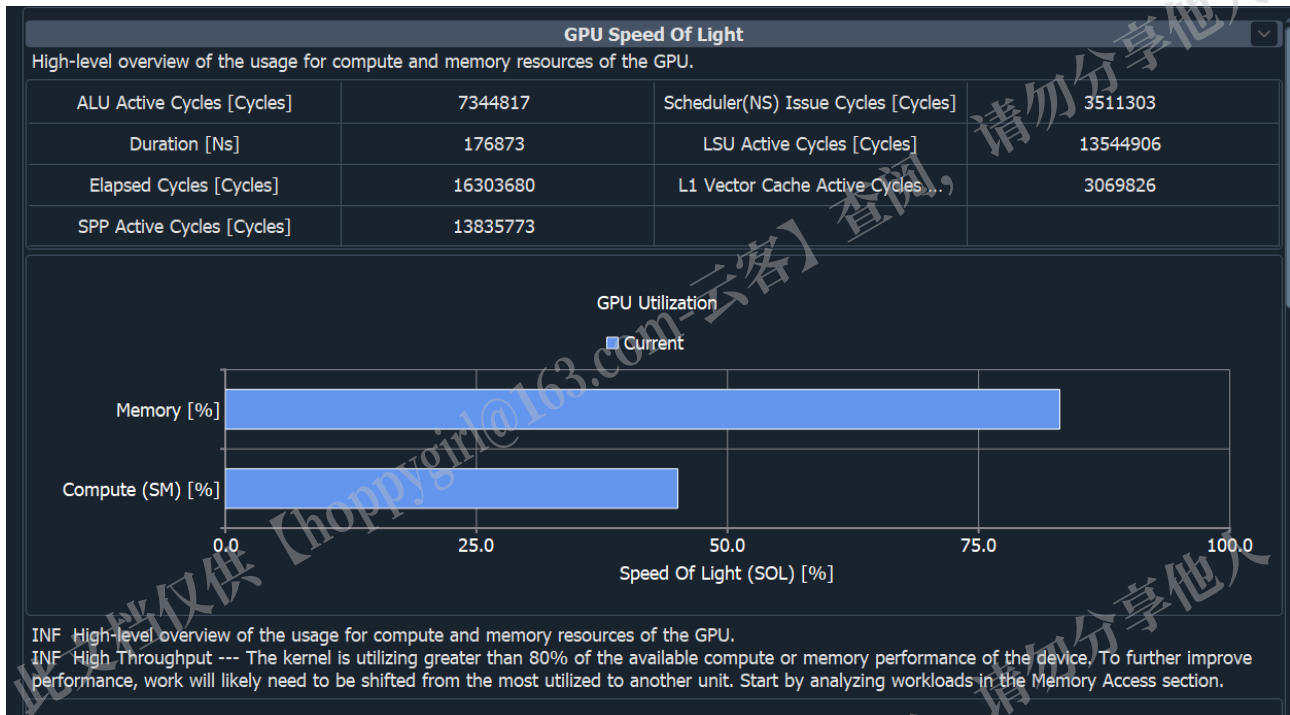


图 70: SpeedOfLight 页面

ComputeWorkloadAnalysis 页面

该页面以表格的方式展示了所有 SPP 的平均计算资源利用率分析并给出建议。如下图所示：

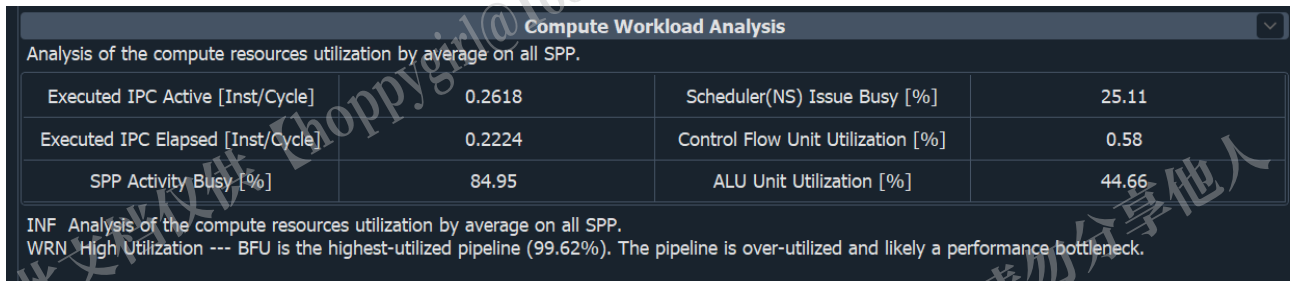


图 71: ComputeWorkloadAnalysis 页面

Memory 页面

该页面展示了内存访问相关操作的一些指标。表格展示了内存相关的吞吐量；流程图以百分比的形式展示了内存的命中率和使用率。如下图所示：

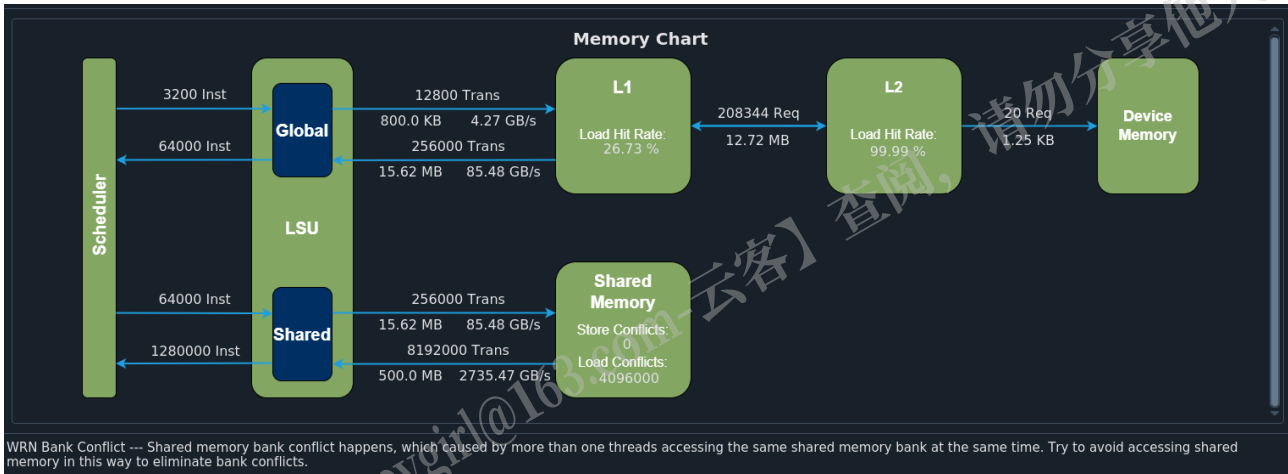


图 72: Memory 页面

SchedulerStats 页面

该页面展示了 NS 上的一些统计信息。数据以表格和柱状图的方式呈现，并提供一些建议。如下图所示：

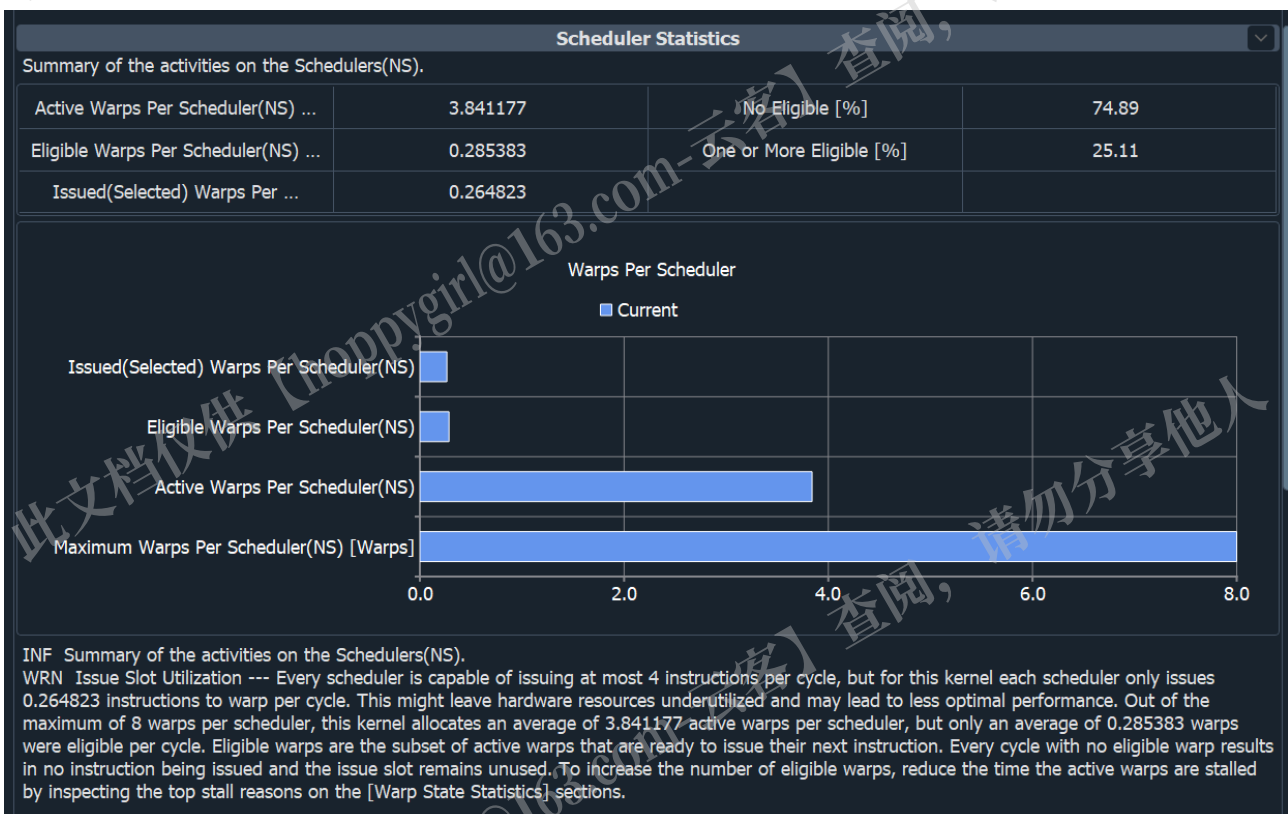


图 73: SchedulerStats 页面

WarpStateStats 页面

该页面展示了 NS 调度的 Warp 状态统计信息。表格展示了一些总体的信息；柱状图分别显示各个部分 stall reason 的占比；最后给出部分建议。如下图所示：

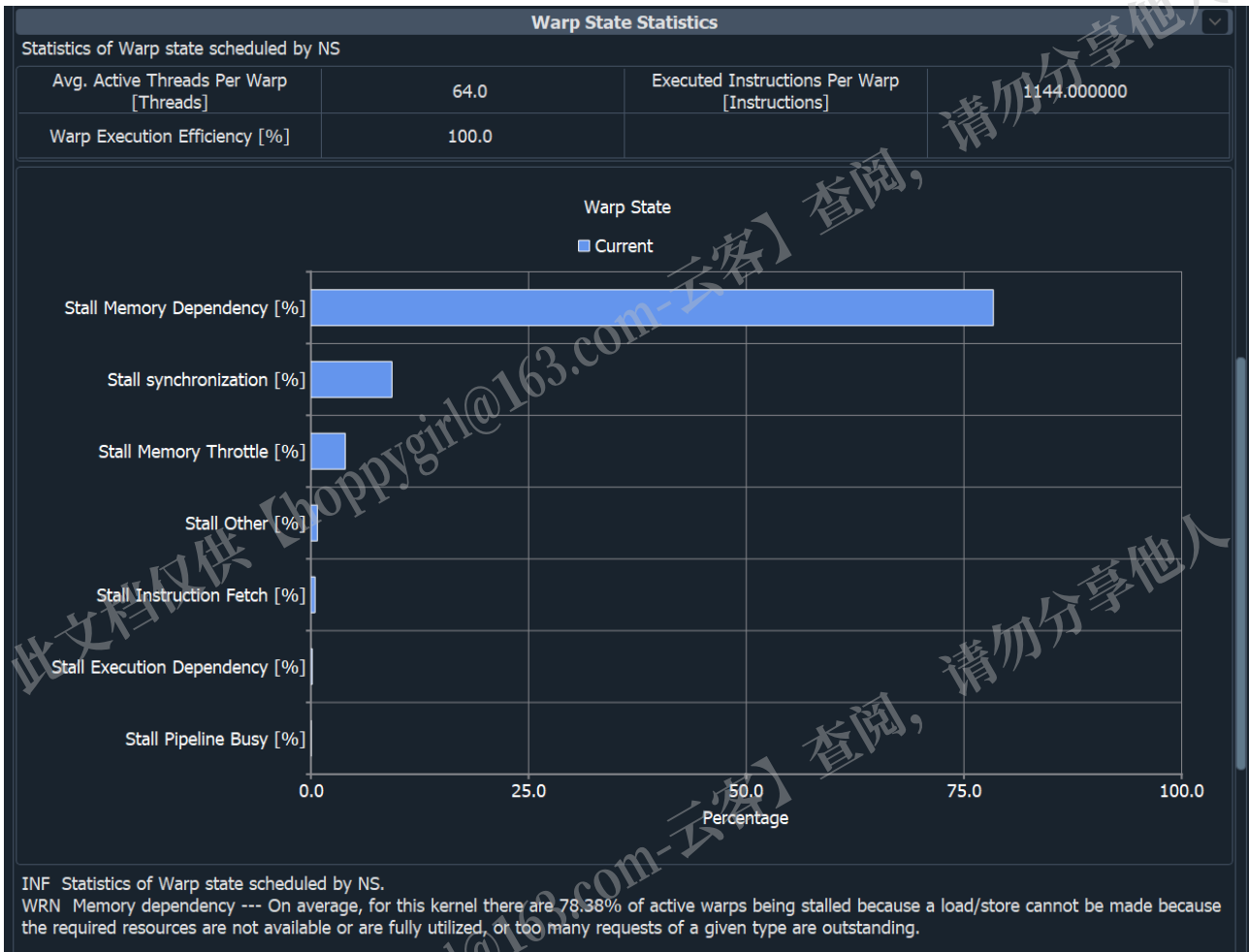


图 74: WarpStateStats 页面

Instruction 页面

该页面展示了该 kernel 发出和执行的指令统计信息。表格显示了指令的一些总的信息；柱状图以降序方式显示了各个类型的指令执行的统计次数。如下图所示：

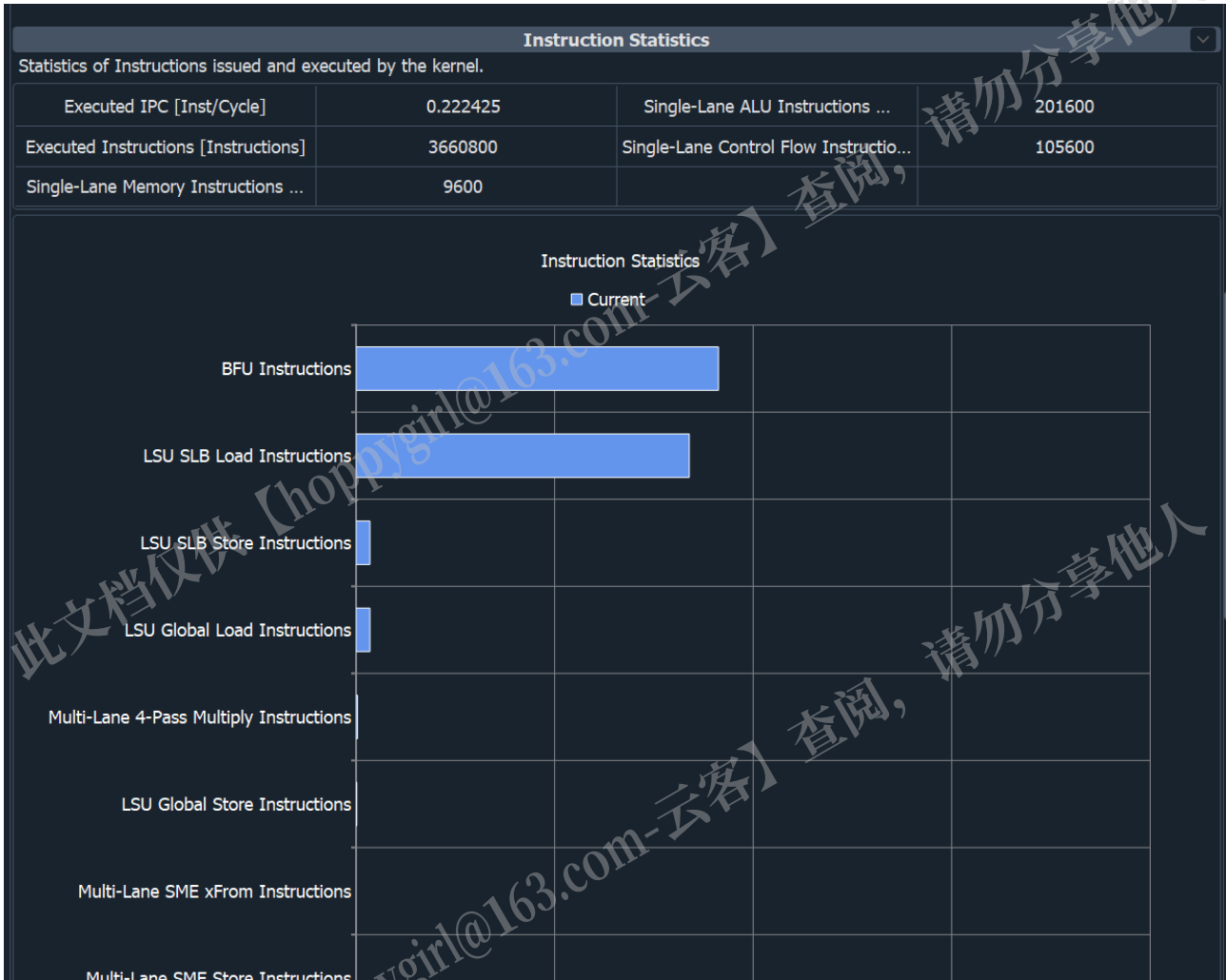


图 75: Instruction 页面

LaunchStats 页面

该页面用于显示启动内核的配置摘要。启动配置定义内核网络的大小、网格划分为块以及执行内核所需的 GPU 资源。如下图所示：

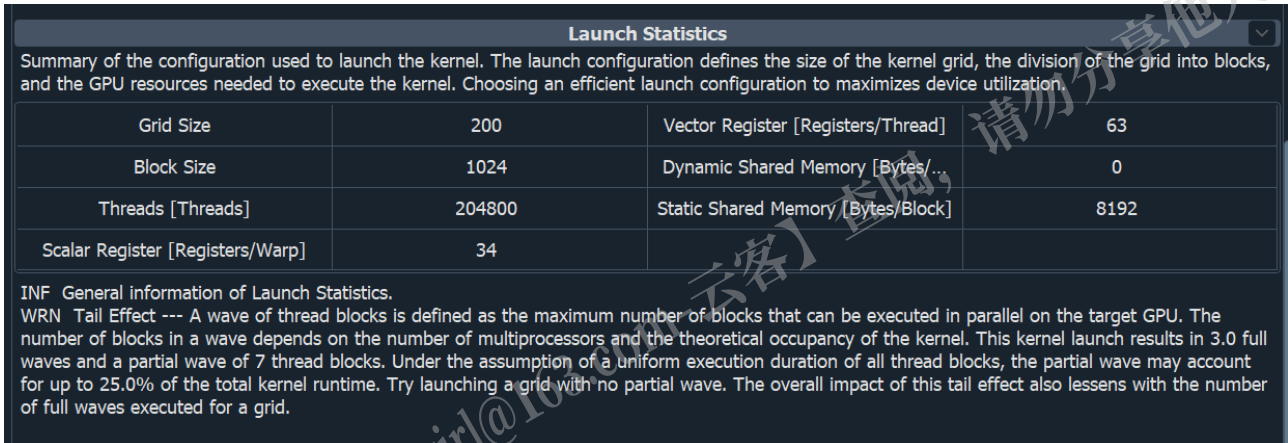


图 76: LaunchStats 页面

Occupancy 页面

该页面展示了占用率的信息，所谓占用率定义为 NR 上的活动 warp 与 NR 支持的最大活动 warp 数（8）的比率，同时给出建议。在不同参数的影响下，可以看出 occupancy 的变化率。如下图所示：

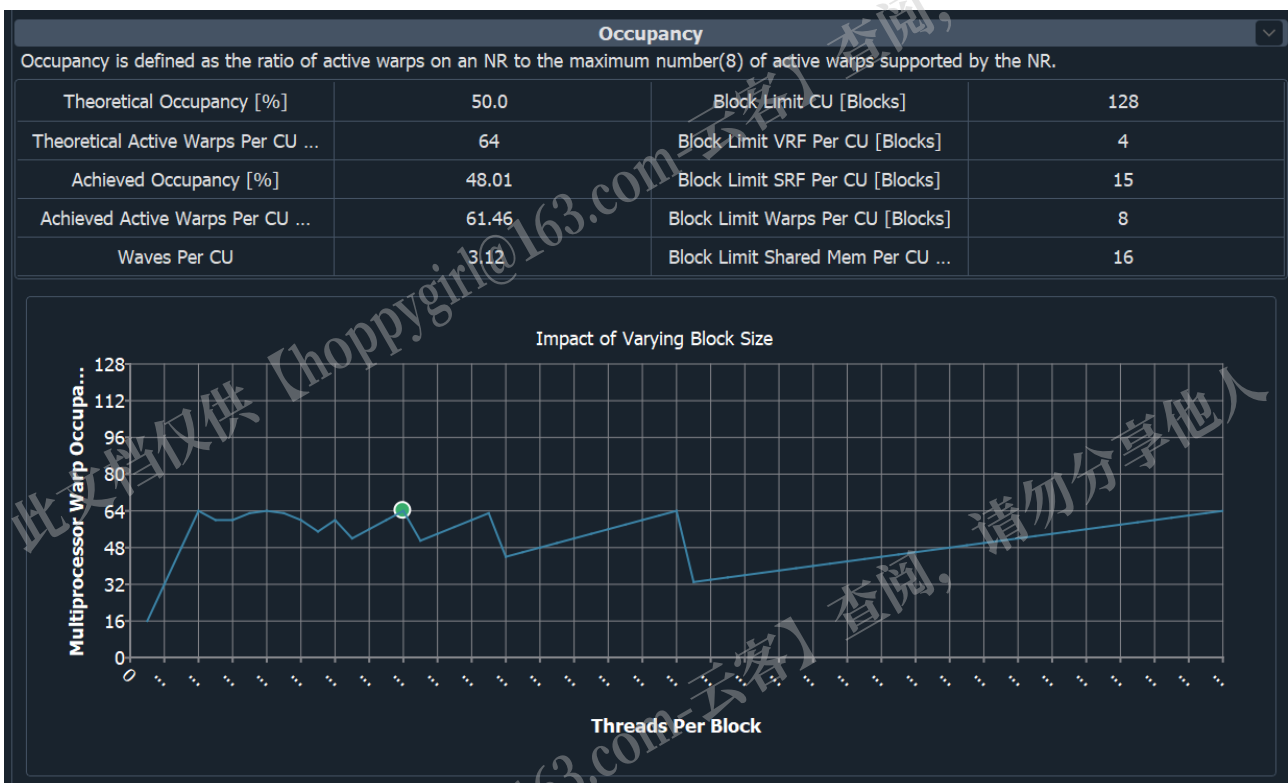


图 77: Occupancy 页面

6.3.3.0.4 Baseline 功能

Baseline 提供数字化和图形化对比的功能，方便用户在多个 kernel 之间进行数字对比，从而直观的展示 kernel 之间的差异。在需要进行对比的 kernel 分析页面点击 Add Baseline 按键进行添加 Baseline，可以最多添加 5

条 baseline 也就是最多可以 6 个 kernel 的数据进行比较。

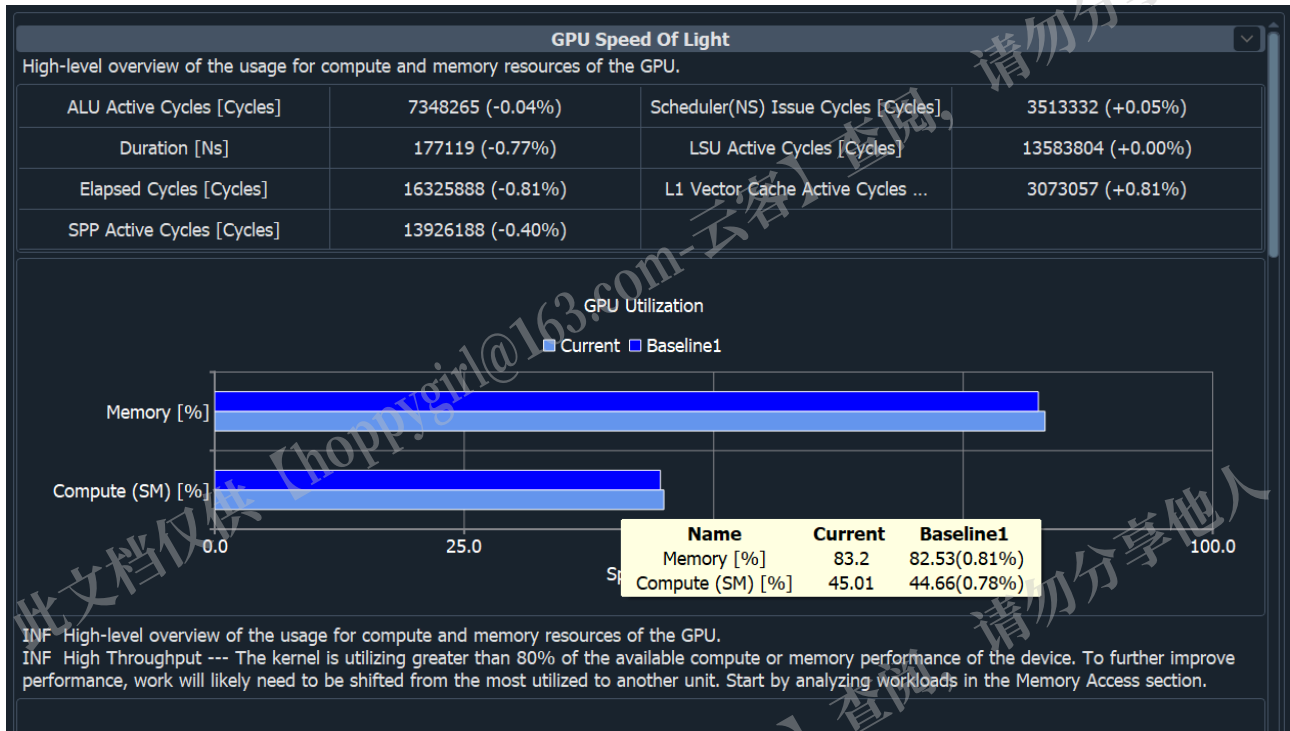


图 78: Baseline 功能页

7 代码调试工具 ixGDB 使用指南

ixGDB 是基于 CUDA-GDB 并提供针对天数智芯加速卡的 CUDA 应用程序的开源调试工具。开发人员可以使用 ixGDB 调试运行在天数智芯加速卡上的 CUDA 应用程序。

目前的 ixGDB 是基于 CUDA-GDB 10.2 版本开发的。

Tip

CUDA-GDB 是 GNU 项目调试器 GDB 的扩展。更多 CUDA-GDB 的介绍请参考: <https://docs.nvidia.com/cuda/cuda-gdb/index.html>

ixGDB 提供以下功能:

- 支持在同一应用程序中同时调试 GPU 和 CPU 代码, 为用户提供无缝调试环境。正如 CUDA C 编程是 C 编程的扩展一样, 使用 ixGDB 进行调试是使用 GDB 进行调试的自然扩展, 您可以像使用 GDB 调试主机代码一样使用 ixGDB 调试主机代码。
- 除了支持调试 C/C++ 程序, 还支持调试所有 CUDA 应用程序, 支持调试 CUDA 驱动程序 API 和 CUDA 运行时 API。
- 支持用户设置断点调试 CUDA 应用程序。

7.1 ixGDB 使用方法

1. 使用 **-g** 选项编译程序, 以编译 example.cu 文件生成 example 程序为例:

```
clang++ -g -std=c++11 example.cu -o example -lcudart
```

-g 选项使编译器在可执行文件中包含调试信息, 并在 `__global__` 函数入口处插入额外代码用于获取设备运行时状态, 这些代码不影响 `__global__` 函数的行为。

注意: 以下情况, 应使用 **-gdwarf-4** 选项, 而非 **-g** 选项,

- 天垓 150 的编译器版本为 3.3.0 及以上
- 智铠 50/100 的编译器版本为 v4.0.0 及以上

使用 **-g** 选项编译 CUDA 程序时, 对优化的影响如下:

- 支持您不使用 **-O** 选项, 即使用编译器默认设置, 开启默认优化: 由于编译器对程序进行优化, 断点位置可能不准确。
- 支持您使用 **-O0** 选项, 即手动关闭编译器默认优化: 有助于避免断点位置不准确的情况。
- 不支持 **-O1** 及以上的优化选项, 否则可能出现程序 hang up 或其它非预期行为。

2. 使用 ixGDB 调试程序, 以 example 程序为例:

```
ixgdb example
```

之后, 您会看到命令行进入 (ixgdb) 模式。

3. 使用 ixGDB 调试选项

请参考 [ixGDB 功能列表](#)。

关于定位线程:

- 从软件维度定位坐标：程序中运行在设备上的线程（thread）隶属于一个线程块（block），进而隶属于一个核函数（kernel），所以可以根据核函数-线程块-线程进行定位。
- 从硬件维度定位坐标：线程（thread）运行在设备硬件的线程（lane）隶属于线程束（warp）、进而隶属于计算单元（sm）和设备（device），所以可以根据设备-计算单元-线程束-线程进行定位。

在定位或切换线程时，您可以使用软件维度（thread, block, kernel）和硬件维度（lane, warp, sm, device）。

7.2 ixGDB 功能列表

该工具提供以下功能：

命令参数和示例	功能类型及详细描述
定位线程	
cuda < coord >	返回当前（in focus）线程的软件/硬件的坐标（current focus） 示例： cuda device sm warp lane block thread cuda kernel block thread
cuda < coord > < num >	切换核函数线程 示例： cuda device 0 sm 1 warp 2 lane 3 cuda block 2 thread 1
设置断点	
break < my_func >	在指定的函数或方法的入口（entry）设置断点
break < my_file.cu:line-num >	设置行断点 示例： break test.cu:40
break < address >	设置地址断点 示例： break *0x12340000
set cuda break_on_launch < option >	设置核函数入口断点，可选参数如下： application：在用户启动的核函数第一条指令处设置断点 system：在所有系统启动的核函数（如 memset）第一条指令处设置断点 all：在所有启动的核函数第一条指令处设置断点 none：取消所有核函数入口断点
检查程序状态	
info cuda devices	显示所有设备的信息

命令参数和示例	功能类型及详细描述
info cuda sms	显示当前设备的所有计算单元的信息
info cuda warps	显示当前计算单元中所有线程束的信息
info cuda lanes	显示当前线程束的所有线程的信息
info cuda kernels	显示所有（active）的核函数的信息
info cuda blocks	显示当前核函数的所有活跃的线程块的信息
info cuda threads	显示当前核函数的所有活跃的线程的信息
info cuda contexts	显示运行在所有设备上的 CUDA 上下文（contexts）信息
info registers	查询或修改设备寄存器代码（register code）
info locals	查看当前的局部变量列表
print <variable name>	查看指定变量
print (@<class> <type>)<address>	查看 GPU memory，支持的 class 有： shared: shared memory 空间 generic: device memory 空间 parameter: kernel 参数空间，address 是参数编号

7.3 常见问题

1. ixGDB 报错 dwarf error cannot handle dw_form_strx1，如何解决？

解决上述报错的方法是，需要将 **-g** 选项替换成 **-gdwarf-4** 选项。在您使用 ixGDB 工具时，如果使用了 **-g** 选项，工具会提示 Iluvatar: replace '-g' with '-gdwarf-4', then rebuild the target and try again.

注意：

- 天垓 150 软件栈，当编译器版本为 v3.3.0 及以上时，需要替换该选项
- 智铠 100/50 软件栈，当编译器版本为 v4.0.0 及以上时，需要替换该选项

详情请参考 [ixGDB 使用方法](#)。

8 加速卡管理工具集 ixDCGM 使用指南

8.1 ixDCGM 工具集概述

Iluvatar Data Center GPU Manager (ixDCGM) 是天数智芯数据中心加速卡管理软件，是一套用于在集群环境中管理和监控天数智芯数据中心加速卡的工具集。ixDCGM 可以独立使用，并可以轻松集成到集群管理工具、资源调度和监控产品中。

ixDCGM 简化了数据中心的加速卡管理，提高了资源可靠性和上线时间，并有助于提高整体基础设施效率。

ixDCGM 当前仅支持运行在 x86_64 架构的 Linux 平台上。

ixDCGM 提供兼容主流产品的 API 接口，详情请参考[支持的 API 列表](#)。

8.2 安装 ixDCGM

支持的平台

ixDCGM 当前仅支持运行在 x86_64 架构的 Linux 平台上。

软件依赖

安装 ixDCGM 前，请您确保已经成功安装：

- 天数智算软件栈
- 随软件栈同时安装的 ixML 工具、Driver API 库、Runtime API 库、ixBLAS 库

安装工具

1. 通过以下方式获取 ixdcgm-`{v.r.m}`_x86_64.run 安装包：

- 登录 [天数智芯官网](#)，进入 **客户支持 > 资源中心** 页面进行下载
- 联系您的应用工程师获取

2. 参考以下命令安装 ixDCGM：

```
$ sudo bash ixdcgm-{v.r.m}_x86_64.run
```

ixDCGM 的默认安装路径为 `/usr/local/ixdcgm`，您也可以在安装时使用 `--prefix` 参数指定安装路径。

ixDCGM 安装完成后，结果显示示例如下：

```
Verifying archive integrity... 100% All good.
Uncompressing IxDCGM 100%
Welcome to use IxDCGM! Now installing IxDCGM library to /usr/local...
Thank you for using IxDCGM!
- library path: /usr/local/ixdcgm/lib64
- binary path: /usr/local/ixdcgm/bin
- header path: /usr/local/ixdcgm/include
```

3. 修改 PATH 和 LD_LIBRARY_PATH 环境变量。以 ixDCGM 的默认安装路径 `/usr/local/ixdcgm` 为例，您需要：

- 为 PATH 环境变量加上 `/usr/local/ixdcgm/bin`
- 为 LD_LIBRARY_PATH 环境变量加上 `/usr/local/ixdcgm/lib64`

8.3 ixDCGM 组件概述

ixDCGM 工具集包含以下组件：

- **用户空间库 libixdcgm.so**：一个用户空间的动态链接库，实现了 ixDCGM 的各种功能，并提供 API 接口
- **主机侧服务端代理软件 ix-hostengine**：依赖 libixdcgm.so 动态库，监视和管理当前服务器上的加速卡
- **主机侧客户端命令行控制程序 ixdcgmi**：与 ix-hostengine 通信，用于控制 ixDCGM 收集相关数据而无需针对编程接口进行构建
- **验证工具集 Iluvatar Validation Suite (ixvs) 及其测试插件**：ixvs 工具用于检测和排除高性能计算环境中天数智芯加速卡常见问题；其测试插件包含：
 - software 插件：用于检查当前软硬件环境是否配置正确
 - diagnostic 插件：执行一系列的运算和比较
 - pcie 插件：用于测试各种情况下的 PCIe 拷贝带宽及延迟
 - memtest 插件：用于检测 HBM 的正确性
 - targeted_power 插件：用于使设备稳定运行在接近设备当前支持的最大功耗下
 - targeted_stress 插件：用于评估当前加速卡是否可以达到理论上的浮点运算目标性能水平
 - pluse 插件：用于验证加速卡在不同工作负载情况下的稳定性
 - targeted_stress 插件：执行 blas 库中矩阵运算，使 GPU 达到指定的目标性能 (GFlops)
 - pluse_test 插件：使 GPU 波动地在最大功耗下工作，模拟产生电流峰值

8.3.1 用户空间库 libixdcgm.so

libixdcgm.so 是一个用户空间的动态链接库，是 ixDCGM 的核心组件，实现了 ixDCGM 的各种功能，并对外提供可使用的 C 语言 API 接口。

ixDCGM 提供以下 API group：

- Administrative
- System
- Field APIs
- Process Statistics
- Job Statistics
- Health Monitor
- Topology
- Metadata
- Modules
- Profiling
- DCGMAPI_Admin_ExecCtrl

具体 API 列表参考如下。

8.3.1.1 支持的 API 列表

Administrative

```

dcgmInit(void);
dcgmShutdown(void);
dcgmStartEmbedded(dcgOperationMode_t opMode, dcgmHandle_t *pDcgmHandle);
dcgmStartEmbedded_v2(dcgStartEmbeddedV2Params_v1 *params);
dcgmStopEmbedded(dcgHandle_t pDcgmHandle);
dcgmConnect(const char *ipAddress, dcgmHandle_t *pDcgmHandle);
dcgmConnect_v2(const char *ipAddress, dcgmConnectV2Params_t *connectParams, dcgmHandle_t
    ↪ *pDcgmHandle);
dcgmDisconnect(dcgHandle_t pDcgmHandle);
dcgmVersionInfo(dcgVersionInfo_t *pVersionInfo);
dcgmHostengineVersionInfo(dcgHandle_t pDcgmHandle, dcgmVersionInfo_t *pVersionInfo);
dcgmHostengineSetLoggingSeverity(dcgHandle_t
    ↪ pDcgmHandle, dcgmSettingsSetLoggingSeverity_t *logging);
*errorString(dcgReturn_t result);
dcgmModuleIdToName(dcgModuleId_t id, char const **name);

```

System

```

dcgmGetAllDevices(dcgHandle_t pDcgmHandle, unsigned int gpuIdList[DCGM_MAX_NUM_DEVICES],
    ↪ int *count);
dcgmGetAllSupportedDevices(dcgHandle_t pDcgmHandle, unsigned int
    ↪ gpuIdList[DCGM_MAX_NUM_DEVICES], int *count);
dcgmGetDeviceAttributes(dcgHandle_t pDcgmHandle, unsigned int gpuId,
    ↪ dcgmDeviceAttributes_t *pDcgmAttr);
dcgmGetEntityGroupEntities(dcgHandle_t dcgmHandle, dcgm_field_entity_group_t
    ↪ entityGroup, dcgm_field_eid_t *entities, int *numEntities, unsigned int flags);
dcgmGroupCreate(dcgHandle_t pDcgmHandle, dcgmGroupType_t type, const char *groupName,
    ↪ dcgmGpuGrp_t *pDcgmGrpId);
dcgmGroupDestroy(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId);
dcgmGroupAddDevice(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId, unsigned int gpuId);
dcgmGroupAddEntity(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId,
    ↪ dcgm_field_entity_group_t entityGroupId, dcgm_field_eid_t entityId);
dcgmGroupRemoveDevice(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId, unsigned int gpuId);
dcgmGroupRemoveEntity(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId,
    ↪ dcgm_field_entity_group_t entityGroupId, dcgm_field_eid_t entityId);
dcgmFieldGroupCreate(dcgHandle_t dcgmHandle, int numFieldIds, unsigned short *fieldIds,
    ↪ const char *fieldGroupName, dcgmFieldGrp_t *dcgmFieldGroupId);
dcgmFieldGroupDestroy(dcgHandle_t dcgmHandle, dcgmFieldGrp_t dcgmFieldGroupId);
dcgmStatusCreate(dcgStatus_t *statusHandle);
dcgmStatusDestroy(dcgStatus_t statusHandle);
dcgmStatusGetCount(dcgStatus_t statusHandle, unsigned int *count);
dcgmStatusPopError(dcgStatus_t statusHandle, dcgmErrorInfo_t *pDcgmErrorInfo);
dcgmStatusClear(dcgStatus_t statusHandle);
dcgmGroupAddEntity(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId,
    ↪ dcgm_field_entity_group_t entityGroupId, dcgm_field_eid_t entityId);
dcgmGroupGetInfo(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupId, dcgmGroupInfo_t
    ↪ *pDcgmGroupInfo);
dcgmGroupGetAllIds(dcgHandle_t pDcgmHandle, dcgmGpuGrp_t groupIdList[], unsigned int
    ↪ *count);

```

Configuration

```
dcgmConfigSet(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmConfig_t
    ↪ *pDeviceConfig, dcgmStatus_t statusHandle);
dcgmConfigGet(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmConfigType_t type, int
    ↪ count, dcgmConfig_t deviceConfigList[], dcgmStatus_t statusHandle);
dcgmConfigEnforce(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmStatus_t
    ↪ statusHandle);
```

Field

```
dcgmEntitiesGetLatestValues(dcgMHandle_t pDcgMHandle, dcgmGroupEntityPair_t
    ↪ entities[], unsigned int entityCount, unsigned short fields[], unsigned int
    ↪ fieldCount, unsigned int flags, dcgmFieldValue_v2 values[]);
dcgmEntityGetLatestValues(dcgMHandle_t pDcgMHandle, dcgm_field_entity_group_t
    ↪ entityGroup, int entityId, unsigned short fields[], unsigned int count, dcgmFieldValue_v1
    ↪ values[]);
dcgmGetFieldSummary(dcgMHandle_t pDcgMHandle, dcgmFieldSummaryRequest_t *request);
dcgmGetLatestValues(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmFieldGrp_t
    ↪ fieldGroupId, dcgmFieldValueEnumeration_f enumCB, void *userData);
dcgmGetLatestValues_v2(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmFieldGrp_t
    ↪ fieldGroupId, dcgmFieldValueEntityEnumeration_f enumCB, void *userData);
dcgmGetLatestValuesForFields(dcgMHandle_t pDcgMHandle, int gpuId, unsigned short
    ↪ fields[], unsigned int count, dcgmFieldValue_v1 values[]);
dcgmGetValuesSince(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmFieldGrp_t
    ↪ fieldGroupId, long long sinceTimestamp, long long
    ↪ *nextSinceTimestamp, dcgmFieldValueEnumeration_f enumCB, void *userData);
dcgmGetValuesSince_v2(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmFieldGrp_t
    ↪ fieldGroupId, long long sinceTimestamp, long long
    ↪ *nextSinceTimestamp, dcgmFieldValueEntityEnumeration_f enumCB, void *userData);
dcgmUnwatchFields(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmFieldGrp_t
    ↪ fieldGroupId);
dcgmWatchFields(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmFieldGrp_t
    ↪ fieldGroupId, long long updateFreq, double maxKeepAge, int maxKeepSamples);
```

Process Statistics

```
dcgmWatchPidFields(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, long long
    ↪ updateFreq, double maxKeepAge, int maxKeepSamples);
dcgmGetPidInfo(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, dcgmPidInfo_t *pidInfo);
```

Job Statistics

```
dcgmJobGetStats(dcgMHandle_t pDcgMHandle, char jobId[64], dcgmJobInfo_t *pJobInfo);
dcgmJobRemove(dcgMHandle_t pDcgMHandle, char jobId[64]);
dcgmJobRemoveAll(dcgMHandle_t pDcgMHandle);
dcgmJobStartStats(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, char jobId[64]);
dcgmJobStopStats(dcgMHandle_t pDcgMHandle, char jobId[64]);
dcgmWatchJobFields(dcgMHandle_t pDcgMHandle, dcgmGpuGrp_t groupId, long long
    ↪ updateFreq, double maxKeepAge, int maxKeepSamples);
```

Health Monitor

```
dcgmHealthSet(dcgHandle_t pDcgHandle, dcgmGpuGrp_t groupId, dcgmHealthSystems_t
↳ systems);
dcgmHealthSet_v2(dcgHandle_t pDcgHandle, dcgmHealthSetParams_v2 *params);
dcgmHealthGet(dcgHandle_t pDcgHandle, dcgmGpuGrp_t groupId, dcgmHealthSystems_t
↳ *systems);
dcgmHealthCheck(dcgHandle_t pDcgHandle, dcgmGpuGrp_t groupId, dcgmHealthResponse_t
↳ *results);
```

Topology

```
dcgmGetDeviceTopology(dcgHandle_t pDcgHandle, unsigned int gpuId, dcgmDeviceTopology_t
↳ *pDcgDeviceTopology);
dcgmSelectGpusByTopology(dcgHandle_t pDcgHandle, uint64_t inputGpuIds, uint32_t
↳ numGpus, uint64_t *outputGpuIds, uint64_t hintFlags);
dcgmGetGroupTopology(dcgHandle_t pDcgHandle, dcgmGpuGrp_t groupId, dcgmGroupTopology_t
↳ *pDcgGroupTopology);
```

Metadata

```
dcgmIntrospectGetHostengineMemoryUsage(dcgHandle_t pDcgHandle, dcgmIntrospectMemory_t
↳ *memoryInfo, int waitIfNoData);
dcgmIntrospectGetHostengineCpuUtilization(dcgHandle_t
↳ pDcgHandle, dcgmIntrospectCpuUtil_t *cpuUtil, int waitIfNoData);
```

Modules

```
dcgmModuleDenylist(dcgHandle_t pDcgHandle, dcgmModuleId_t moduleId);
dcgmModuleGetStatuses(dcgHandle_t pDcgHandle, dcgmModuleGetStatuses_t *moduleStatuses);
```

Profiling

```
dcgmProfGetSupportedMetricGroups(dcgHandle_t pDcgHandle, dcgmProfGetMetricGroups_t
↳ *metricGroups);
```

DCGMAPI_Admin_ExecCtrl

```
dcgmUpdateAllFields(dcgHandle_t pDcgHandle, int waitForUpdate);
```

8.3.1.2 支持的 Field 列表

当前，ixDCGM 支持以下 Field 列表，其定义与主流生态兼容。

DCGM_FI_DRIVER_VERSION
 DCGM_FI_NVML_VERSION
 DCGM_FI_PROCESS_NAME
 DCGM_FI_DEV_COUNT
 DCGM_FI_CUDA_DRIVER_VERSION
 DCGM_FI_DEV_NAME
 DCGM_FI_DEV_NVML_INDEX
 DCGM_FI_DEV_SERIAL
 DCGM_FI_DEV_UUID
 DCGM_FI_DEV_MINOR_NUMBER
 DCGM_FI_DEV_PCI_BUSID
 DCGM_FI_DEV_PCI_COMBINED_ID
 DCGM_FI_DEV_PCI_SUBSYS_ID
 DCGM_FI_GPU_TOPOLOGY_PCI
 DCGM_FI_GPU_TOPOLOGY_NVLINK
 DCGM_FI_GPU_TOPOLOGY_AFFINITY
 DCGM_FI_DEV_CUDA_COMPUTE_CAPABILITY
 DCGM_FI_DEV_COMPUTE_MODE
 DCGM_FI_DEV_PERSISTENCE_MODE
 DCGM_FI_DEV_CUDA_VISIBLE_DEVICES_STR
 DCGM_FI_DEV_CPU_AFFINITY_0
 DCGM_FI_DEV_CPU_AFFINITY_1
 DCGM_FI_DEV_CPU_AFFINITY_2
 DCGM_FI_DEV_CPU_AFFINITY_3
 DCGM_FI_DEV_MEM_AFFINITY_0
 DCGM_FI_DEV_MEM_AFFINITY_1
 DCGM_FI_DEV_MEM_AFFINITY_2
 DCGM_FI_DEV_MEM_AFFINITY_3
 DCGM_FI_DEV_SM_CLOCK
 DCGM_FI_DEV_MEM_CLOCK
 DCGM_FI_DEV_CLOCK_THROTTLE_REASONS
 DCGM_FI_DEV_MAX_SM_CLOCK
 DCGM_FI_DEV_MAX_MEM_CLOCK
 DCGM_FI_DEV_MEMORY_TEMP
 DCGM_FI_DEV_GPU_TEMP
 DCGM_FI_DEV_GPU_MAX_OP_TEMP
 DCGM_FI_DEV_POWER_USAGE
 DCGM_FI_DEV_SLOWDOWN_TEMP
 DCGM_FI_DEV_SHUTDOWN_TEMP
 DCGM_FI_DEV_POWER_MGMT_LIMIT
 DCGM_FI_DEV_POWER_MGMT_LIMIT_DEF
 DCGM_FI_DEV_ENFORCED_POWER_LIMIT
 DCGM_FI_DEV_PSTATE
 DCGM_FI_DEV_FAN_SPEED
 DCGM_FI_DEV_PCIE_TX_THROUGHPUT
 DCGM_FI_DEV_PCIE_RX_THROUGHPUT
 DCGM_FI_DEV_PCIE_REPLAY_COUNTER
 DCGM_FI_DEV_GPU_UTIL
 DCGM_FI_DEV_MEM_COPY_UTIL
 DCGM_FI_DEV_DEC_UTIL

```
DCGM_FI_DEV_PCIE_MAX_LINK_GEN
DCGM_FI_DEV_PCIE_MAX_LINK_WIDTH
DCGM_FI_DEV_PCIE_LINK_GEN
DCGM_FI_DEV_PCIE_LINK_WIDTH
DCGM_FI_DEV_POWER_VIOLATION
DCGM_FI_DEV_THERMAL_VIOLATION
DCGM_FI_DEV_FB_TOTAL
DCGM_FI_DEV_FB_FREE
DCGM_FI_DEV_FB_USED
DCGM_FI_DEV_FB_RESERVED
DCGM_FI_DEV_FB_USED_PERCENT
DCGM_FI_DEV_ECC_CURRENT
DCGM_FI_DEV_ECC_SBE_VOL_DEV
DCGM_FI_DEV_ECC_DBE_VOL_DEV DCGM_FI_PROF_SM_ACTIVE
DCGM_FI_PROF_SM_OCCUPANCY
DCGM_FI_PROF_DRAM_ACTIVE
DCGM_FI_PROF_PCIE_TX_BYTES
DCGM_FI_PROF_PCIE_RX_BYTES
DCGM_FI_DEV_DIAG_DIAGNOSTIC_RESULT
DCGM_FI_DEV_DIAG_PCIE_RESULT
DCGM_FI_DEV_DIAG_TARGETED_STRESS_RESULT
DCGM_FI_DEV_DIAG_TARGETED_POWER_RESULT
DCGM_FI_DEV_DIAG_MEMTEST_RESULT
DCGM_FI_DEV_DIAG_PULSE_TEST_RESULT
DCGM_FI_DEV_DIAG_SOFTWARE_RESULT
```

8.3.2 主机侧服务端代理软件 ix-hostengine

ix-hostengine 是主机侧服务端代理软件，依赖 libixdcgm.so 动态库。ix-hostengine 的主要工作是将 ixDCGM 库实例化为持久的独立进程，用于监视和管理当前服务器上的加速卡。

8.3.2.1 ix-hostengine 功能列表

ix-hostengine 提供以下功能。

命令参数	命令描述
-h, --help	显示使用信息并退出
--version	显示版本信息并退出
-f <FILENAME>, --log-filename <FILENAME>	指定 ix-hostengine 应使用的文件名以转储日志记录信息

命令参数	命令描述
--log-level <LEVEL>	指定日志记录级别，默认值为 INFO 支持以下日志记录级别选项： NONE：禁用日志记录 FATAL：仅将日志级别设置为 FATAL ERROR：将日志级别设置为 ERROR 及以上 WARN：将日志级别设置为 WARNING 及以上 INFO：将日志级别设置为 INFO 及以上 DEBUG：将日志级别设置为 DEBUG 及以上 VERB：将日志级别设置为 VERB 及以上
-b <IP_ADDRESS>, --bind-interface <IP_ADDRESS>	指定 ix-hosegnine 应侦听的网络接口的 IP 地址，ALL 表示绑定到所有接口
-d <SOCKET_PATH>, --domain-socket <SOCKET_PATH>	指定启动 ix-hostengine 的 Unix 域套接字路径，指定该选项时，不会打开任何 TCP 监听端口
-p <PORT>, --port <PORT>	指定 ix-hosegnine 的端口
-n, --no-daemon	以非 daemon (守护进程) 的方式启动 ix-hostengine
-t, --term	结束已经启动的 ix-hosegnine
--, --ignore_rest	忽略该选项后的所有命令参数

8.3.2.2 ix-hostengine 使用示例

示例 1

以普通用户身份和默认参数启动 ix-hostengine。

```
$ ./ix-hostengine
```

Note

*：这里的权限问题仅有 Linux 系统的用户组策略决定，ixDCGM 本身并未对系统权限做额外的要求。

示例 2

以 root 用户启动 ix-hostengine，并绑定到 5555 端口进行监听。

```
$ sudo ./ix-hostengine --port 5555
```

示例 3

结束已经启动的 ix-hostengine。

```
$ sudo ./ix-hostengine/ix-hostengine -t
```

8.3.3 验证工具集 Iluvatar Validation Suite (ixvs) 及其测试插件

ixvs 是用于检测和排除高性能计算环境中天数智芯加速卡常见问题的工具。ixvs 主要关注软件和系统配置，诊断以及系统基本硬件性能等问题。

ixvs 提供的测试插件有：

- `plugins/libpluginSoftware.so`: software 插件用于检查当前的软硬件环境是否配置正确，以便于执行用户的应用程序
- `plugins/libpluginDiagnostic.so`: diagnostic 插件用于执行一系列的运算和比较，用于检测加速卡的 SM、HBM、PCIe 的正确性
- `plugins/libpluginPcie.so`: pcie 插件用于测试各种情况下的 PCIe 拷贝带宽及延迟
- `plugins/libpluginMemtest.so`: memtest 插件通过向加速卡的存储空间写入不同的测试模板并校验，用于检测 HBM 的正确性
- `plugins/libpluginTargetedPower.so`: targeted_power 插件通过逐步增加计算任务的负载，使设备稳定运行在接近设备当前支持的最大功耗下
- `plugins/libpluginTargetedStress.so`: targeted_stress 插件执行 blas 库中矩阵运算，使 GPU 达到指定的目标性能 (GFlops)
- `plugins/libpluginPulse.so`: pluse 插件使 GPU 波动地在最大功耗下工作，模拟产生电流峰值

8.3.3.1 ixvs 功能列表

ixvs 提供以下功能。

命令参数	命令描述
<code>-h, --help</code>	显示使用信息并退出
<code>--version</code>	显示版本信息并退出
<code>--ixdcmHostname <ixDCGM hostname></code>	指定 ix-hostengine 的 IP 地址，如果 ix-hostengine 以 socket 方式启动，该项要加上前缀“unix://”
<code>-t, --listTests</code>	列出可通过 ixvs 执行的测试
<code>-v, --verbose</code>	输出每个 GPU 执行结果
<code>-j, --jsonOutput</code>	json 格式输出打印输出结果
<code>--configless</code>	ixvs 执行在 configless 模式下，在所有 GPU 上执行“xlong”级别测试
<code>--specifiedtest <specific test to run></code>	执行指定的特定测试
<code>--parameters <parameters to set for tests></code>	在 configless 模式下，为插件测试项设定执行参数
<code>--quiet</code>	禁止默认控制台输出 可以通过查看日志和返回代码以查找错误
COREX01-BI430-UG04-01	141 V4.3.0 (x86) V4.3.6 (ARM)

命令参数	命令描述
-p <path to plugins>, --pluginpath <path to plugins>	为 ixvs 插件指定自定义路径 默认插件搜索路径为 ixvs 二进制可执行文件同级别的 plugins 子目录
-l <debug file>, --debugLogFile <debug file>	指定调试日志文件路径 如果未指定，则默认使用当前目录的 ixvs.log 文件作为日志文件，如果该文件已经存在，新的 log 会以追加的方式添加到文件尾
-d <debug level>, --debugLevel <debug level>	指定日志记录级别，默认值为 DEBUG 支持的调试级别有：NONE, FATAL, ERROR, WARN, INFO, DEBUG, VERB
-g, --listGpus	列出系统中可以被 ixvs 检测的 GPU
-i <indexes>, --indexes <indexes>	指定需要使用 ixvs 检测的 GPU，指定多个 GPU 中间需以逗号分隔
--, --ignore_rest	忽略该项后的所有参数

8.3.3.2 ixvs 使用示例

示例 1

以默认参数启动 ixvs。

```
$ ./ixvs
```

示例 2

列出当前系统中可被 ixvs 检测的 GPU。

```
$ ./ixvs --listGpus
```

示例 3

列出可以被 ixvs 执行的测试用例。

```
$ ./ixvs --listTests
```

示例 4

在指定设备上执行特定的测试用例。

```
$ ./ixvs -i 0 --specifiedtest memtest
```

示例 5

在指定设备上执行特定的测试用例，并指定测试项参数。

```
$ ./ixvs -i 0 --specifiedtest targeted_stress --parameters
  ↪ targeted_stress.test_duration=600 --configless
```

8.3.4 主机侧客户端命令行控制程序 ixdcgmi

ixdcgmi 是 ixDCGM 的命令行工具，用于实现 ixDCGM 的各个功能。

以下是 ixdcgmi 命令行工具提供的功能。

命令参数	命令描述
subsystem	ixdcgmi 的子系统命令，子系统有： discovery group fieldgroup topo health dmon diag profile policy introspect set
-v	查询 ixdcgmi 和 hostengine 版本信息
-h, --help	列出指定主机上所有 GPU
--version	显示 ixdcgmi 版本信息
--	让 ixdcgmi 忽略后续参数

8.3.4.1 ixdcgmi diag

ixdcgmi diag 子系统命令主要用于诊断 GPU 的健康状态和性能。

功能列表

ixdcgmi diag 提供以下功能。

命令参数	命令描述
-g, --group	指定待查询的 group ID

命令参数	命令描述
--host	指定待查询的主机 IP，不指定则默认查询本机
-r, --run	运行一次诊断 参数可以是数字 (较大数字包含较小数字的测试): 1 - Quick 2 - Medium 3 - Long 4 - Extended 也可以指定具体的测试项目，如： ixdcgmi diag -r "pcie,diagnostic" 目前测试项目有： diagnostic, memtest, pcie, pulse_test, software, targeted_power, targeted_stress
-p, --parameters	为测试项设置执行参数，格式为： test_name.variable_name=variable_value; test_name.variable_name2=variable_value
-i, --gpuList	指定 GPU ID，用逗号分隔
-v, --verbose	显示每项测试的 info 和 warning
--debugLogFile	指定 debug log 文件的名字 如果只指定 debug 等级，文件名默认为"ixvs.log"
-d, --debugLevel	指定 debug level，可选项有： NONE, FATAL, ERROR, WARN, INFO, DEBUG, VERB
-j, --json	输出显示为 json 格式
-t, --timeout	指定超时的时间 (单位秒)，不指定默认则无限时间
-h, --help	显示用法的信息并退出
--, --ignore_rest	忽略后面的参数

用法和示例

ixdcgmi diag 命令行的使用方法如下所示。

```
$ ixdcgmi diag --host <IP/FQDN> -g <groupId> -p <test_name.variable_name=variable_value;  
test_name.variable_name2=variable_value> -r <diag> -i <gpuList> -v --debugLogFile <debug  
↪ file>  
-j -t <timeout>
```

Important

注意，启动 ix-hostengine 时，如果 ixDCGM 不是安装到默认路径，需要为 ix-hostengine 指明 ixvs 的路径，如：**export IXVS_BIN_PATH=/<path>/ixdcgm/bin; ix-hostengine**

示例 1

对 group 0 进行 pcie 诊断。

```
$ ixdcgmi diag -g 0 -r pcie
```

示例 2

对 GPU 2 进行 4 级别的诊断，显示详细信息。

```
$ ixdcgmi diag -i 2 -r 4 -v
```

示例 3

对 GPU 2 进行 software 诊断，输出 json 格式。

```
$ ixdcgmi diag -i 2 -r software -j
```

8.3.4.2 ixdcgmi discovery

ixdcgmi discovery 子系统命令主要用于查询指定主机上的 GPU 信息。

功能列表

ixdcgmi discovery 提供以下功能。

命令参数	命令描述
-g, --group	指定待查询的 group ID
--host	指定待查询的主机 IP，不指定则默认查询本机
-l, --list	列出指定主机上的所有 GPU
-i, --info	指定待查询的信息，包括： [default = atp] a - device info p - power limits t - thermal limits
--gpuid	指定待查询的 GPU ID
-v, --verbose	显示每张 GPU 的详细信息
-h, --help	显示用法的信息并退出
--, --ignore_rest	忽略后面的参数

用法和示例

ixdcgmi discovery 命令行的使用方法如下所示。

```
$ ixdcgmi discovery --host <IP/FQDN> -l
$ ixdcgmi discovery --host <IP/FQDN> -i <flags> -g <groupId> -v
```

示例 1

列出所有 group 信息。

```
$ ixdcgmi discovery -l
```

示例 2

列出 group 0 的设备，温度限制，功耗限制的信息。

```
$ ixdcgmi discovery -i atp -g 0
```

示例 3

列出 group 0 的设备，温度限制，功耗限制的详细信息。

```
$ ixdcgmi discovery -i atp -g 0 -v
```

示例 4

列出 GPU 0 的设备，温度限制，功耗限制的信息。

```
$ ixdcgmi discovery -i atp --gpuid 0
```

8.3.4.3 ixdcgmi group

ixdcgmi group 子系统命令用于管理主机上的 GPU 集群。

功能列表

ixdcgmi group 提供以下功能。

命令参数	命令描述
--host	指定待查询的主机 IP，不指定则默认查询本机
-g, --group	指定待查询的 group ID (此处 group 是子命令，与参数 -g , --group 有冲突，无法在--help 中显示)
-l, --list	列出指定主机上所有 group
-i, --info	根据指定的 group 显示信息
-d, --delete	指定 ID 删除一个 group
-c, --create	指定名字创建一个 group

命令参数	命令描述
-r, --remove	指定 GPU ID (可以是列表, 用"," 分隔), 从一个 group 中删除该 GPU
-a, --add	指定 GPU ID (可以是列表, 用"," 分隔), 添加到一个 group 中
-j, --json	返回结果用 json 格式显示, 用于 -l, --list 时
-h, --help	显示用法的信息并退出
--, --ignore_rest	忽略后面的参数

用法和示例

ixdcgmi group 命令行的使用方法如下所示。

```
$ ixdcgmi group --host <IP/FQDN> -l -j
$ ixdcgmi group --host <IP/FQDN> -c <groupName>
$ ixdcgmi group --host <IP/FQDN> -c <groupName> -a <entityId>
$ ixdcgmi group --host <IP/FQDN> -d <groupId>
$ ixdcgmi group --host <IP/FQDN> -g <groupId> -i -j
$ ixdcgmi group --host <IP/FQDN> -g <groupId> -a <entityId>
$ ixdcgmi group --host <IP/FQDN> -g <groupId> -r <entityId>
```

示例 1

显示 group 信息。

```
$ ixdcgmi group -l
```

示例 2

json 格式显示 group 信息。

```
$ ixdcgmi group -l -j
```

示例 3

创建名字为 test 的 group。

```
$ ixdcgmi group -c test
```

示例 4

给 ID 为 4 的 group 添加 ID 为 1, 3, 5 的 GPU。

```
$ ixdcgmi group -g 4 -a 1,3,5
```

示例 5

创建名字为 test2 的 group，同时添加 ID 为 1, 3, 5 的 GPU 到该 group。

```
$ ixdcgmi group -c test2 -a 1,3,5
```

示例 6

ID 为 4 的 group，删除 ID 为 3 的 GPU。

```
$ ixdcgmi group -g 4 -r 3
```

示例 7

显示 ID 为 5 的 group 信息。

```
$ ixdcgmi group -g 5 -i
```

示例 8

json 格式显示 ID 为 5 的 group 信息。

```
$ ixdcgmi group -g 5 -i -j
```

示例 9

删除 ID 为 4 的 group。

```
$ ixdcgmi group -d 4
```

8.3.4.4 ixdcgmi fieldgroup

ixdcgmi fieldgroup 子系统命令主要用于管理和监控 GPU 的多种性能指标和健康状态。通过 Field Group，您可以将一组相关的 GPU 指标 (如温度、功耗、显存使用率等) 组合在一起，方便集中监控和分析。

功能列表

ixdcgmi fieldgroup 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上的 host enging, [default = localhost]
-l, --list	显示指定主机上的所有 fieldgroup
-i, --info	显示指定 fieldgroup 的信息

命令参数	命令描述
-d, --delete	删除指定的 fieldgroup
-c, --create	输入 fieldgroup 名字并创建一个 fieldgroup
-g, --fieldgroup	指定 fieldgroup
-f, --fieldids	创建 fieldgroup 时, 指定 field ID 加入该 fieldgroup 使用"," 分隔指定多个 ID
-j, --json	以 json 格式打印输出

用法和示例

ixdcgmi fieldgroup 命令行的使用方法如下所示。

```
$ ixdcgmi fieldgroup --host <IP/FQDN> -l -j
$ ixdcgmi fieldgroup --host <IP/FQDN> -c <fieldGroupName> -f <fieldIds>
$ ixdcgmi fieldgroup --host <IP/FQDN> -i -g <fieldGroupId> -j
$ ixdcgmi fieldgroup --host <IP/FQDN> -d -g <fieldGroupId>
```

示例 1

列出本机的 fieldgroup。

```
$ ixdcgmi fieldgroup -l
```

示例 2

以 json 格式列出本机 group。

```
$ ixdcgmi fieldgroup -l -j
```

示例 3

创建一个名称为 test 的 group, 加入 100, 101 两个 fileld。

```
$ ixdcgmi fieldgroup -c test -f 100,101
```

示例 4

查看 ID 为 6 的 fieldgroup 信息。

```
$ ixdcgmi fieldgroup -i -g 6
```

示例 5

查看 ID 为 6 的 fieldgroup 信息 json 格式。

```
$ ixdcgmi fieldgroup -i -g 6 -j
```

示例 6

删除 ID 为 6 的 fieldgroup。

```
$ ixdcgmi fieldgroup -d -g 6
```

8.3.4.5 ixdcgmi health

ixdcgmi health 子系统命令行主要用于检查 GPU 的健康状态。

功能列表

ixdcgmi health 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上的 host enging, [default = localhost]
-g, --group	指定主机上待查询的 GPU 组
-s, --set	设置 health 监测对象，有效值有： a - all watches p - PCIe watches (*) t - thermal and power watches (*) a - all watches (*), (*) watch requires 60 sec before first query. [default = p]
-f, --fetch	查看监测的对象状态，On/Off
--clear	把所有监测对象状态置为 Off
-c, --check	检查当前监测的对象中是否发生任何错误或警告，打印健康信息
-m, --max-keep-age	ixDCGM 缓存监测对象样本的最长时间，单位是秒
-u, --update-interval	获取监测健康状态的频率，即监测对象 sample 值的更新频率，单位是秒
-j, --json	以 json 格式打印输出

用法和示例

ixdcgmi health 命令行的使用方法如下所示。

```
$ ixdcgmi health --host <IP/FQDN> -g <groupId> -s <flags> -m <sec> -u <sec>
$ ixdcgmi health --host <IP/FQDN> -g <groupId> -f
$ ixdcgmi health --host <IP/FQDN> -g <groupId> -c
```

示例 1

为本地主机，GPU group 2 设置 PCIe 监测对象，参数为默认值。

```
$ ixdcgmi health -g 2 -s p
```

示例 2

获取本地主机 GPU group 2 的监测对象状态。

```
$ ixdcgmi health -g 2 -f
```

示例 3

检查本地主机 GPU group 2 的监测对象健康状态。

```
$ ixdcgmi health -g 2 -c
```

8.3.4.6 ixdcgmi dmon

ixdcgmi dmon 子系统命令主要用于实时监控 GPU 的性能指标。

功能列表

ixdcgmi dmon 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上的 host enging, [default = localhost]
-l, --list	列出主机上支持的所有 field，包括 long names, short names 和 field IDs
-g, --group-id	指定主机上待查询的 GPU 组
-i, --entity-id	指定执行 dmon 的 GPU List (用"," 分隔) [default = -1, 表示指定所有当前设备支持的 GPU] 执行 ixdcgmi discovery -l 命令可查看所有可用的 GPU
-f, --field-group-id	指定主机上待查询的 field group
-e, --field-id	指定主机上待查询的 field List (用"," 分隔)
-d, --delay	ixdcgm dmon 获取 field 结果和打印输出的频率，单位是 ms [default = 1000 msec, Minimum value = 1 msec]
-c, --count	ixdcgm dmon 获取 field 结果和打印输出的次数 [default - runs forever]

用法和示例

ixdcgmi dmon 命令行的使用方法如下所示。

```
$ ixdcgmi dmon --host <IP/FQDN> -l
$ ixdcgmi dmon --host <IP/FQDN> -i <flags> /-g <groupId> -e <flags> /-f <fieldgroupId> -c
  <flags>
```

示例 1

列出主机上支持的所有 field。

```
$ ixdcgmi dmon -l
```

示例 2

查询主机上 GPU0 和 GPU1，field ID 为 100，101 值的输出，输出 50 次。

```
$ ixdcgmi dmon -i 0,1 -c 50 -e 100,101
```

8.3.4.7 ixdcgmi topo

ixdcgmi topo 子系统命令主要用于查询和显示 GPU 设备之间的拓扑关系。

功能列表

ixdcgmi topo 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上的 host enging, [default = localhost]
-g, --group	指定待查询的 GPU Group ID，并打印其 topo 信息
--gpuid	指定待查询的 GPU ID，并打印其 topo 信息
-j, --json	以 json 格式打印 topo 信息

用法和示例

ixdcgmi topo 命令行的使用方法如下所示。

```
$ ixdcgmi topo --host <IP/FQDN> -g <groupId>
$ ixdcgmi topo --host <IP/FQDN> --gpuid <gpuId>
```

示例 1

指定 GPU Group ID 为 3，查询其 topo 信息，Group 3 包含 GPU1 和 GPU2。

```
$ ixdcgmi topo -g 3
```

示例 2

指定 GPU ID 为 3，查询其 topo 信息。

```
$ ixdcgmi topo --gpuid 3
```

8.3.4.8 ixdcgmi profile

ixdcgmi profile 子系统命令主要用于配置和管理 GPU 的性能分析功能，可以帮助用户优化 GPU 的使用效率，通过监控和分析 GPU 的性能指标来识别潜在的性能瓶颈。

Note

ixdcgmi profile 仅用作列出支持的 profiling metrics 条目，您可以使用 **ixdcgmi dmon** 命令获取 profiling metrics 的值。

功能列表

ixdcgmi profile 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上的 host enging, [default = localhost]
-g, --group-id	指定待查询的 Profiling Metrics 信息的 GPU Group ID
-i, --entity-id	指定待查询的 Profiling Metrics 信息的 GPU ID (ID 之间使用"," 分隔)
-l, --list	列出指定 GPU 或 GPU Group 的可用 Profiling Metrics
-j, --json	以 json 格式打印 topo 信息

用法和示例

ixdcgmi profile 命令行的使用方法如下所示。

```
$ ixdcgmi profile --host <IP/FQDN> -g <groupId> -l
$ ixdcgmi profile --host <IP/FQDN> -i <gpuId> -l
```

示例 1

指定 GPU Group ID 为 1，查询其可用的 Profiling Metrics。

Note

GPU Group 1 包含 GPU0、GPU2、GPU3、GPU5，但是只会选取 Group 中的第一个 GPU 对象进行查询。

```
$ ixdcgmi profile -g 1 -l
```

示例 2

指定 GPU ID 为 0，查询其可用的 Profiling Metrics。

```
$ ixdcgmi profile -i 0 -l
```

8.3.4.9 ixdcgmi policy

ixdcgmi policy 子系统命令主要用于管理 GPU 的策略，包括错误处理、通知机制和自动操作。

ixdcgmi policy 最简单的工作形式是 ixDCGM 在满足目标条件时通知客户端 (notify)，除此之外不会执行任何进一步的操作 (action)。

Note

当前 ixdcgmi policy 命令仅支持 notify。

功能列表

ixdcgmi policy 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上的 host enging, [default = localhost]
-g, --group-id	指定主机上待设置和查询的 GPU 组
--set	设置当前的事件违规策略，形式 --set action,validation ，当前不支持设置 action 行为 set - 使用默认值 --set 0,0 action - 当指定的违规事件发生时采取的动作 validation - 当执行 GPU Reset action 后，进一步对 GPU 执行的验证行为 (通常是执行 ixvs 来验证)
--get	获取当前的 violation policy
-v, --verbose	分别显示 GPU 组中每个 GPU 的 violation policy
--clear	清除当前的事件违规策略
--reg	注册用于更新 policy 的进程，该进程处在无限循环中，用于等待接收自 policy manager 的消息回调，并将发生的违规事件、发生时间打印在控制台 注册的违规事件值的更新间隔为 10s
-T, --maxtemp	指定在触发违规之前 GPU 组中 GPU 可以达到的最高温度
-P, --maxpower	指定在触发违规之前 GPU 组中 GPU 可以达到的最大功率
-e, --eccerrors	将 ECC double bit error 添加到策略条件中

命令参数	命令描述
-x, --xiderrors	将 XID (Exception ID, 异常报错码), error 添加到策略条件中
-p, --pcierrors	将 PCIe replay error 添加到策略条件中
-j, --json	终端以 json 格式输出显示
--, --ignore_rest	忽略该标签以后所有的参数

用法和示例

ixdcgmi policy 命令行的使用方法如下所示。

```
$ ixdcgmi policy --host <IP/FQDN> -g <groupId> --set <0, 0> -T <max> -P <max> -e -p
$ ixdcgmi policy --host <IP/FQDN> -g <groupId> --get
$ ixdcgmi policy --host <IP/FQDN> -g <groupId> --reg
```

示例 1

指定 GPU Group ID 为 1, 设置违规 policy, 以及指定触发违规之前 GPU 可以达到的最高温度 70 摄氏度和最大功率 80W。

```
$ ixdcgmi policy -g 1 --set 0,0 -P 80 -T 70
```

示例 2

指定 GPU Group ID 为 1, 注册用于输出 policy 违规事件的进程。

```
$ ixdcgmi policy -g 1 --reg
```

8.3.4.10 ixdcgmi introspect

ixdcgmi introspect 子命令用于获取 ix-hostengine 进程本身所占用系统的实时内存大小, 以及 CPU 占用率。

功能列表

ixdcgmi introspect 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP, 以连接指定主机上 ix-hostengine, [default = localhost], 或指定 Unix 域套接字"unix://"
-s, --show	输出 ix-hostengine 进程本身占用信息, 必须跟随 -H, --hostengine 参数

命令参数	命令描述
-H, --hostengine	指定查询对象为 ix-hostengine 进程
--, --ignore_rest	忽略该标签以后所有的参数

用法和示例

ixdcgmi introspect 命令行的使用方法如下所示。

```
$ ixdcgmi introspect --host <IP/FQDN> -s -H
```

示例

指定查询本地 localhost 上 ix-hostengine 进程的系统占用信息。

```
$ ixdcgmi introspect -s -H
```

8.3.4.11 ixdcgmi set

ixdcgmi set 子命令用于控制 ix-hostengine 和 ixdcgmi 的 log 输出级别。

功能列表

ixdcgmi set 提供以下功能。

命令参数	命令描述
-h, --help	显示使用帮助信息
--host	指定主机 IP，以连接指定主机上 ix-hostengine, [default = localhost], 或指定 Unix 域套接字"unix://"
--logging-severity <tragetSeverity>	设置 logger 的输出级别，包括： NONE VERB DEBUG INFO WARN ERROR FATAL
--target-logger <tragetLogger>	设置 logger 目标，默认为 BASE，仅支持 BASE
--, --ignore_rest	忽略该标签以后所有的参数

用法和示例

ixdcgmi set 命令行的使用方法如下所示。

```
$ ixdcgmi set --host <IP/FQDN> --logging-severity <targetSeverity>
```

示例

设置 ix-hostengine 和 ixdcgmi 的 log 输出级别为 INFO。

```
$ ixdcgmi set --logging-severity INFO
```

9 Log 收集工具 ixLogCollect 使用指南

ixLogCollect 是天数智芯自研的一款脚本工具，旨在帮助用户一键收集系统 Log 以及 UMD/KMD 相关 Log，方便您进行问题排查。

9.1 环境要求

使用 ixLogCollect 工具，有如下环境要求：

- 操作系统为 Ubuntu 等 Linux 发行版
- Python v3.6 及以上
- 天数智算软件栈 v4.1.0 及以上

9.2 安装指南

直接将工具包传到您的环境中即可。请向您的应用工程师获取 ixLogCollect 工具包。

9.3 功能说明

ixLogCollect 工具支持如下功能：

- 收集系统 Log，包括 dmesg、lspci、ixsmi 等
- (可选) 当您执行 **source enable_umd**，工具增量收集 UMD 相关 Log
- (可选) 当您执行 **sudo bash enable_kmd**，工具增量收集 KMD 相关 Log

9.4 使用说明

您在 ixLogCollect 工具所在根目录，在 bash 中输入以下命令，工具即可按需收集 Log：

```
# 如您需要开启增量收集 UMD 相关 Log，执行以下命令
# source enable_umd

# 如您需要开启增量收集 KMD 相关 Log，必须以 root 权限执行以下命令
# sudo bash enable_kmd

# 运行您的应用

# 使用 ixLogCollect 工具收集 log
$ python main.py
```

9.5 系统 Log 收集范围及如何自定义

9.5.1 默认系统 Log 收集范围

ixLogCollect 工具会读取目录下的配置文件 configs/config.default.json，并根据其中的配置收集 Log。

默认情况下，ixLogCollect 会收集以下 Log：

- dmesg，对应配置文件中的节 contributes -> syscmd -> syscmd.dmesg.H，等效 **dmesg -H** 命令
- lspci，对应配置文件中的节 contributes -> syscmd -> syscmd.lspci.t，等效 **lspci -t** 命令
- ixcmd 相关，对应配置文件中的节 contributes -> ixcmd，默认采集以下三类信息：
 - ixsmi -q
 - ixsmi -q -d hardware_monitor
 - ixsmi topo -t

9.5.2 修改系统 Log 收集范围

如您需要自定义收集目标，可参考配置文件 configs/config.default.json 的格式，并修改 contributes 节下的内容即可。如需帮助，请联系天数智芯技术支持。

9.6 UMD 相关 Log 收集范围及如何自定义

收集 UMD 相关 Log，需要配置 UMD 相关的环境变量，执行 **source enable_umd** 命令即可配置 UMD 相关的环境变量。

如果需要关闭 UMD 相关的环境变量，执行 **source disable_umd** 命令即可。

9.6.1 默认 UMD 相关 Log 收集范围

enable_umd 文件中默认设置了以下 UMD 相关的环境变量：

- UMD_ENABLELOGGING：使能
- MD_MINIMUMLOGLEVEL：最小 Log 级别
- UMD_LOG_KERNELSTATISENABLE，UMD kernel statis 信息
- UMD_LOG_LEVEL：UMD Log 级别，可选项有 [ERROR, WARN, INFO, TRACE, CMDPACET, KERNEL]
- UMD_LOG_SUBSYS：UMD Log 子系统，可选项有 [CONTEXT, STREAM, EVENT, MODULE, MEMORY, GRAPH, VMM, ALL]
- UMD_FILEOUTPUTPATH：UMD Log 文件路径，默认脚本会将 Log 保存至 \$HOME/log/iluvatar_log_collect，与系统 Log 保持一致

UMD 环境变量开启后，会自动将生成的 Log 文件添加至 \$UMD_FILEOUTPUTPATH 目录中。如果更换目录，请先创建对应文件夹。

9.6.2 修改 UMD 相关 Log 收集范围

如您需要自定义环境变量参数 (在 `enable_umd` 文件中进行修改), 例如, 修改 UMD Log 级别, 可参考 `UMD_LOG_LEVEL` 环境变量的可选项并修改为所需级别, 修改完成后执行 **source enable_umd** 命令完成 UMD 相关环境变量的配置。

如需帮助, 请联系天数智芯技术支持。

Important

为保证 UMD Log 信息的完整性, 请勿随意删减环境变量!

9.7 KMD 相关 Log 收集范围

收集 KMD 相关 Log, 您需要以 root 权限执行 **sudo bash enable_kmd** 命令。配置成功后, KMD 相关 Log 信息会通过 `dmesg` 的方式输出, 与系统 Log 中的 **dmesg -H** 命令输出的文件内容一致。

如果需要关闭 KMD 的 Log 开关, 执行 **insmod kmd** 命令即可。

Important

为避免执行出错, 不建议自行修改 `enable_kmd` 文件!

9.8 使用说明

1. 设置 UMD 环境。

```
$ source enable_umd
```

2. 以 root 用户开启 KMD Log 开关。

```
$ sudo bash enable_kmd
```

3. 运行测试用例并保存失败日志, 示例如下:

```
$ ./vectorAdd > vec.log 2>&1
```

4. 运行脚本用于收集日志报告。

```
$ python3 main.py # This will save log to iluvatar_log_collect_xxxx.tar.gz in $HOME/
↪ log directory.
$ sudo bash bug_report/iluvatar-bug-report.sh # This saves log to
↪ iluvatar-log-xxxx.tar.gz in $HOME/log directory.
```

说明:

该脚本的输出是一个压缩文本文件, 形如 `iluvatar-log-xxxxxx.tar.gz` (其中, `xxxxxx` 是系统时间)。此文件中包含如下现场环境信息:

- 系统配置的基本信息, 包括 BIOS、主板、处理器、内存、缓存信息等
- 操作系统版本信息、GCC 版本、系统动态链接库版本、系统 NUMA 配置
- PCI 设备信息、连接拓扑

- 当前操作系统运行的状态信息，包括内存使用、中断统计、启动参数、模块信息等
- 操作系统运行的日志信息
- 驱动状态以及驱动日志信息

5. 向天数智芯应用工程师发送步骤 4 中收集到的 `iluvatar_log_collect_xxxx.tar.gz`、`iluvatar-log-xxxx.tar.gz` 与失败日志，帮助分析和排查问题。

Note

如果存在与 mmu page fault (XID 24) 相关的报错，且该测试程序确认不存在 out-of-bounds 或 UAF 问题，请开启 KMD 模块的 `mmu_trace` 开关。

10 内存检测工具 ixSAN 使用指南

10.1 ixSAN 概述

ixSAN (Iluvatar Sanitizer) 是天数智芯提供的一款用于检查 CUDA 应用程序功能正确性的工具集，包含多种检测工具，可以按需执行不同类型的检查，包括：

- memcheck 工具：用于精确检测和归因 CUDA 应用程序中的内存访问错误
- racecheck 工具：用于报告可能导致数据争用的共享内存数据访问危险
- initcheck 工具：用于报告 GPU 对全局内存进行未初始化访问的情况

本文档旨在介绍以上检查工具的使用方法。

10.2 ixSAN 使用方法

天数智算软件栈安装完成后，ixSAN 工具也会一并安装好，默认安装路径为 `/usr/local/corex-{v.r.m}/bin/`。

您可以通过运行可执行文件来调用 ixSAN 工具，运行命令如下：

```
$ ixsan [<options>] [application] [<application args>]
```

参考[命令行选项](#)了解 ixSAN 的支持选项及其默认值的完整列表。

10.2.1 命令行选项

ixSAN 选项的格式通常为 `--option value` (部分情况例外)。

选项	默认值/有效值	描述
<code>--help, -h</code>	无	打印 ixSAN 工具的帮助信息
<code>--version, -v</code>	无	打印 ixSAN 工具的版本信息
<code>--tool</code>	默认为 memcheck，可选 racecheck 和 initcheck	指定检测类型对 CUDA 应用程序进行检测

10.2.2 编译选项

ixSAN 工具支持使用 `-g` 或不带 `-g` 编译标志对 CUDA 应用程序进行编译。

10.2.3 环境变量

在启动 ixSAN 工具前，请按需设置如下环境变量。

环境变量	有效值	默认值	描述
CUDA_VISIBLE_DEVICES	0 ~ [device number]-1	0	指定某个编号的加速卡为主卡
SHOW_MEM_CKTYPE	1, 2, 4, 8	默认未设置	默认未设置，表示打印全部错误类型 该环境变量指定打印 memcheck 错误的类型： 1，表示只打印 invalid 类错误 2，表示只打印 out of bound 类错误 3，表示只打印 misaligned 类错误 4，表示只打印 used after free 类错误 仅支持设置一种错误类型 参考 内存访问错误 了解使用详情
SHOW_MEM_CKREL	0 ~ 任意正整数	0	指定打印 memcheck 打印错误的数量 参考 内存访问错误 了解使用详情
IXSAN_ONLY_LEAKCHECK_ENABLE	0, 1	0	设置为 1 时，只进行 memcheck 的 Device 端内存泄漏检测 参考 ixSAN 使用技巧 了解更多信息
IXSAN_BACKTRACE_LEVEL	0 ~ 任意正整数	默认未设置	指定输出错误堆栈信息层数 未设置时，默认输出最多 15 层的错误堆栈信息；设置为 0 时，输出全部的错误堆栈信息

10.3 memcheck 工具使用指南

memcheck 工具是一个运行时错误检测工具，主要针对 CUDA 应用程序。该工具可以精确地检测和报告越界以及对 CUDA 中全局和共享内存的内存未对齐访问。

此外，memcheck 工具还可以检测和报告用户应用程序中的 device 内存泄漏。

10.3.1 memcheck 支持的功能

memcheck 支持对以下四种内存访问错误进行检查，以及对 device 端内存泄漏进行检查：

- invalid 错误，表示内存非法读写
- out of bound 错误，表示内存访问越界
- misaligned 错误，表示内存未对齐访问
- used after free 错误，表示内存释放后再使用
- leak 错误，表示内存泄漏

10.3.2 memcheck 使用方法

当运行 ixSAN 工具时，memcheck 时默认的检查类型，您也可以通过使用 `--tool memcheck` 选项进行 memcheck 检查。以下是使用方法：

```
$ ixsan [application] [<application args>]
$ ixsan --tool memcheck [application] [<application args>]
```

10.3.3 memcheck 错误报告详述

memcheck 错误报告主要由以下 3 部分组成：

- 内存泄漏 报告
- 内存访问错误数量提示
- 内存访问错误 报告

10.3.3.1 内存泄漏

即申请的 device 内存未释放，所有的内存泄漏会全部进行打印。

下面是一个内存泄漏示例：

```
===== Leaked 200000 bytes at 0x7eb814200000
=====      Saved host backtrace up to driver entry point at allocation time
=====      Host Frame: cudaMalloc [0xf4636]
=====      in /home/ubuntu/sw_home/local/corex/lib64/libcudart.so.10.2
=====      Host Frame: main [0x1443]
=====      in /home/ubuntu/ixsan-test/build/bin/vectorAdd
=====      Host Frame: _start [0x11ad]
=====      in /home/ubuntu/ixsan-test/build/bin/vectorAdd
```

上述示例中，第一行显示泄漏内存的大小和地址，其余行则是 backtrace 信息。

10.3.3.2 内存访问错误数量提示

即显示多种错误的数量，并详细指明读/写错误。

下面是一个错误数量报告示例：

```
[mem_check]: ILLEGAL OPERATION (invalid 16 (r:15 w:1)) (out of bound 1 (r:0 w:1))
↳ (misaligned 0) (used after free 2) SUMMARY: 19 errors
```

10.3.3.3 内存访问错误

即 device 端内存访问错误，包括了 invalid, out of bound, misaligned 和 used after free 四种错误。此部分非必要打印，需要通过 SHOW_MEM_CKTYPE 和 SHOW_MEM_CKREL 环境变量来控制 (更多信息请参考[环境变量](#))。

其中，

- SHOW_MEM_CKTYPE 环境变量指定打印的错误类型，即设置 **export SHOW_MEM_CKTYPE=1/2/4/8** 表示分别规定了只打印 invalid 或 out of bound 或 misaligned 或 used after free 错误；不设置该环境变量，则四种类型的错误都会被打印
- SHOW_MEM_CKREL 环境变量规定了错误打印的数量，不设置该环境变量则默认为 0，最终的打印数量为 SHOW_MEM_CKREL 设定值与实际错误个数的最小值

下面是一个内存访问错误示例：

```
===== Invalid __global__ read of size 4 bytes
===== at 0x11610 in vectorAdd(float const*, float const*, float*, int)
===== by thread (198,0,0) in block (195,0,0)
===== Address 0x7eb814230f18 is out of bounds
===== and is 472 bytes before the nearest allocation at 0x7eb814200000 of size
↳ 200000 bytes
===== Saved host backtrace up to driver entry point at allocation time
===== Host Frame: cudaLaunchKernel [0xf4281]
===== in /home/ubuntu/sw_home/local/corex/lib64/libcudart.so.10.2
===== Host Frame: __device_stub__vectorAdd(float const*, float const*, float*,
↳ int) [0x1313]
===== in /home/ubuntu/ixsan-test/build/bin/vectorAdd
===== Host Frame: main [0x1633]
===== in /home/ubuntu/ixsan-test/build/bin/vectorAdd
===== Host Frame: _start [0x11ad]
===== in /home/ubuntu/ixsan-test/build/bin/vectorAdd
```

上述示例中，

- 第一行显示访问内存的类型、访问方式和大小
- 第二行包含 CUDA 内核名称和指令偏移量
- 第三行显示执行错误访问的线程信息
- 第四行显示访问的内存地址和访问错误的类型 (越界地址为 0x7eb814230f18)
- 第五行显示详细的越界信息 (原地址为 0x7eb814200000，大小为 200000 bytes，越界地址超出地址边界 472 bytes)

- 其余行则是 backtrace 信息

10.4 racecheck 工具使用指南

racecheck 工具是运行时共享内存数据访问风险的探测器。该工具的主要用途是帮助识别 CUDA 应用程序中内存访问使用共享内存的争用情况。

racecheck 工具可识别程序中的三种典型风险，包括：

- 写后写 (WAW)
- 读后写 (WAR)
- 写后读 (RAW)

racecheck 旨在识别这些风险并帮助用户编写没有共享 __shared__ 内存竞争的程序。

10.4.1 racecheck 支持的功能

通过 racecheck 工具检测到的危险可通过设置 **--racecheck-report** 命令来指定输出类型，有以下两种类型输出：

- hazard 模式：该模式提示了争用共享内存的每一个危险，指明了发生争用的线程的信息和共享内存的信息
- analysis 模式：该模式是一组后期分析报告，由 racecheck 工具通过分析所有危险报告而生成的

10.4.2 racecheck 使用方法

racecheck 不是 ixSAN 的默认错误检查类型，需要您通过使用 **--tool racecheck** 选项指定进行 racecheck 检查。

```
$ ixsan [application] [<application args>]
```

同时，您还可以通过 **--racecheck-report** 命令指定错误输出模式，不指定则默认为 analysis 模式，使用方法如下：

```
$ ixsan --tool racecheck --racecheck-report analysis [application] [<application args>]
```

```
$ ixsan --tool racecheck --racecheck-report hazard [application] [<application args>]
```

10.4.3 racecheck 错误报告详述

10.4.3.1 analysis 模式

即简略的危险分析报告。

下面是一个 analysis 模式的报告示例：

```

===== Warning: Race reported between Write access at 0x11178 in sumKernel(int*, int*)
=====   and Read access at 0x111d0 in sumKernel(int*, int*) [252 hazards]
=====
===== Error: Race reported between Read access at 0x111d0 in sumKernel(int*, int*)
=====   and Write access at 0x11178 in sumKernel(int*, int*) [256 hazards]

```

上述示例中的两行分别显示发生争用的内存访问类型、CUDA 内核名称与指令偏移量，以及危险数量，危险等级 Warning/Error。其中，Warning 为在同一个 warp 内的两个线程访问冲突危险，而 error 为同一个 block 而不同 warp 的线程访问冲突危险。

10.4.3.2 hazard 模式

即详细的危险报告。

下面是一个 hazard 模式的报告示例：

```

===== Error: Potential WAR hazard detected at __shared__ 0x1ff in block (0,0,0) :
=====   Read Thread (0,0,0) at 0x111d0 in sumKernel(int*, int*)
=====   Write Thread (127,0,0) at 0x11178 in sumKernel(int*, int*)
=====   Saved host backtrace up to driver entry point at allocation time
=====   Host Frame: cudaLaunchKernel [0xf4281]
=====   in /home/ubuntu/sw_home/local/corex/lib64/libcudart.so.10.2
=====   Host Frame: /home/ubuntu/sw_home/sdk/ixtools/ixsan/examples/racechecksample/
↳ block_error.cu:39:__device_stub__sumKernel(int*, int*) [0x1293]
=====   in /home/ubuntu/sw_home/sdk/ixtools/ixsan/build/bin/block_error
=====   Host Frame: /home/ubuntu/sw_home/sdk/ixtools/ixsan/examples/racechecksample/
↳ block_error.cu:208:main [0x17cc]
=====   in /home/ubuntu/sw_home/sdk/ixtools/ixsan/build/bin/block_error
=====   Host Frame: _start [0x114d]
=====   in /home/ubuntu/sw_home/sdk/ixtools/ixsan/build/bin/block_error

```

上述示例中，第一行显示危险等级 Warning/Error，还指明了危险类型 WAW/WAR/RAW，以及发生冲突的两个线程的 block 索引和内存地址；第二行和第三行则显示发生争用的内存访问类型、CUDA 内核名称与指令偏移量；其余行则为 backtrace 信息。

10.5 initcheck 工具使用指南

initcheck 工具是运行时未初始化的 device 全局内存访问检测器。该工具可以识别未经 device 端写入或通过 memcpy 和 cudaMemcpy 进行初始化而进行的访问。

10.5.1 initcheck 支持的功能

initcheck 工具目前仅支持对 device 端全局内存访问的检测。

10.5.2 initcheck 使用方法

initcheck 不是 ixSAN 的默认错误检查类型，需要您通过使用 **--tool initcheck** 选项指定进行 racecheck 检查。

```
$ ixsan --tool initcheck [application] [<application args>]
```

10.5.3 initcheck 错误报告详述

下面是一个 initcheck 错误报告示例：

```
===== Uninitialized __global__ memory read of size 4 bytes
=====      at 0x11538 in vectorAdd(int*)
=====      by thread (26,0,0) in block (0,0,0)
=====      Address 0x7f2c14200068
=====      Saved host backtrace up to driver entry point at allocation time
=====      Host Frame: cudalaunchKernel [0xf4281]
=====              in /home/ubuntu/sw_home/local/corex/lib64/libcudart.so.10.2
=====      Host Frame: /home/ubuntu/ixsan-test/initcheck/memset_error.cu:
↳ 12: __device_stub__vectorAdd(int*) [0x1294]
=====              in /home/ubuntu/ixsan-test/build/bin/memset_error
=====      Host Frame: /home/ubuntu/ixsan-test/initcheck/memset_error.cu:26:main
↳ [0x13a1]
=====              in /home/ubuntu/ixsan-test/build/bin/memset_error
=====      Host Frame: _start [0x115d]
=====              in /home/ubuntu/ixsan-test/build/bin/memset_error
```

在上述示例中，

- 第一行显示访问(读)未初始化内存的大小
- 第二行包含 CUDA 内核名称和指令偏移量
- 第三行显示执行未初始化访问的线程信息
- 第四行显示未初始化访问的内存地址
- 其余行则是 backtrace 信息

10.6 ixSAN 使用技巧

- ixSAN 可以检测应用程序的内存泄漏和 CUDA Kernel 的内存越界、对齐、未初始化等问题。
- 当只检查应用程序内存泄漏时，您需要设置环境变量 **export IXSAN_ONLY_LEAKCHECK_ENABLE=1**，否则检查时间会较长。
- 当需要分析 CUDA Kernel 的内存越界、对齐、未初始化等问题时，无需设置 IXSAN_ONLY_LEAKCHECK_ENABLE 环境变量。此时检测时间较长，具体检测时长和调用 Kernel 的数量与测试规模有关，**请尽量缩减测试规模和调用 Kernel 的次数。**
- 当分析 CUDA Kernel 的内存越界、对齐、未初始化等问题时，ixSAN 的检测时间可能会较长。此时如果使用 **Ctrl+C** 中断了检测，则需要执行 **ixsmi -r** 命令重置 GPU。

11 二进制文件分析工具 ixobjdump 使用指南

11.1 ixobjdump 概述

ixobjdump 是天数智芯提供的一款二进制文件分析工具，主要从 CUDA 二进制文件中提取信息，并以可读性较强的格式呈现给用户。

ixobjdump 的输出内容包括每个内核的 CUDA 汇编代码，CUDA ELF 部分头文件，字符串表，重定位器和其它 CUDA 特定部分；ixobjdump 另外还会从主机二进制文件中提取嵌入式 IR 文本。

有关天数智芯 ivcore11 芯片架构的 CUDA 伪汇编指令详情，请参考[伪汇编指令集](#)。

11.2 ixobjdump 使用方法

- ixobjdump 在每次的分析过程中仅支持接受一个输入文件。基本用法如下：

```
$ ixobjdump [options] <file>
```

- 需要反汇编一个或多个嵌入在主机中的可执行文件中的独立库，并显示内核的 CUDA 汇编，可以使用如下命令：

```
$ ixobjdump --sass <file>
```

- 需要从一个 cubin 文件中以可读的格式转储 CUDA ELF 部分，可以使用如下命令：

```
$ ixobjdump --elf <file>
```

- 从主机二进制文件中提取 IR 文本，可以使用如下命令：

```
$ ixobjdump --IR <file>
```

下面是使用 ixobjdump 工具的输出示例：

```
$ ixobjdump vectorAdd --IR
```

```
Fatbin LLVM IR code:
```

```
=====
```

```
arch = ivcore10
```

```
host = Linux
```

```
compile_size = 64bit
```

```
; ModuleID = 'tmpLLVM.bc'
```

```
source_filename = "/home/corex/sw_home/sdk/ixtools/ixkn/samples/vectorAdd/
```

```
↪ vectorAdd.cu"
```

```
target datalayout = "e-p:64:64-p1:64:64-p2:32:32-p3:32:32-p4:64:64-p5:32:32-p6:32:32-
```

```
↪ i64:64-v16:16-v24:32-v32:32-v48:64-v96:128-v192:256-v256:256-v512:512-
```

```
↪ v1024:1024-v2048:2048-n32:64-S32-A5"
```

```
target triple = "bi-iluvatar-ilurt"
```

```
@gSyncThreadsFlag = dso_local local_unnamed_addr @drspace(3) global i32 undef, align
```

```
↪ 4, !dbg !0
```

```
@gPrintBuf = dso_local addrspace(1) externally_initialized global ptr null, align 8
@gAssertReason = dso_local addrspace(1) externally_initialized global ptr null, align
↪ 8
@gMallocHeapBase = addrspace(1) externally_initialized global ptr null, align 8
@gMallocHeapSize = addrspace(1) externally_initialized global i32 0, align 4
@llvm.compiler.used = appending global [4 x ptr] [ptr addrspacecast (ptr addrspace(1)
↪ @gAssertReason to ptr), ptr addrspacecast (ptr addrspace(1) @gMallocHeapBase to
↪ ptr), ptr addrspacecast (ptr addrspace(1) @gMallocHeapSize to ptr), ptr
↪ addrspacecast (ptr addrspace(1) @gPrintBuf to ptr)], section "llvm.metadata"
; Function Attrs: mustprogress norecurse nosync nounwind willreturn
↪ memory(argmem: readwrite, inaccessiblemem: write)
define dso_local iluvatar_kernel void @_Z9vectorAddPKfS0_Pfi(ptr addrspace(1)
↪ nocapture noundef readonly %A.coerce, ptr addrspace(1) nocapture noundef readonly
↪ %B.coerce, ptr addrspace(1) nocapture noundef writeonly %C.coerce, i32 noundef
↪ %numElements) local_unnamed_addr #0 !dbg !969 {
entry:
tail call void @llvm.dbg.value(metadata ptr poison, metadata !976, metadata !
↪ DIExpression()), !dbg !981
tail call void @llvm.dbg.value(metadata ptr poison, metadata !977, metadata !
↪ DIExpression()), !dbg !981

.....
```

```
$ ixobjdump vectorAdd --sass
```

```
Arch:BI_OS:Linux
Function : _Z9vectorAddPKfS0_Pfi
/home/corex/sw_home/sdk/ixtools/ixkn/samples/vectorAdd/vectorAdd.cu:32
BRP.ENT // 00000000115C0: 00000E23 000000FF
MOV 8, S102, S0 // 00000000115C8: 08336C2B 00000000
/home/corex/sw_home/sdk/ixtools/ixkn/samples/vectorAdd/vectorAdd.cu:33
MOV V0, V255, V0 // 00000000115D0: 007FEC0B 00000000
/home/corex/sw_home/local/corex/lib/clang/18/
↪ include/_clang_cuda_builtin_vars.h:92
LD.B32x8 s0, s[4:5], noreg, #0, ca // 00000000115D8: 000039BB 0000027F
WAIT.SMEM.0 // 00000000115E0: 020001A3 00000000
LOP.AND S7, S103, 0xffff // 00000000115E8: 0733A46B 00000FFF
CSEL S7, S7, 0x1000 // 00000000115F0: 0703EDEB 00001000
SHF.R.U32 S9, S103, 0xc // 00000000115F8: 0933AC6B 0000000C
LOP.AND S9, S9, 0xffff // 0000000011600: 0904A46B 00000FFF
SHF.R.U32 S9, S103, 0x18 // 0000000011608: 0933AC6B 00000018
/home/corex/sw_home/sdk/ixtools/ixkn/samples/vectorAdd/vectorAdd.cu:33
IMUL S7, S7, S28 // 0000000011610: 0703B4AB 0000001C
IADD S8, S7, S102 // 0000000011618: 0803842B 00000066
IADD.I32 V0, V8, V0 // 0000000011620: 0004040B 00000000
/home/corex/sw_home/sdk/ixtools/ixkn/samples/vectorAdd/vectorAdd.cu:35
ISET.GT TCR V6, V0 // 0000000011628: 00032413 00000000
AND_SAVETMSK S[6:7], tcr, tmsk // 0000000011630: 063321AB 0010007E
00000000000011638 <BB0_1>:
```

```
; BB0_1():
/home/corex/sw_home/sdk/ixtools/ixkn/samples/vectorAdd/vectorAdd.cu:37
SHF.R.I32 S9, S8, 0x1f // 0000000011638: 09042CEB 0000001F
SHF.L.U64 S[12:13], S[8:9], S[2:3] // 0000000011640: 0C042F6B 00000002

.....
```

从上述示例输出中可以看到，vectorAdd CUDA 二进制文件包含天数智芯芯片的 cubin 和其 IR 代码。

- 需要在 CUDA 二进制文件中列出 cubin 文件，可以使用 **--elf** 选项：

```
$ ixobjdump vectorAdd --elf

ELF file 1 : tmpxft_0000448e_00000000-1.BI_Linux.ixbin
ELF file 2 : tmpxft_0000aa05_00000000-2.MR_Linux.ixbin
ELF file 3 : tmpxft_0000e0e3_00000000-3.QS_Linux.ixbin
```

- 使用 **--xelf all** 选项从 CUDA 二进制文件中提取所有子类文件：

```
$ ixobjdump vectorAdd --xelf all

Extracting ELF file IXBIN0.ixbin
Extracting ELF file IXBIN1.ixbin
Extracting ELF file IXBIN2.ixbin
```

11.3 ixobjdump 命令选项

下表列出了 ixobjdump 支持的命令选项以及每个选项对应的功能描述。其中，每个选项都包含一个长名称和一个短名称，二者可以互换使用。

命令 (长)	命令 (短)	描述
--all-fatbin	--all	列出 fatbin 中所有的段，默认情况下列出 CUDA 二进制文件中包含可在设备上执行 fatbin (如果存在的话) 中所有的内容 如果不包含有效的 fatbin 文件，则给出相应的提示
--dump-elf	--elf	列出 fatbin 中 elf 文件的所有 section 的基本信息
--dump-elf-symbols	--symbols	列出 fatbin 中 elf 文件的 symbol section 段中所包含的所有 symbol 信息
--dump-IR	--IR	列出 fatbin 中包含的所有平台的 IR 文件内容
--dump-sass	--sass	列出 fatbin 中包含的 elf 可执行文件的设备伪码或汇编伪码

命令 (长)	命令 (短)	描述
<code>--extract-elf TEXT</code>	<code>--xelf</code>	提取出 CUDA 二进制文件中的 fatbin 所包含的 elf 文件，每个 elf 文件单独保存
<code>--extract-IR TEXT</code>	<code>--xIR</code>	提取出 CUDA 二进制文件中的 fatbin 所包含的 elf 文件的“.text” 段的内容，以二进制格式保存为单独的文件
<code>--function TEXT</code>	<code>--fun</code>	在使用 --sass 命令时指定需要被列出的设备函数名称，该命令必须是经过 C++ 转换后的函数命令，例如： ixobjdump --fun _Z9vectorAddPKfs0_Pfi --sass vectorAdd
<code>--function-index</code>	<code>--findex</code>	在使用 --sass 命令时指定需要被列出的设备函数的序列号 (elf 文件中包含多个设备函数情况下)，该顺序按照 symbol section 中的 symbol 顺序排列。如果超过有效的序列，则给出相应提示 例如： ixobjdump --findex 1 --sass matrixMul
<code>--gpu-architecture TEXT</code>	<code>--arch</code>	仅展示某个平台上的内容，目前有效的平台名称包括：“ivcore10”, “ivcore11”
<code>--list-elf</code>	<code>--lelf</code>	列出 CUDA 二进制文件中的 fatbin 所包含的 elf 文件，目标对象可以是主机上的可执行程序，也可以是编译过程的目标文件以及最终的库文件 如果使用此命令，其它命令会被忽略
<code>--list-IR</code>	<code>--IIR</code>	列出 CUDA 二进制文件中的 fatbin 所包含的 IR 文件 如果使用此命令，其它命令会被忽略
<code>--list-text</code>	<code>--ltext</code>	列出 CUDA 二进制文件中的 fatbin 中 elf 文件中所包含的有效设备函数的名称 如果使用此命令，其它命令会被忽略
<code>--options-file TEXT</code>	<code>--optf</code>	指定一个选项文件，ixobjdump 可根据最终解析出的文件内容执行命令
<code>--version</code>	<code>-V</code>	打印版本信息以及编译时间等
<code>--help</code>	<code>-h</code>	打印帮助信息

11.4 伪汇编指令集

本小节提供的信息包含天数智芯芯片架构的伪汇编指令集。

天数智芯 ivcore10 和 ivcore11 芯片架构的伪汇编指令按照以下格式构成：

```
(instruction) (destination) (source1), (source2) ...,SR..
```

有效的目标和源寄存器包括：

- SRF
- VRF
- 0xFF 等立即数
- 特殊寄存器

天数智芯 ivcore11 芯片架构的伪汇编指令集如下表所列。

操作码	描述
TRUNCF	获取浮点型数据的整数部分
SFMAX	获取 VRF[src0] 和位于 addr 地址中的最大值
HADD2	执行 FP16 类型数据的加法操作
ST	将数据存储至通用寄存器中
EXIT	结束并退出程序
IMUL	执行整数的乘法操作
SCDEC	获取 VRF[src0] 和位于 SLB[addr] 地址中的最小值
BRP	进入或退出断点
SCINC	当 S[addr] 大于等于 V[src0] 的值，返回 0，否则返回 S[addr]+1
SIADD	执行 V[src0] 和 SLB 上的加法操作
IADD	执行整形操作数的加法操作
JMX	执行绝对跳转
JMP	执行绝对跳转
BREVP	执行反转比特位操作
LDS	执行 SLB 上的加载操作
I2F	将整形操作数转换成浮点数
BAR	执行同步等待操作
FFMA	执行 FP32 类型数据的乘加操作
HMNMx2	获取 FP16 类型数据的最大值或最小值
F2I	将浮点型数据值转换成整型数据
SMLD4x1	从全局内存中加载数据到 SME 中
IABS	获取整形数据的绝对值

操作码	描述
IDP	执行整形数据的点乘和累加操作
I2I	将整形数据转换成整型数据
SHF	执行桶形移位操作
HMUL2	执行 FP16 类型数据的乘法操作
HFMA2	执行 FP16 类型数据的融合乘法操作
BREV	执行反转比特位操作
LGC	执行比特位的逻辑操作实现提出或插入等操作
LD	从全局内容中加载数据到指定位置
FLO	找出数据中的前导 1 的位置
FADD	执行 FP32 类型数据的加法操作
SSWAP	执行 SLB 的数据交换操作
PCSWAP	执行将 PC 的值和 SRF[src] 的操作
SMLD1x16	从全局内存中加载数据至 SME 中
PRMT	执行字节置换操作
SFADD	SLB 上执行浮点数的加操作
FDP	执行浮点数的点成和累加操作
BLD	将全局内存中的数据加载至 VRF 中
SMST	执行将 SME 中的数据存储至全局内存中
DEPBAR	执行依赖等待操作
SLGC	在 SLB 上执行逻辑操作
SIMAX	获取 VRF[src0] 和 SLB[addr] 上的最大值
LEPC	重置 PC 值
POPC	计算源数据中 1 的或 0 的数量
F2F	将浮点型数据转换成浮点型数据
ATOM	在全局内存上执行原子操作
BST	将 VRF 上的数据存储至全局内存中
HMMA	执行浮点矩阵乘法，并将结果累加至浮点矩阵中
SCSWAP	执行数据交换操作
FMUL	执行 FP32 的乘法操作

操作码	描述
FMNMX	获取浮点数的最大值或最小值
MOV	执行移动操作
ROUNDf	对浮点型数据执行取整操作
IMAD	执行整形数据的乘加操作
TIME	读取位于时间寄存器上的值
ISSET	执行整型数逻辑操作
IMNMx	获取整型数据的最大值或最小值
SIMIN	获取 VRF[src0] 和 SLB[addr] 上的最小值
NANOSLEEP	暂停执行指令
CR2S	获取硬件寄存器上的值保存至 SRF 中
MUFU	执行 FP32 类型数据的超越函数
BRX	执行相对跳转指令
SFMIN	获取 VRF[src0] 和 SLB[addr] 上的最小值
LOP3	执行逻辑操作运算
SMLD8x1	从全局内存中加载数据至 SME 中
IADDC	执行无符号整型带有进位且支持饱和运算的加法操作
BRA	执行相对跳转指令
HADD2x2	执行 FP16 类型的包加法运算
LOP	执行逻辑操作运算
BSTS	将 VRF 中的数据存储至 SLB 中
SMLD16x1	从全局内存中加载数据至 SME 中
STS	将数据存储至共享内存中
MEMWBINV	执行缓存写回和失效操作
BLDS	将 SLB 上的数据存储至 VRF 中
CSEL	根据 TRC 状态获取数据存储至目标寄存器
SPERMUTE	在 SLB 上执行重新排序操作
FEO	从低位开始, 找到第一个是 1 的位置
FRACT	获取浮点数据的小数部分
IMMA	执行整形数据的矩阵的乘加操作

操作码	描述
HMMNX2	执行 FP16 类型数据的包数据比较
VINT	初始化 VRF 寄存器
SMLD1x8	从全局内存中加载数据至 SME 中
IDP4A	执行整型数据的点乘和累加操作
CS2R	将数据存储至硬件状态寄存器中
FABS	获取浮点型数据的绝对值
RPCMOV	将当前 PC 值保存至其它寄存器
SENDMSG	向其它部分发送消息

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。